

JAVA 17 OCP · YDS STUDY SERIES

Unit 11 · Exceptions and Localization · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) içindeki ilgili chapter gövdesini ve Appendix resmî cevaplarını kaynak sırasını koruyan English → Türkçe paragraf çiftleriyle bir araya getirir. Kod ve terminal çıktıları çevrilmeden, bir kez ve kaynak konumunda gösterilir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak kapsam manifesti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 11 · Exceptions and Localization
- Chapter PDF sayfaları: 591–660
- Appendix cevap sayfaları: 945–948
- Beklenen sayfa marker'ı: 70
- Beklenen resmî cevap: 26
- Eşleme biçimi: English paragraf → Türkçe çeviri → varsa kod

İçindekiler

- 1 [Understanding Exceptions](#)
- 2 [Recognizing Exception Classes](#)
- 3 [Handling Exceptions](#)
- 4 [Automating Resource Management](#)
- 5 [Formatting Values](#)
- 6 [Supporting Internationalization and Localization](#)
- 7 [Summary](#)
- 8 [Exam Essentials](#)
- 9 [Review Questions](#)
- 10 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 11 · Exceptions and Localization · Eksiksiz çift dilli kaynak

Chapter 11 · Exceptions and Localization

English: OCP exam objectives covered in this chapter: Handling Exceptions and Implementing Localization. **Türkçe:** Bu bölümde kapsanan OCP sınav hedefleri: exceptions'ı ele alma ve localization uygulama. **English:** Handle exceptions using try/catch/finally, try-with-resources, and multi-catch blocks, including custom exceptions. **Türkçe:** Özel istisnalar da dahil olmak üzere try/catch/finally, try-with-resources ve multi-catch blocks kullanarak istisnaları ele alın. **English:** Implement localization using locales, resource bundles, parse and format messages, dates, times, and numbers including currency and percentage values. **Türkçe:** Locales ve resource bundles kullanarak localization uygulayın; currency ve percentage değerleri dâhil mesajları, tarihleri, saatleri ve sayıları parse edip formatlayın.

English: This chapter is about creating applications that adapt to change. What happens if a user enters invalid data on a web page? What if our connection to a database goes down in the middle of a sale? Finally, how do we build applications that can support multiple languages or geographic regions? **Türkçe:** Bu bölüm değişime uyum sağlayan uygulamalar yaratmakla ilgilidir. Bir kullanıcı bir web sayfasına geçersiz veri girerse ne olur? Ya bir veri tabanıya olan bağlantımız bir satışın ortasında düşerse? Son olarak, birden fazla dili veya coğrafi bölgeyi destekleyebilecek uygulamaları nasıl inşa edeceğiz? **English:** In this chapter, we discuss these problems and solutions to them using exceptions, formatting, and localization. One way to make sure your applications respond to change is to build in support early on. For example, supporting localization doesn't mean you actually need to support specific languages right away. It just means your application can be more easily adapted in the future. By the end of this chapter, we hope we've provided structure for designing applications that better adapt to change. **Türkçe:** Bu bölümde, bu sorunları ve çözümleri istisnalar, biçimlendirme ve yerelleştirme kullanarak tartışıyoruz. Uygulamalarınızın değişime yanıt vermesini sağlamanın bir yolu, erken destek oluşturmaktır. Örneğin, yerelleştirmeyi desteklemek, aslında hemen belirli dilleri desteklemeniz gerektiği anlamına gelmez. Bu sadece uygulamanızın gelecekte daha kolay uyarlanabileceği anlamına gelir. Bu bölümün sonunda, değişime daha iyi uyum sağlayan uygulamalar tasarlamak için yapı sağladığımızı umuyoruz.

Understanding Exceptions

English: A program can fail for just about any reason. Here are just a few possibilities: ■ ■ The code tries to connect to a website, but the Internet connection is down. ■ ■ You made a coding mistake and tried to access an invalid index in an array. ■ ■ One method calls another with a value that the method doesn't support. **Türkçe:** Bir program hemen hemen her nedenle başarısız olabilir. İşte sadece birkaç olasılık: Kod bir web sitesine bağlanmaya çalışır, ancak İnternet bağlantısı kapalıdır. Kodlama hatası yaptınız ve bir dizideki geçersiz bir indekse erişmeye çalıştınız. Bir yöntem, yöntemin desteklemediği bir değerle diğerini çağırır. **English:** As you can see, some of these are coding mistakes. Others are completely beyond your control. Your program can't help it if the Internet connection goes down. What it can do is deal with the situation. **Türkçe:** Gördüğünüz gibi bunlardan bazıları kodlama hatalarıdır. Diğerleri tamamen kontrolünüz dışındadır. Programınız İnternet bağlantısı kesilirse yardımcı olamaz. Yapabileceği şey durumla başa çıkmaktır.

The Role of Exceptions

English: An exception is Java's way of saying, "I give up. I don't know what to do right now. You deal with it." When you write a method, you can either deal with the exception or make it the calling code's problem.

Türkçe: Bir exception, Java'nın "Pes ediyorum. Şu anda ne yapacağımı bilmiyorum; bununla sen ilgilen." deme biçimidir. Bir method yazdığınızda exception'ı kendiniz ele alabilir veya sorumluluğu calling code'a bırakabilirsiniz.

English: As an example, think of Java as a child who visits the zoo. The happy path is when nothing goes wrong. The child continues to look at the animals until the program ends nicely.

Nothing went wrong, and there were no exceptions to deal with. **Türkçe:** Örnek olarak, hayvanat bahçesini ziyaret eden bir çocuk olarak Java düşünün. Mutlu yol, hiçbir şeyin yanlış gitmediği zamandır.

Çocuk, program güzelce bitene kadar hayvanlara bakmaya devam eder. Hiçbir şey ters gitmedi ve başa çıkılması gereken bir istisna yoktu.

English: This child's younger sister doesn't experience the happy path. In all the excitement, she trips and falls. Luckily, it isn't a bad fall. The little girl gets up and proceeds to look at more animals. She has handled the issue all by herself. Unfortunately, she falls again later in the day and starts crying. This time, she has declared that she needs help by crying. The story

Türkçe: Bu çocuğun küçük kız kardeşi mutlu yolu yaşamaz. Tüm bu heyecan içinde, o atlar ve düşer. Neyse ki kötü bir düşüş değil. Küçük kız ayağa kalkar ve daha fazla hayvana bakmaya başlar. Konuyu tek başına ele aldı. Ne yazık ki, günün ilerleyen saatlerinde tekrar düşer ve ağlamaya başlar. Bu sefer ağlayarak yardıma ihtiyacı olduğunu açıkladı. Hikaye

English: ends well. Her daddy rubs her knee and gives her a hug. Then they go back to seeing more animals and enjoy the rest of the day. **Türkçe:** iyi biter. Babası dizini ovuşturur ve ona sarılır. Daha sonra daha fazla hayvan görmeye ve günün geri kalanında eğlenmeye geri dönerler.

English: These are the two approaches Java uses when dealing with exceptions. A method can handle the exception case itself or make it the caller's responsibility. **Türkçe:** Bunlar, Java istisnalarla uğraşırken kullandığı iki yaklaşımdır. Bir yöntem istisna davasının kendisi ile başa çıkabilir veya onu arayanın sorumluluğu haline getirebilir.

English: Return Codes vs. Exceptions Exceptions are used when "something goes wrong." However, the word wrong is subjective. The following code returns -1 instead of throwing an exception if no match is found:

Türkçe: İade Kodları vs. İstisnalar İstisnalar "bir şey yanlış gittiğinde" kullanılır. Ancak, yanlış kelime öznelidir. Aşağıdaki kod, eşleşme bulunmazsa bir istisna atmak yerine -1 döndürür:

```
public int indexOf(String[] names, String name) {
    for (int i = 0; i < names.length; i++) {
        if (names[i].equals(name)) { return i; }
    }
    return - 1;
}
```

English: While common for certain tasks like searching, return codes should generally be avoided. After all, Java provided an exception framework, so you should use it! **Türkçe:** Arama gibi belirli görevler için yaygın olsa da, geri dönüş kodlarından genellikle kaçınılmalıdır. Sonuçta, Java bir istisna çerçevesi sağladı, bu yüzden kullanmalısınız!

Understanding Exception Types

English: An exception is an event that alters program flow. Java has a Throwable class for all objects that represent these events. Not all of them have the word exception in their class name, which can be confusing. Figure 11.1 shows the key subclasses of Throwable. **Türkçe:** Bir istisna, program akışını değiştiren bir olaydır. Java, bu olayları temsil eden tüm nesneler için bir Throwable sınıfına sahiptir. Hepsinin sınıf isimlerinde istisna kelimesi yoktur, bu da kafa karıştırıcı olabilir. Şekil 11.1, Throwable anahtar alt sınıflarını gösterir. **English:** FIGURE 11.1 Categories of exception java.lang.Throwable **Türkçe:** FIGURE 11.1 İstisna kategorileri java.lang.Throwable

```
java.lang.Exception
java.lang.Error
```

English: Checked **Türkçe:** Kontrol edildi

```
java.lang.RuntimeException
```

English: Unchecked **Türkçe:** Kontrolsüz

Checked Exceptions

English: A checked exception is an exception that must be declared or handled by the application code where it is thrown. In Java, checked exceptions all inherit Exception but not RuntimeException. Checked exceptions tend to be more anticipated—for example, trying to read a file that doesn't exist. **Türkçe:** checked exception, atıldığı uygulama kodu tarafından ilan edilmesi veya ele alınması gereken bir istisnadır. Java içinde checked exceptions hepsi Exception varisli ama RuntimeException değil. Checked exceptions daha fazla öngörülme eğilimindedir – örneğin, var olmayan bir dosyayı okumaya çalışmak.

English: Checked exceptions also include any class that inherits Throwable but not Error or RuntimeException, such as a class that directly extends Throwable. For the exam, you just need to know about checked exceptions that extend Exception. **Türkçe:** Checked exceptions ayrıca Throwable varisli olan ancak Error veya RuntimeException olmayan herhangi bir sınıfı da içerir, örneğin Throwable doğrudan genişleyen bir sınıf gibi. Sınav için, sadece Exception uzatan kontrol edilmiş exceptions hakkında bilgi sahibi olmanız gerekir. **English:** Checked exceptions? What are we checking? Java has a rule called the handle or declare rule. The handle or declare rule means that all checked exceptions that could be thrown within a method are either wrapped in compatible try and catch blocks or declared in the method signature. **Türkçe:** Checked exceptions? Neyi kontrol ediyoruz? Java handle or declare rule adında bir kurala sahiptir. handle or declare rule, bir yöntem içinde atılabilecek tüm checked exceptions uyumlu try ve catch blocks ile sarılır veya method signature ile ilan edilir. **English:** Because checked exceptions tend to be anticipated, Java enforces the rule that the programmer must do something to show that the exception was thought about. Maybe it was handled in the method. Or maybe the method declares that it can't handle the exception and someone else should. **Türkçe:** checked exceptions öngörülme eğiliminde olduğundan, Java, programcının istisnanın düşünüldüğünü göstermek için bir şeyler yapması gerektiği kuralını uygular. Belki de yöntemde ele alınmıştır. Ya da belki de yöntem, istisnayı kaldıramayacağını ve bir başkasının yapması gerektiğini beyan eder. **English:** Let's take a look at an example. The following fall() method declares that it might throw an IOException, which is a checked exception: **Türkçe:** Bir örneğe bakalım. Aşağıdaki fall() yöntemi, checked exception olan bir IOException atabileceğini beyan eder:

```
void fall(int distance) throws IOException {  
    if(distance > 10) {  
        throw new IOException();  
    }  
}
```

English: Notice that you're using two different keywords here. The throw keyword tells Java that you want to throw an Exception, while the throws keyword simply declares that the method might throw an Exception. It also might not. **Türkçe:** Burada iki farklı anahtar kelime kullandığınıza dikkat edin. Atma anahtarı Java'ye bir Exception atmak istediğinizi belirtirken, atma anahtarı basitçe yöntemin Exception atabileceğini belirtir. Aynı zamanda olmayabilir. **English:** Now that you know how to declare an exception, how do you handle it? The following alternate version of the fall() method handles the exception: **Türkçe:** Artık bir istisnayı nasıl ilan edeceğinizi bildiğinize göre, bunu nasıl hallediyorsunuz? fall() yönteminin aşağıdaki alternatif sürümü istisnayı ele alır:

```
void fall(int distance) {  
    try {  
        if(distance > 10) {  
            throw new IOException();  
        }  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

English: Notice that the catch statement uses Exception, not IOException. Since IOException is a subclass of Exception, the catch block is allowed to catch it. We cover try and catch blocks in more detail later in this chapter. **Türkçe:** Yakalama ifadesinin IOException değil Exception kullandığına dikkat edin. IOException Exception alt sınıfı olduğundan, catch block alt sınıfının onu yakalamasına izin verilir. Bu bölümde daha sonra catch blocks'i daha ayrıntılı olarak ele alıyoruz.

Unchecked Exceptions

English: An unchecked exception is any exception that does not need to be declared or handled by the application code where it is thrown. Unchecked exceptions are often referred to as runtime exceptions, although in Java, unchecked exceptions include any class that inherits RuntimeException or Error.

Türkçe: unchecked exception, atıldığı uygulama kodu tarafından ilan edilmesi veya ele alınması gerekmeyen herhangi bir istisnadır. Unchecked exceptions genellikle runtime exceptions olarak adlandırılır, ancak Java, unchecked exceptions RuntimeException veya Error miras alan herhangi bir sınıfı içerir.

English: It is permissible to handle or declare an unchecked exception. That said, it is better to document the unchecked exceptions callers should know about in a Javadoc comment rather than declaring an unchecked exception.

Türkçe: unchecked exception ile başa çıkmak veya beyan etmek yasaktır. Bununla birlikte, unchecked exceptions arayanların bir unchecked exception ilan etmek yerine bir Javadoc yorumunda bilmesi gerekenleri belgelemek daha iyidir.

English: A runtime exception is defined as the RuntimeException class and its subclasses. Runtime exceptions tend to be unexpected but not necessarily fatal. For example, accessing an invalid array index is unexpected. Even though they do inherit the Exception class, they are not checked exceptions.

Türkçe: Bir runtime istisnası RuntimeException sınıfı ve alt sınıfları olarak tanımlanır. Runtime exceptions beklenmedik olma eğilimindedir, ancak mutlaka ölümcül değildir. Örneğin, geçersiz bir dizi dizinine erişmek beklenmediktir. Exception sınıfını miras alsalar da checked exceptions sınıfı değildirler.

English: An unchecked exception can occur on nearly any line of code, as it is not required to be handled or declared. For example, a NullPointerException can be thrown in the body of the following method if the input reference is null:

Türkçe: Bir unchecked exception neredeyse her kod satırında oluşabilir, çünkü işlenmesi veya ilan edilmesi gerekmez. Örneğin, girdi referansı null ise aşağıdaki yöntemin gövdesine bir NullPointerException atılabilir:

```
void fall(String input) {  
    System.out.println(input.toLowerCase());  
}
```

English: We work with objects in Java so frequently that a NullPointerException can happen almost anywhere. If you had to declare unchecked exceptions everywhere, every single method would have that clutter! The code will compile if you declare an unchecked exception. However, it is redundant.

Türkçe: Java'deki nesnelerle o kadar sık çalışıyoruz ki, bir NullPointerException hemen hemen her yerde olabilir. Her yerde unchecked exceptions ilan etmek zorunda olsaydınız, her bir yöntemde bu dağınıklık olurdu! Bir unchecked exception beyan ederseniz kod derlenecektir. Ancak, gereksizdir.

Error and Throwable

English: Error means something went so horribly wrong that your program should not attempt to recover from it. For example, the disk drive “disappeared” or the program ran out of memory. These are abnormal conditions that you aren’t likely to encounter and cannot recover from. **Türkçe:** Error, bir şeyin o kadar korkunç bir şekilde yanlış gittiği anlamına gelir ki, programınız ondan kurtulmaya çalışmamalıdır. Örneğin, disk sürücüsü "ortadan kayboldu" veya program bellekten çıktı. Bunlar, karşılaşma ihtimaliniz olmayan ve iyileşemeyeceğiniz anormal durumlardır. **English:** For the exam, the only thing you need to know about Throwable is that it’s the parent class of all exceptions, including the Error class. While you can handle Throwable and Error exceptions, it is not recommended you do so in your application code. When we refer to exceptions in this chapter, we generally mean any class that inherits Throwable, although we are almost always working with the Exception class or subclasses of it. **Türkçe:** Sınav için, Throwable hakkında bilmeniz gereken tek şey, Error sınıfı da dahil olmak üzere tüm istisnaların ana sınıfı olmasıdır. Throwable ve Error istisnalarını ele alabilirken, uygulama kodunuzda bunu yapmanız önerilmez. Bu bölümdeki istisnalara değindiğimizde, genellikle Throwable miras alan herhangi bir sınıfı kastediyoruz, ancak hemen hemen her zaman Exception sınıfı veya alt sınıflarıyla çalışıyoruz.

Reviewing Exception Types

English: Be sure to closely study everything in Table 11.1. For the exam, remember that a Throwable is either an Exception or an Error. You should not catch Throwable directly in your code. **Türkçe:** Tablo 11.1'deki her şeyi yakından incelediğinizden emin olun. Sınav için, bir Throwable Exception veya Error olduğunu unutmayın. Throwable kodunuzu doğrudan yakalamamalısınız.

English: TABLE 11.1 Types of exceptions and errors Okay for Is program required to Type How to recognize program to catch? handle or declare? **Türkçe:** TABLE 11.1 İstisnaların ve hataların türleri Tamam, Programın nasıl yakalanacağını nasıl tanıyacağımızı yazmak için gerekli mi? Tut veya ilan et?

English: Unchecked Subclass of RuntimeException Yes No exception Checked Yes Yes Subclass of Exception but not exception subclass of RuntimeException Error Subclass of Error No No **Türkçe:** Kontrol edilmemiş RuntimeException Alt sınıfı Evet istisna yok Kontrol Edildi Evet Evet Exception alt sınıfı ama istisna değil RuntimeException Error Alt sınıfı Error Hayır

Throwing an Exception

English: Any Java code can throw an exception; this includes code you write. Some exceptions are provided with Java. You might encounter an exception that was made up for the exam. This is fine. The question will make it obvious that this is an exception by having the class name end with Exception. For example, MyMadeUpException is clearly an exception. **Türkçe:** Herhangi bir Java kodu bir istisna atabilir; Bu, yazdığınız kodu içerir. Bazı istisnalar Java ile sağlanır. Sınav için uydurulmuş bir istisna ile karşılaşabilirsiniz. Bu iyi. Soru, Exception ile sınıf adının sona ermesiyle bunun bir istisna olduğunu açıkça gösterecektir. Örneğin, MyMadeUpException açıkça bir istisnadır. **English:** On the exam, you will see two types of code that result in an exception. The first is code that’s wrong. Here’s an example: **Türkçe:** Sınavda, bir istisna ile sonuçlanan iki tür kod göreceksiniz. Birincisi yanlış olan koddur. İşte bir örnek:

```
String[] animals = new String[0];
System.out.println(animals[0]); // ArrayIndexOutOfBoundsException
```

English: This code throws an `ArrayIndexOutOfBoundsException` since the array has no elements. That means questions about exceptions can be hidden in questions that appear to be about something else.

Türkçe: Bu kod bir `ArrayIndexOutOfBoundsException` atar, çünkü dizinin hiçbir elemanı yoktur. Bu, istisnalarla ilgili soruların başka bir şeyle ilgili gibi görünen sorularda gizlenebileceği anlamına gelir.

English: On the exam, some questions have a choice about not compiling and about throwing an exception. Pay special attention to code that calls a method on a null reference or that references an invalid array or List index. If you spot this, you know the correct answer is that the code throws an exception at runtime.

Türkçe: Sınavda, bazı soruların derlememe ve bir istisna atma konusunda bir seçeneği vardır. null referansı üzerinde bir yöntemi çağıran veya geçersiz bir dizi veya List dizini referans alan koda özel dikkat gösterin. Bunu fark ederseniz, doğru cevabın kodun çalışma zamanında bir istisna attığını bilirsiniz.

English: The second way for code to result in an exception is to explicitly request Java to throw one. Java lets you write statements like these: **Türkçe:** Kodun bir istisna ile sonuçlanmasının ikinci yolu, açıkça bir tane atmak için Java istemektir. Java bu gibi ifadeler yazmanızı sağlar:

```
throw new Exception();
throw new Exception("Ow! I fell.");
throw new RuntimeException();
throw new RuntimeException("Ow! I fell.");
```

English: The throw keyword tells Java that you want some other part of the code to deal with the exception. This is the same as the young girl crying for her daddy. Someone else needs to figure out what to do about the exception.

Türkçe: Atma tuşu, Java kodunun başka bir bölümünün istisnayla başa çıkmasını istediğinizi söyler. Bu, babası için ağlayan genç kızın aynısı. İstisna konusunda başkasının ne yapması gerektiğini anlaması gerekiyor.

English: throw vs. throws Anytime you see throw or throws on the exam, make sure the correct one is being used. The throw keyword is used as a statement inside a code block to throw a new exception or rethrow an existing exception, while the throws keyword is used only at the end of a method declaration to indicate what exceptions it supports.

Türkçe: Sınava attığınızı veya attığınızı gördüğünüz her an, doğru olanın kullanıldığından emin olun. Atma anahtarı, yeni bir istisna atmak veya mevcut bir istisnayı yeniden atmak için bir kod bloğunun içindeki bir ifade olarak kullanılırken, atma anahtarı yalnızca hangi istisnaları desteklediğini belirtmek için bir yöntem beyanının sonunda kullanılır.

English: When creating an exception, you can usually pass a String parameter with a message, or you can pass no parameters and use the defaults. We say usually because this is a convention.

Someone has declared a constructor that takes a String. Someone could also create an exception class that does not have a constructor that takes a message.

Türkçe: Bir istisna oluştururken, genellikle bir mesajla bir String parametresini geçebilir veya hiçbir parametreyi geçemez ve varsayılanları kullanabilirsiniz. Genellikle diyoruz çünkü bu bir kongre. Birisi String alan bir constructor ilan etti. Birisi ayrıca bir mesaj alan bir yapıcıya sahip olmayan bir istisna sınıfı da oluşturabilir.

English: Additionally, you should know that an Exception is an Object. This means you can store it in an object reference, and this is legal:

Türkçe: Ayrıca, bir Exception nesnesi olduğunu bilmelisiniz. Bu, bir nesne referansında depolayabileceğiniz anlamına gelir ve bu yasaldir:

```
var e = new RuntimeException();
throw e;
```

English: The code instantiates an exception on one line and then throws on the next. The exception can come from anywhere, even passed into a method. As long as it is a valid exception, it can be thrown.

Türkçe: Kod, bir satırda bir istisnayı anında yapar ve ardından bir sonraki satıra atar. İstisna her yerden gelebilir, hatta bir yönteme bile geçebilir. long geçerli bir istisna olduğu için atılabilir. **English:** The exam might also try to trick you. Do you see why this code doesn't compile? **Türkçe:** Sınav da sizi kandırmaya çalışabilir. Bu kodun neden derlenmediğini anlıyor musunuz?

```
throw RuntimeException(); // DOES NOT COMPILE
```

English: If your answer is that there is a missing keyword, you're absolutely right. The exception is never instantiated with the new keyword. **Türkçe:** Cevabınız eksik bir anahtar kelime varsa, kesinlikle haklısın.

İstisna hiçbir zaman yeni anahtar kelime ile anlık hale getirilmez. **English:** Let's take a look at another place the exam might try to trick you. Can you see why the following does not compile? **Türkçe:** Sınavın sizi kandırmaya çalışabileceği başka bir yere bir göz atalım. Aşağıdakilerin neden derlenmediğini görebiliyor musunuz?

```
try {  
    throw new RuntimeException();  
    throw new ArrayIndexOutOfBoundsException(); // DOES NOT COMPILE  
} catch (Exception e) {}
```

English: Since line 4 throws an exception, line 5 can never be reached during runtime. The compiler recognizes this and reports an unreachable code error. **Türkçe:** Hat 4 bir istisna attığından, hat 5'e çalışma süresi boyunca asla ulaşılamaz. Derleyici bunu tanır ve erişilemez bir kod hatası bildirir.

Calling Methods That Throw Exceptions

English: When you're calling a method that throws an exception, the rules are the same as within a method. Do you see why the following doesn't compile? **Türkçe:** Bir istisna atan bir yöntem ararken, kurallar bir yöntem içinde aynıdır. Aşağıdakilerin neden derlenmediğini görüyor musunuz?

```
class NoMoreCarrotsException extends Exception {}  
public class Bunny {  
    public static void main(String[] args) {  
        eatCarrot(); // DOES NOT COMPILE  
    }  
    private static void eatCarrot() throws NoMoreCarrotsException {}  
}
```

English: The problem is that NoMoreCarrotsException is a checked exception. Checked exceptions must be handled or declared. The code would compile if you changed the main() method to either of these:

Türkçe: Sorun şu ki NoMoreCarrotsException bir checked exception. Checked exceptions ele alınmalı veya ilan edilmelidir. main() yöntemini şunlardan birine değiştirirseniz kod derlenir:

```
public static void main(String[] args) throws NoMoreCarrotsException {
    eatCarrot();
}
public static void main(String[] args) {
    try {
        eatCarrot();
    } catch (NoMoreCarrotsException e) {
        System.out.print("sad rabbit");
    }
}
```

English: You might have noticed that `eatCarrot()` didn't throw an exception; it just declared that it could. This is enough for the compiler to require the caller to handle or declare the exception. **Türkçe:** `eatCarrot()` bir istisna atmadığını fark etmiş olabilirsiniz; sadece atabileceğini ilan etti. Bu, derleyicinin arayanın istisnayı ele almasını veya ilan etmesini gerektirmesi için yeterlidir. **English:** The compiler is still on the lookout for unreachable code. Declaring an unused exception isn't considered unreachable code. It gives the method the option to change the implementation to throw that exception in the future. Do you see the issue here? **Türkçe:** Derleyici hala erişilemeyen kodu arıyor. Kullanılmayan bir istisnayı ilan etmek erişilemez bir kod olarak kabul edilmez. Yönteme gelecekte bu istisnayı atmak için uygulamayı değiştirme seçeneği sunar. Buradaki sorunu görüyor musunuz?

```
public void bad() {
    try {
        eatCarrot();
    } catch (NoMoreCarrotsException e) { // DOES NOT COMPILE
        System.out.print("sad rabbit");
    }
}
private void eatCarrot() {}
```

English: Java knows that `eatCarrot()` can't throw a checked exception— which means there's no way for the catch block in `bad()` to be reached. **Türkçe:** Java `eatCarrot()` checked exception atamayacağını biliyor, bu da `bad()` içinde catch block için ulaşılacak bir yol olmadığı anlamına geliyor. **English:** When you see a checked exception declared inside a catch block on the exam, make sure the code in the associated try block is capable of throwing the exception or a subclass of the exception. If not, the code is unreachable and does not compile. Remember that this rule does not extend to unchecked exceptions or exceptions declared in a method signature. **Türkçe:** Sınavda bir catch block içinde ilan edilen bir checked exception gördüğünüzde, ilgili try block içindeki kodun istisnayı veya istisnanın bir alt sınıfını atabileceğinden emin olun. Değilse, kod erişilemez ve derlemez. Bu kuralın unchecked exceptions ya da method signature olarak ilan edilen istisnalara uzanmadığını unutmayın.

Overriding Methods with Exceptions

English: When we introduced overriding methods in Chapter 6, "Class Design," we included a rule related to exceptions. An overridden method may not declare any new or broader checked exceptions than the method it inherits. For example, this code isn't allowed: **Türkçe:** Bölüm 6, "Sınıf Tasarımı"nda overriding yöntemlerini tanıttığımızda, istisnalarla ilgili bir kural ekledik. Abartılmış bir yöntem, miras aldığı yöntemden daha yeni veya daha geniş bir checked exceptions beyan edemez. Örneğin, bu koda izin verilmez:

```
class CanNotHopException extends Exception {}
class Hopper {
    public void hop() {}
}
class Bunny extends Hopper {
    public void hop() throws CanNotHopException {} // DOES NOT COMPILE
}
```

English: Java knows `hop()` isn't allowed to throw any checked exceptions because the `hop()` method in the superclass `Hopper` doesn't declare any. Imagine what would happen if the subclasses' versions of the method could add checked exceptions—you could write code that calls `Hopper`'s `hop()` method and not handle any exceptions. Then, if `Bunny` were used in its place, the code wouldn't know to handle or declare `CanNotHopException`. **Türkçe:** Java, `hop()`'nin herhangi bir checked exceptions atmasına izin verilmediğini, çünkü süper sınıf `Hopper`'daki `hop()` yönteminin hiçbir beyanda bulunmadığını biliyor. Metodun alt sınıflarının sürümleri checked exceptions ekleyebilirse ne olacağını hayal edin `Hopper`'ın `hop()` yöntemini çağıran ve herhangi bir istisnayı ele almayan kod yazabilirsiniz. Daha sonra, eğer `Bunny` yerine kullanılırsa, kod `CanNotHopException` ile başa çıkmayı veya ilan etmeyi bilmeyecekti. **English:** An overridden method in a subclass is allowed to declare fewer exceptions than the superclass or interface. This is legal because callers are already handling them. **Türkçe:** Bir alt sınıftaki geçersiz bir yöntemin, süper sınıf veya arayüzden daha az istisna beyan etmesine izin verilir. Bu yasaldır, çünkü arayanlar zaten onları ele almaktadır.

```
class Hopper {
    public void hop() throws CanNotHopException {}
}
class Bunny extends Hopper {
    public void hop() {} // This is fine
}
```

English: An overridden method not declaring one of the exceptions thrown by the parent method is similar to the method declaring that it throws an exception it never actually throws. This is perfectly legal. Similarly, a class is allowed to declare a subclass of an exception type. The idea is the same. The superclass or interface has already taken care of a broader type. **Türkçe:** Ebeveyn yöntemi tarafından atılan istisnalardan birini beyan etmeyen aşınmış bir yöntem, aslında hiç atmadığı bir istisna attığını beyan eden yöntemle benzer. Bu tamamen yasal. Benzer şekilde, bir sınıfın bir istisna türünün alt sınıfını ilan etmesine izin verilir. Fikir aynı. Süper sınıf veya arayüz zaten daha geniş bir tiple ilgilendi.

Printing an Exception

English: There are three ways to print an exception. You can let Java print it out, print just the message, or print where the stack trace comes from. This example shows all three approaches: **Türkçe:** Bir istisnayı yazdırmanın üç yolu vardır. Java yazdırabilir, sadece mesajı yazdırabilir veya yığın izinin nereden geldiğini yazdırabilirsiniz. Bu örnek üç yaklaşımı da göstermektedir:

```
public static void main(String[] args) {
    try {
        hop();
    } catch (Exception e) {
        System.out.println(e + "\n");
        System.out.println(e.getMessage() + "\n");
        e.printStackTrace();
    }
}

private static void hop() {
    throw new RuntimeException("cannot hop");
}
```

English: This code prints the following: **Türkçe:** Bu kod aşağıdakileri yazdırır:

```
java.lang.RuntimeException: cannot hop
```

English: cannot hop **Türkçe:** Atlayamazsın

```
java.lang.RuntimeException: cannot hop
at Handling.hop(Handling.java:15)
at Handling.main(Handling.java:7)
```

English: The first line shows what Java prints out by default: the exception type and message. The second line shows just the message. The rest shows a stack trace. The stack trace is usually the most helpful because it shows the hierarchy of method calls that were made to reach the line that threw the exception. **Türkçe:** İlk satır, Java'nın varsayılan olarak ne yazdığını gösterir: istisna türü ve mesajı. İkinci satır sadece mesajı gösterir. Gerisi yığın izi gösteriyor. Yığın izi genellikle en yararlı olanıdır, çünkü istisnayı atan çizgiye ulaşmak için yapılan yöntem çağrılarının hiyerarşisini gösterir.

Recognizing Exception Classes

English: You need to recognize three groups of exception classes for the exam: RuntimeException, checked Exception, and Error. We look at common examples of each type. For the exam, you'll need to recognize which type of an exception it is and whether it's thrown by the Java Virtual Machine (JVM) or by a programmer. For some exceptions, you also need to know which are inherited from one another.

Türkçe: Sınav için üç grup istisna sınıfı tanımanız gerekir: RuntimeException, Exception ve Error. Her türün ortak örneklerine bakıyoruz. Sınav için, hangi tür bir istisna olduğunu ve Java Sanal Makine (JVM) tarafından mı yoksa bir programcı tarafından mı atıldığını tanımanız gerekir. Bazı istisnalar için hangisinin birbirinden miras kaldığını da bilmeniz gerekir.

RuntimeException Classes

English: RuntimeException and its subclasses are unchecked exceptions that don't have to be handled or declared. They can be thrown by the programmer or the JVM. Common unchecked exception classes are listed in Table 11.2. **Türkçe:** RuntimeException ve alt sınıfları, ele alınması veya ilan edilmesi gerekmeyen unchecked exceptions 'dir. Programcı veya JVM tarafından atılabilirler. Ortak unchecked exception sınıfları Tablo 11.2'de listelenmiştir. **English:** TABLE 11.2 Unchecked exceptions Unchecked exception Description ArithmeticException Thrown when code attempts to divide by zero. **Türkçe:** TABLE 11.2 Unchecked exceptions Unchecked exception Açıklama ArithmeticException Kod sıfıra bölmeye çalışıldığında atıldı. **English:** ArrayIndexOutOfBoundsException Thrown when code uses illegal index to access array. **Türkçe:** ArrayIndexOutOfBoundsException Kod dizine erişmek için yasadışı indeks kullandığında atılır. **English:** ClassCastException Thrown when attempt is made to cast object to class of which it is not an instance. **Türkçe:** ClassCastException Bir nesneyi bir örnek olmadığı sınıfa atmaya teşebbüs edildiğinde atılır. **English:** NullPointerException Thrown when there is a null reference where an object is required. **Türkçe:** NullPointerException Bir nesnenin gerekli olduğu bir null referansı olduğunda atılır. **English:** IllegalArgumentException Thrown by programmer to indicate that method has been passed illegal or inappropriate argument. **Türkçe:** Yasadışı Argüman Exception Bu yöntemin yasa dışı veya uygunsuz bir argümandan geçtiğini belirtmek için programcı tarafından atılmıştır. **English:** NumberFormatException Subclass of IllegalArgumentException. **Türkçe:** NumberFormatException IllegalArgumentException alt sınıfı. **English:** Thrown when attempt is made to convert String to numeric type but String doesn't have appropriate format. **Türkçe:** String numerik tipe dönüştürme girişiminde bulunulduğunda atılır, ancak String appropriate formatına sahip değildir.

ArithmeticException

English: Trying to divide an int by zero gives an undefined result. When this occurs, the JVM will throw an ArithmeticException: **Türkçe:** int yi sıfıra bölmeye çalışmak tanımsız bir sonuç verir. Bu gerçekleştiğinde, JVM bir ArithmeticException atar:

```
int answer = 11 / 0;
```

English: Running this code results in the following output: **Türkçe:** Bu kodun çalıştırılması aşağıdaki çıktıyla sonuçlanır: **English:** Exception in thread "main" java.lang.ArithmeticException: / by zero Java doesn't spell out the word divide. That's okay, though, because we know that / is the division operator and that Java is trying to tell you division by zero occurred. **Türkçe:** Exception iş parçacığında "ana" java.lang.ArithmeticException: / sıfır Java kelime bölünmesini hecelemez. Yine de sorun değil, çünkü biliyoruz ki / bölme operatörü ve Java size bölünmenin sıfır meydana geldiğini söylemeye çalışıyor. **English:** The thread "main" is telling you the code was called directly or indirectly from a program with a main method. On the exam, this is all the output you will see. Next comes the name of the exception, followed by extra information (if any) that goes with the exception. **Türkçe:** "Ana" iplik, kodun doğrudan veya dolaylı olarak ana yönteme sahip bir programdan çağrıldığını söylüyor. Sınavda, göreceğiniz tüm çıktılar bunlardır. Daha sonra istisnanın adı gelir, ardından istisna ile birlikte gelen ekstra bilgiler (varsa) gelir.

ArrayIndexOutOfBoundsException

English: You know by now that array indexes start with 0 and go up to 1 less than the length of the array— which means this code will throw an `ArrayIndexOutOfBoundsException`: **Türkçe:** Şimdiye kadar dizi indekslerinin 0 ile başladığını ve dizinin uzunluğundan 1'e kadar daha az çıktığını biliyorsunuz - bu kod bir `ArrayIndexOutOfBoundsException` atacağı anlamına gelir:

```
int[] countsOfMoose = new int[3];
System.out.println(countsOfMoose[- 1]);
```

English: This is a problem because there's no such thing as a negative array index. Running this code yields the following output: **Türkçe:** Bu bir sorundur çünkü negatif dizi indeksi diye bir şey yoktur. Bu kodu çalıştırmak aşağıdaki çıktıyı verir: **English:** Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index - 1 out of bounds for length 3 **Türkçe:** Exception iş parçasığında "ana" java.lang.ArrayIndexOutOfBoundsException: Index - 1 uzunluğu için sınırlar dışında 3

ClassCastException

English: Java tries to protect you from impossible casts. This code doesn't compile because `Integer` is not a subclass of `String`: **Türkçe:** Java sizi imkansız kadrolardan korumaya çalışır. Bu kod derlemez çünkü `Integer` `String` alt sınıfı değildir:

```
String type = "moose";
Integer number = (Integer) type; // DOES NOT COMPILE
```

English: More complicated code thwarts Java's attempts to protect you. When the cast fails at runtime, Java will throw a `ClassCastException`: **Türkçe:** Daha karmaşık kod Java'in sizi koruma girişimlerini engeller. Kadro çalışma zamanında başarısız olduğunda, Java bir `ClassCastException` atar:

```
String type = "moose";
Object obj = type;
Integer number = (Integer) obj; // ClassCastException
```

[illegible]

NullPointerException

English: Instance variables and methods must be called on a non-null reference. If the reference is null, the JVM will throw a NullPointerException. **Türkçe:** Instance değişkenleri ve yöntemleri non-null referansında çağrılmalıdır. Eğer referans null ise, JVM bir NullPointerException atar.

English: 5: Türkçe: 5:

```
public static void main(String[] args) {
    new Frog().hop(null, 1);
}
```

English: Running this code results in the following output: **Türkçe:** Bu kodun çalıştırılması aşağıdaki çıktıyla sonuçlanır: **English:** Exception in thread "main" java.lang.NullPointerException: Cannot invoke **Türkçe:** Exception iş parçacığında "ana" java.lang.NullPointerException: çağrılmıyor

```
"String.toLowerCase()" because "<parameter1>" is null
```

English: If you're new to Java 17, you should have noticed something special about the output. The JVM now tells you the object reference that triggered the `NullPointerException`! This new feature is called `Helpful NullPointerExceptions`. **Türkçe:** Java 17'de yeniyseniz, çıktıyla ilgili özel bir şey fark etmiş olmalısınız. JVM şimdi size `NullPointerException`'i tetikleyen nesne referansını söyler! Bu yeni özellik `Helpful NullPointerExceptions` olarak adlandırılır. **English:** As another example, suppose we change line 7: **Türkçe:** Başka bir örnek olarak, satır 7'yi değiştirdiğimizi varsayalım:

```
new Frog().hop("Kermit", null);
```

English: Then the output at runtime changes as follows: **Türkçe:** Ardından çalışma zamanındaki çıktı aşağıdaki gibi değişir: **English:** Exception in thread "main" java.lang.NullPointerException: Cannot invoke **Türkçe:** Exception iş parçacığında "ana" java.lang.NullPointerException: çağıramıyor

```
"java.lang.Integer.intValue()" because "<parameter2>" is null
```

English: By default, a `NullPointerException` on a local variable or method parameter is printed with a number indicating the order in which it appears in the method, such as `<local2>` or `<parameter4>`. If you're like us and want the actual variable name to be shown, compile the code with the `---g:vars` flag, which adds debug info. In the previous examples, `<parameter1>` and `<parameter2>` are then replaced with name and jump, respectively. **Türkçe:** Varsayılan olarak, yerel bir değişken veya yöntem parametresinde bir `NullPointerException`, `local2>` veya `parametre4>` gibi yöntemde görüldüğü sırayı belirten bir sayı ile basılır. Bizim gibiyse ve gerçek değişken adının gösterilmesini istiyorsanız, kodu hata ayıklama bilgisi ekleyen `---g:vars` bayrağı ile derleyin. Önceki örneklerde, `parameter1>` ve `parameter2>` sırasıyla isim ve atlama ile değiştirilir. **English:** Since this is a new feature in Java, it's possible you'll see it in a question on the exam. **Türkçe:** Bu, Java 'da yeni bir özellik olduğundan, sınavdaki bir soruda görmemiz mümkündür. **English:** Enabling/Disabling `Helpful NullPointerExceptions` When `helpful NullPointerExceptions` were added in Java 14, the feature was disabled by default and had to be enabled via a command-line argument `ShowCodeDetailsInExceptionMessages` to the JVM: **Türkçe:** Java 14'te yararlı `NullPointerExceptions` eklendiğinde, özellik varsayılan olarak devre dışı bırakıldı ve JVM'ye bir komut satırı argümanı `ShowCodeDetailsInExceptionMessages` aracılığıyla etkinleştirilmek zorunda kaldı: **English:** `java -XX:+ShowCodeDetailsInExceptionMessages Frog` In Java 15 and above, the default behavior was changed so that it is enabled by default, although it can still be disabled via the command-line argument. **Türkçe:** `java -XX:+ShowCodeDetailsInExceptionMessages Frog` In Java 15 ve üstü, varsayılan davranış varsayılan olarak etkinleştirilecek şekilde değiştirildi, ancak yine de komut satırı argümanı aracılığıyla devre dışı bırakılabilir. **English:** `java -XX:-ShowCodeDetailsInExceptionMessages Frog` **Türkçe:** `java -XX:-ShowCodeDetailsInExceptionMesajlar Kurbağa`

IllegalArgumentException

English: `IllegalArgumentException` is a way for your program to protect itself. You want to tell the caller that something is wrong—preferably in an obvious way that the caller can't ignore so the programmer will fix the problem. Seeing the code end with an exception **Türkçe:** `IllegalArgumentException` programınızın kendisini korumanın bir yoludur. Arayana bir şeylerin yanlış olduğunu söylemek istersiniz - tercihen arayanın görmezden gelemeyeceği açık bir şekilde, böylece programcı sorunu çözecektir. Kodun bir istisna ile sona erdiğini görmek

English: is a great reminder that something is wrong. Consider this example when called as `setNumberEggs(- 2)`: **Türkçe:** Bir şeylerin yanlış olduğunu büyük bir hatırlatmadır. `setNumberEggs(- 2)` olarak çağrıldığında bu örneği düşünün:

```
public void setNumberEggs(int numberEggs) {
```

English: if (`numberEggs < 0`) **Türkçe:** if (`numberEggs < 0`)

```
    throw new IllegalArgumentException("# eggs must not be negative");  
    this.numberEggs = numberEggs;  
}
```

English: The program throws an exception when it's not happy with the parameter values. The output looks like this: **Türkçe:** Program, parametre değerlerinden memnun olmadığında bir istisna atar. Çıkış şu şekilde görünür: **English:** Exception in thread "main" **Türkçe:** Exception iş parçacığında "ana"

```
java.lang.IllegalArgumentException: # eggs must not be negative
```

English: Clearly, this is a problem that must be fixed if the programmer wants the program to do anything useful. **Türkçe:** Açıkçası, programcı programın yararlı bir şey yapmasını istiyorsa bu düzeltilmesi gereken bir sorundur.

NumberFormatException

English: Java provides methods to convert strings to numbers. When these are passed an invalid value, they throw a `NumberFormatException`. The idea is similar to `IllegalArgumentException`. Since this is a common problem, Java gives it a separate class. In fact, `NumberFormatException` is a subclass of `IllegalArgumentException`. Here's an example of trying to convert something non-numeric into an `int`:

Türkçe: Java sayılara strings dönüştürmek için yöntemler sağlar. Bunlar geçersiz bir değer geçtiğinde, bir `NumberFormatException` atarlar. Bu fikir `IllegalArgumentException` ile benzerdir. Bu yaygın bir sorun olduğundan, Java ona ayrı bir sınıf verir. Aslında, `NumberFormatException` `IllegalArgumentException` alt sınıfıdır. İşte numerik olmayan bir şeyi `int`'ye dönüştürmeye çalışmanın bir örneği:

```
Integer.parseInt("abc");
```

English: The output looks like this: **Türkçe:** Çıkış şu şekilde görünür: **English:** Exception in thread "main" **Türkçe:** Exception iş parçacığında "ana"

```
java.lang.NumberFormatException: For input string: "abc"
```

English: For the exam, you need to know that `NumberFormatException` is a subclass of `IllegalArgumentException`. We cover more about why that is important later in the chapter. **Türkçe:** Sınav için, `NumberFormatException` `IllegalArgumentException` alt sınıfı olduğunu bilmeniz gerekir. Bunun neden daha sonra bölümde önemli olduğu hakkında daha fazla bilgi veriyoruz.

Checked Exception Classes

English: Checked exceptions have Exception in their hierarchy but not RuntimeException. They must be handled or declared. Common checked exceptions are listed in Table 11.3. **Türkçe:** Checked exceptions hiyerarşisinde Exception var ama RuntimeException yok. Bunlar ele alınmalı veya ilan edilmelidir. Yaygın checked exceptions Tablo 11.3'te listelenmiştir. **English:** For the exam, you need to know that these are all checked exceptions that must be handled or declared. You also need to know that FileNotFoundException and NotSerializableException are subclasses of IOException. You see these three classes in Chapter 14, "I/O," and SQLException in Chapter 15, "JDBC." **Türkçe:** Sınav için, bunların hepsinin ele alınması veya ilan edilmesi gereken checked exceptions olduğunu bilmeniz gerekir. Ayrıca FileNotFoundException ve NotSerializableException alt sınıflarının IOException olduğunu bilmeniz gerekir. Bu üç sınıfı Bölüm 14, "I/O" ve Bölüm 15, "JDBC" de SQLException de görebilirsiniz.

English: TABLE 11.3 Checked exceptions Checked exception Description FileNotFoundException Subclass of IOException. Thrown programmatically when code tries to reference file that does not exist. **Türkçe:** TABLE 11.3 Checked exceptions Checked exception Açıklama FileNotFoundException IOException alt sınıfı. Kod mevcut olmayan dosyaya referans vermeye çalışıldığında programatik olarak atılır. **English:** IOException Thrown programmatically when problem reading or writing file. **Türkçe:** IOException Sorun okuma veya dosya yazma sırasında programatik olarak atılır. **English:** NotSerializableException Subclass of IOException. Thrown programmatically when attempting to serialize or deserialize non-serializable class. **Türkçe:** NotSerializableException IOException alt sınıfı. Serialize olmayan sınıfı serileştirmeye veya deserialize etmeye çalışırken programatik olarak atılmalıdır. **English:** ParseException Indicates problem parsing input. **Türkçe:** ParseException Problem ayrıştırma girdisini belirtir. **English:** SQLException Thrown when error related to accessing database. **Türkçe:** SQLException Veritabanına erişim ile ilgili hata atıldığında.

Error Classes

English: Errors are unchecked exceptions that extend the Error class. They are thrown by the JVM and should not be handled or declared. Errors are rare, but you might see the ones listed in Table 11.4. **Türkçe:** Error sınıfını genişleten unchecked exceptions hatalardır. JVM tarafından atılırlar ve ele alınmamalı veya ilan edilmemelidirler. Hatalar nadirdir, ancak Tablo 11.4'te listelenenleri görebilirsiniz. **English:** TABLE 11.4 Errors Error Description ExceptionInInitializerError Thrown when static initializer throws exception and doesn't handle it StackOverflowError Thrown when method calls itself too many times (called infinite recursion because method typically calls itself without end) **Türkçe:** TABLE 11.4 Hatalar Error Açıklama ExceptionInInitializerError Statik başlatıcı istisna attığında ve StackOverflowError yöntemi kendisini çok fazla çağırdığında fırlatılır (sonsuz tekrarlama olarak adlandırılır, çünkü yöntem tipik olarak kendisini sonsuz olarak çağırır) **English:** NoClassDefFoundError Thrown when class that code uses is available at compile time but not runtime For the exam, you just need to know that these errors are unchecked and the code is often unable to recover from them. **Türkçe:** NoClassDefFoundError Kodun kullandığı sınıf eş zamanlı olarak kullanılabilir ancak çalışma zamanı değilken fırlatılır Sınav için, bu hataların kontrol edilmediğini ve kodun genellikle onlardan kurtarılamayacağını bilmeniz yeterlidir.

Handling Exceptions

English: What do you do when you encounter an exception? How do you handle or recover from the exception? In this section, we show the various statements in Java that support handling exceptions.

Türkçe: Bir istisnaya karşılaştığınızda ne yaparsınız? İstisnadan nasıl kurtulur veya kurtulursunuz? Bu bölümde, kullanım istisnalarını destekleyen Java 'daki çeşitli ifadeleri gösteriyoruz.

Using try and catch Statements

English: Now that you know what exceptions are, let's explore how to handle them. Java uses a try statement to separate the logic that might throw an exception from the logic to handle that exception. Figure 11.2 shows the syntax of a try statement.

Türkçe: Artık istisnaların ne olduğunu bildiğinize göre, bunları nasıl ele alacağınızı araştıralım. Java, bu istisnayı işlemek için mantıktan bir istisna atabilecek mantığı ayırmak için bir deneme ifadesi kullanır. Şekil 11.2, bir deneme ifadesinin sözdizimini gösterir.

English: FIGURE 11.2 The syntax of a try statement The try keyword try { The identifier of the Curly braces are exception object // Protected code required. **Türkçe:** FIGURE 11.2 Bir deneme ifadesinin sözdizimi Anahtar kelime deneyin Kıvrık dış tellerinin tanımlayıcısı istisna nesnesidir // Korunmalı kod gereklidir. **English:** } catch (exception_type identifier) { The type of exception being caught// Exception handler } The catch keyword The code in the try block is run normally. If any of the statements throws an exception that can be caught by the exception type listed in the catch block, the try block stops running, and execution goes to the catch statement. If none of the statements in the try block throws an exception that can be caught, the catch clause is not run. **Türkçe:** yakalama (istisna_tipi tanımlayıcı) Yakalanan istisna türü/ Exception işleyici Yakalama anahtar kelimesi try block içindeki kod normal olarak çalıştırılır. İfadelerden herhangi biri, catch block bölümünde listelenen istisna türüne göre yakalanabilecek bir istisna atarsa, try block çalışmayı durdurur ve yürütme yakalama ifadesine gider. try block içindeki ifadelerin hiçbiri yakalanabilecek bir istisna atmazsa, yakalama maddesi çalıştırılmaz. **English:** You probably noticed the words block and clause used interchangeably. The exam does this as well, so get used to it. Both are correct. Block is correct because there are braces present. Clause is correct because it is part of a try statement. **Türkçe:** Muhtemelen birbirinin yerine kullanılan blok ve madde kelimelerini fark ettiniz. Sınav da bunu yapıyor, buna alışın. İkisi de doğru. Blok doğrudur, çünkü dış telleri mevcuttur. Clause doğrudur çünkü bir deneme ifadesinin bir parçasıdır. **English:** There aren't a ton of syntax rules here. The curly braces are required for try and catch blocks. In our example, the little girl gets up by herself the first time she falls. Here's what this looks like: **Türkçe:** Burada tonlarca sözdizimi kuralı yoktur. Kıvrık dış telleri denemek ve catch blocks için gereklidir. Bizim örneğimizde, küçük kız ilk düştüğünde tek başına kalkar. İşte bu neye benziyor:

```
void explore() {
    try {
        fall();
        System.out.println("never get here");
    } catch (RuntimeException e) {
        getUp();
    }
    seeAnimals();
}

void fall() { throw new RuntimeException(); }
```

English: First, line 5 calls the `fall()` method. Line 12 throws an exception. This means Java jumps straight to the catch block, skipping line 6. The girl gets up on line 8. Now the try statement is over, and execution proceeds normally with line 10. **Türkçe:** İlk olarak, 5. satır `fall()` yöntemini çağırır. 12. hat bir istisna yaratıyor. Bu, Java doğrudan catch block'e atlayarak 6. satırı atladığı anlamına gelir. Kız 8. hatta kalkıyor. Şimdi deneme ifadesi sona erdi ve yürütme normal olarak 10. satırla devam ediyor. **English:** Now let's look at some invalid try statements that the exam might try to trick you with. Do you see what's wrong with this one? **Türkçe:** Şimdi sınavın sizi kandırmaya çalışabileceği bazı geçersiz deneme ifadelerine bakalım. Bunun nesi olduğunu görüyor musun?

```
try // DOES NOT COMPILE
fall();
```

English: catch (Exception e) **Türkçe:** yakalama (Exception e)

```
System.out.println("get up");
```

English: The problem is that the braces `{}` are missing. The try statements are like methods in that the curly braces are required even if there is only one statement inside the code blocks, while if statements and loops are special and allow you to omit the curly braces. **Türkçe:** Sorun şu ki, dış telleri eksik. Deneme ifadeleri, kod bloklarının içinde sadece bir ifade olsa bile, ifadeler ve döngüler özelse ve kıvrıkcık dış tellerini atlamanıza izin verirse, kıvrıkcık dış tellerinin gerekli olduğu yöntemler gibidir. **English:** What about this one? **Türkçe:** Peki ya bu?

```
try { // DOES NOT COMPILE
fall();
}
```

English: This code doesn't compile because the try block doesn't have anything after it. Remember, the point of a try statement is for something to happen if an exception is thrown. Without another clause, the try statement is lonely. As you see shortly, there is a special type of try statement that includes an implicit finally block, although the syntax is quite different from this example. **Türkçe:** Bu kod derlemez, çünkü try block ondan sonra hiçbir şeye sahip değildir. Unutmayın, bir deneme ifadesinin amacı, bir istisna atılırsa bir şeyin gerçekleşmesidir. Başka bir madde olmadan, deneme ifadesi yalnızdır. Kısa sürede gördüğünüz gibi, sözdizimi bu örnekten oldukça farklı olmasına rağmen, örtülü bir finally block içeren özel bir deneme ifadesi türü vardır.

Chaining catch Blocks

English: For the exam, you may be given exception classes and need to understand how they function. Here's how to tackle them. First, you must be able to recognize if the exception is a checked or an unchecked exception. Second, you need to determine whether any of the exceptions are subclasses of the others. **Türkçe:** Sınav için, size istisna dersleri verilebilir ve onların function nasıl olduğunu anlamanız gerekir. İşte bunlarla nasıl başa çıkılacağı. İlk olarak, istisnanın bir kontrol veya unchecked exception olup olmadığını fark edebilmelisiniz. İkincisi, istisnalardan herhangi birinin diğerlerinin alt sınıfı olup olmadığını belirlemeniz gerekir.

```
class AnimalsOutForAWalk extends RuntimeException {}
class ExhibitClosed extends RuntimeException {}
class ExhibitClosedForLunch extends ExhibitClosed {}
```

English: In this example, there are three custom exceptions. All are unchecked exceptions because they directly or indirectly extend RuntimeException. Now we chain both types of exceptions with two catch blocks and handle them by printing out the appropriate message: **Türkçe:** Bu örnekte, üç özel istisna vardır. Hepsi unchecked exceptions çünkü doğrudan veya dolaylı olarak RuntimeException uzatıyorlar. Şimdi her iki tür istisnayı iki catch blocks ile zincirliyoruz ve uygun mesajı yazdırarak ele alıyoruz:

```
public void visitPorcupine() {
    try {
        seeAnimal();
    } catch (AnimalsOutForAWalk e) { // first catch block
        System.out.print("try back later");
    }
```

```
    } catch (ExhibitClosed e) { // second catch block
        System.out.print("not today");
    }
}
```

English: There are three possibilities when this code is run. If seeAnimal() doesn't throw an exception, nothing is printed out. If the animal is out for a walk, only the first catch block runs. If the exhibit is closed, only the second catch block runs. It is not possible for both catch blocks to be executed when chained together like this. **Türkçe:** Bu kod çalıştırıldığında üç olasılık vardır. seeAnimal() bir istisna atmazsa, hiçbir şey yazdırılmaz. Hayvan yürüyüşe çıkmışsa, sadece ilk catch block çalışır. Sergi kapalıysa, yalnızca ikinci catch block çalışır. Her iki catch blocks'ün de bu şekilde zincirlendiğinde çalıştırılması mümkün değildir.

English: A rule exists for the order of the catch blocks. Java looks at them in the order they appear. If it is impossible for one of the catch blocks to be executed, a compiler error about unreachable code occurs. For example, this happens when a superclass catch block appears before a subclass catch block.

Remember, we warned you to pay attention to any subclass exceptions. **Türkçe:** catch blocks düzeni için bir kural vardır. Java gördükleri sıraya bakarlar. catch blocks 'den birinin çalıştırılması imkansızsa, erişilemez kodla ilgili bir derleyici hatası oluşur. Örneğin, bu, bir alt sınıf catch block önce bir süper sınıf catch block görüldüğünde olur. Unutmayın, herhangi bir alt sınıf istisnasına dikkat etmeniz konusunda sizi uyarımiştık. **English:** In the porcupine example, the order of the catch blocks could be reversed because the exceptions don't inherit from each other. And yes, we have seen a porcupine be taken for a walk on a leash. **Türkçe:** Kirpi örneğinde, istisnalar birbirinden miras almadığı için catch blocks sırası tersine çevrilebilir. Ve evet, bir kirpinin tasma üzerinde yürüyüşe çıkarıldığını gördük. **English:** The following example shows exception types that do inherit from each other: **Türkçe:** Aşağıdaki örnek, birbirinden miras kalan istisna türlerini göstermektedir:

```
public void visitMonkeys() {
    try {
        seeAnimal();
    } catch (ExhibitClosedForLunch e) { // Subclass exception
        System.out.print("try back later");
    } catch (ExhibitClosed e) {
        // Superclass exception
        System.out.print("not today");
    }
}
```

English: If the more specific `ExhibitClosedForLunch` exception is thrown, the first catch block runs. If not, Java checks whether the superclass `ExhibitClosed` exception is thrown and catches it. This time, the order of the catch blocks does matter. The reverse does not work. **Türkçe:** Daha spesifik `ExhibitClosedForLunch` istisnası atılırsa, ilk catch block çalışır. Değilse, Java süper sınıf `ExhibitClosed` istisnasının atılıp atılmadığını kontrol eder ve yakalar. Bu kez, catch blocks'in sırası önemlidir. Tam tersi işe yaramıyor.

```
public void visitMonkeys() {
    try {
        seeAnimal();
    } catch (ExhibitClosed e) {
        System.out.print("not today");
    } catch (ExhibitClosedForLunch e) { // DOES NOT COMPILE
        System.out.print("try back later");
    }
}
```

English: If the more specific `ExhibitClosedForLunch` exception is thrown, the catch block for `ExhibitClosed` runs—which means there is no way for the second catch block to ever run. Java correctly tells you there is an unreachable catch block. **Türkçe:** Daha spesifik `ExhibitClosedForLunch` istisnası atılırsa, `ExhibitClosed` çalışır için catch block yani ikinci catch block'in hiç çalışması mümkün değildir. Java doğru bir şekilde ulaşılabilir catch block olduğunu söyler.

English: Let's try this one more time. Do you see why this code doesn't compile? **Türkçe:** Bunu bir kez daha deneyelim. Bu kodun neden derlenmediğini anlıyor musunuz?

```
public void visitSnakes() {
    try {
    } catch (IllegalArgumentException e) {
    } catch (NumberFormatException e) { // DOES NOT COMPILE
    }
}
```

English: Remember we said earlier that you needed to know that `NumberFormatException` is a subclass of `IllegalArgumentException`? This example is the reason why. Since `NumberFormatException` is a subclass, it will always be caught by the first catch block, making the second catch block unreachable code that does not compile. Likewise, for the exam, you need to know that `FileNotFoundException` is a subclass of `IOException` and cannot be used in a similar manner. **Türkçe:** Daha önce `NumberFormatException` `IllegalArgumentException` alt sınıfı olduğunu bilmeniz gerektiğini söylemiştik hatırlıyor musunuz? Bu örnek bunun nedenidir. `NumberFormatException` bir alt sınıf olduğundan, her zaman ilk catch block tarafından yakalanır ve derlenmeyen ikinci catch block erişilemez kod haline gelir. Aynı şekilde, sınav için, `FileNotFoundException` `IOException` alt sınıfının olduğunu ve benzer bir şekilde kullanılamayacağını bilmeniz gerekir. **English:** To review multiple catch blocks, remember that at most one catch block will run, and it will be the first catch block that can handle the exception. Also, remember that an exception defined by the catch statement is only in scope for that catch block. For example, the following causes a compiler error since it tries to use the exception object outside the block for which it was defined: **Türkçe:** Birden fazla catch blocks gözden geçirmek için, en fazla bir catch block çalıştırılacağını ve istisnayı kaldırabilecek ilk catch block olacağını unutmayın. Ayrıca, yakalama ifadesiyle tanımlanan bir istisnanın yalnızca bu catch block için kapsam dahilinde olduğunu unutmayın. Örneğin, aşağıdakiler, tanımlandığı bloğun dışındaki istisna nesnesini kullanmaya çalıştığı için bir derleyici hatasına neden olur:

```
public void visitManatees() {
    try {
    } catch (NumberFormatException e1) {
        System.out.println(e1);
    } catch (IllegalArgumentException e2) {
        System.out.println(e1); // DOES NOT COMPILE
    }
}
```

Applying a Multi-catch Block

English: Often, we want the result of an exception that is thrown to be the same, regardless of which particular exception is thrown. For example, take a look at this method: **Türkçe:** Genellikle, hangi istisnanın atıldığına bakılmaksızın, atılan bir istisnanın sonucunun aynı olmasını isteriz. Örneğin, bu yöntemle bir göz atın:

```
public static void main(String args[]) {
    try {
        System.out.println(Integer.parseInt(args[1]));
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Missing or invalid input");
    } catch (NumberFormatException e) {
        System.out.println("Missing or invalid input");
    }
}
```

English: Notice that we have the same `println()` statement for two different catch blocks. We can handle this more gracefully using a multi-catch block. A multi-catch block allows multiple exception types to be caught by the same catch block. Let's rewrite the previous example using a multi-catch block: **Türkçe:** İki farklı catch blocks için aynı `println()` ifadesine sahip olduğumuza dikkat edin. Bunu bir multi-catch block kullanarak daha zarif bir şekilde halledebiliriz. Bir multi-catch block, birden fazla istisna türünün aynı catch block tarafından yakalanmasına izin verir. Önceki örneği bir multi-catch block kullanarak yeniden yazalım:

```
public static void main(String[] args) {
    try {
        System.out.println(Integer.parseInt(args[1]));
    } catch (ArrayIndexOutOfBoundsException | NumberFormatException e) {
        System.out.println("Missing or invalid input");
    }
}
```

English: This is much better. There's no duplicate code, the common logic is all in one place, and the logic is exactly where you would expect to find it. If you wanted, you could still have a second catch block for Exception in case you want to handle other types of exceptions differently. **Türkçe:** Bu çok daha iyi. Kopyalanmış bir kod yoktur, ortak mantık hepsi tek bir yerdedir ve mantık tam olarak onu bulmayı beklediğiniz yerdir. İsterseniz, diğer istisna türlerini farklı bir şekilde ele almak istiyorsanız, Exception için ikinci bir catch block sahibi olabilirsiniz. **English:** Figure 11.3 shows the syntax of multi-catch. It's like a regular catch clause, except two or more exception types are specified, separated by a pipe. The pipe (|) is also used as the "or" operator, making it easy to remember that you can use either/or of the exception types. Notice how there is only one variable name in the catch clause. Java is saying that the variable named e can be of type Exception1 or Exception2. **Türkçe:** Şekil 11.3, çoklu yakalama sözdizimi gösterir. Normal bir yakalama maddesi gibidir, iki veya daha fazla istisna türü belirtilmedikçe, bir boru ile ayrılır. Boru (|) aynı zamanda "veya" operatörü olarak da kullanılır, bu da istisna tiplerinden birini / veya birini kullanabileceğinizi hatırlamanızı kolaylaştırır. Yakalama maddesinde sadece bir değişken adı olduğuna dikkat edin. Java, e adlı değişkenin Exception1 veya Exception2 tipi olabileceğini söylüyor. **English:** FIGURE 11.3 The syntax of a multi-catch block **Türkçe:** FIGURE 11.3 Bir multi-catch block sözdizimi

```
try {
```

English: Catch either of **Türkçe:** İkisini de yakala

```
// Protected code
```

English: these exceptions. **Türkçe:** Bu istisnalar.

```
} catch (Exception1 | Exception2 e) {
```

English: Single identifier for **Türkçe:** için tek tanımlayıcı

```
// Exception handler
```

English: all exception types **Türkçe:** tüm istisna türleri

```
} Required | between
```

English: exception types The exam might try to trick you with invalid syntax. Remember that the exceptions can be listed in any order within the catch clause. However, the variable name must appear only once and at the end. Do you see why these are valid or invalid? **Türkçe:** istisna türleri Sınav sizi geçersiz sözdizimi ile kandırmaya çalışabilir. İstisnaların yakalama maddesindeki herhangi bir sırada listelenebileceğini unutmayın. Bununla birlikte, değişken adı sadece bir kez ve sonunda görünmelidir. Bunların neden geçerli veya geçersiz olduğunu görüyor musunuz?

```
catch(Exception1 e | Exception2 e | Exception3 e) // DOES NOT COMPILE
catch(Exception1 e1 | Exception2 e2 | Exception3 e3) // DOES NOT COMPILE
catch(Exception1 | Exception2 | Exception3 e)
```

English: The first line is incorrect because the variable name appears three times. Just because it happens to be the same variable name doesn't make it okay. The second line is incorrect because the variable name again appears three times. Using different variable names doesn't make it any better. The third line does compile. It shows the correct syntax for specifying three exceptions. **Türkçe:** İlk satır yanlıştır, çünkü değişken adı üç kez görünür. Sadece aynı değişken isim olması onu iyi yapmaz. İkinci satır yanlıştır, çünkü değişken adı tekrar üç kez görünür. Farklı değişken isimleri kullanmak onu daha iyi yapmaz. Üçüncü satır derlenir. Üç istisnayı belirtmek için doğru söz dizimini gösterir. **English:** Java intends multi-catch to be used for exceptions that aren't related, and it prevents you from specifying redundant types in a multi-catch. Do you see what is wrong here? **Türkçe:** Java, ilgili olmayan istisnalar için çoklu yakalamanın kullanılmasını amaçlar ve çok yakalamada gereksiz türleri belirtmenizi önler. Burada neyin yanlış olduğunu görüyor musun?

```
try {
    throw new IOException();
} catch (FileNotFoundException | IOException p) {} // DOES NOT COMPILE
```

English: Specifying related exceptions in the multi-catch is redundant, and the compiler gives a message such as this: **Türkçe:** Çoklu yakalamada ilgili istisnaları belirtmek gereksizdir ve derleyici şu gibi bir mesaj verir: **English:** The exception `FileNotFoundException` is already caught by the alternative `IOException`. Since `FileNotFoundException` is a subclass of `IOException`, this code will not compile. A multi-catch block follows rules similar to chaining catch blocks together, which you saw in the previous section. For example, both trigger compiler errors when they encounter unreachable code or duplicate exceptions being caught. The one difference between multicatch blocks and chaining catch blocks is that order does not matter for a multi-catch block within a single catch expression. **Türkçe:** `FileNotFoundException` istisnası zaten alternatif `IOException` tarafından yakalanır `FileNotFoundException` `IOException` alt sınıfı olduğundan, bu kod derlemez. Bir multi-catch block, bir önceki bölümde gördüğünüz catch blocks zincirlemeye benzer kuralları takip eder. Örneğin, her ikisi de erişilemeyen kodla karşılaştıklarında veya yakalanması gereken yinelenen istisnalarla karşılaştıklarında derleyici hatalarını tetikler. Çok yakalama blokları ile zincirleme catch blocks arasındaki tek fark, düzenin tek bir yakalama ifadesi içinde bir multi-catch block için önemli olmamasıdır. **English:** Getting back to the example, the correct code is just to drop the extraneous subclass reference, as shown here: **Türkçe:** Örneğimize geri dönersek, doğru kod sadece burada gösterildiği gibi yabancı alt sınıf referansını düşürmektir:

```
try {
    throw new IOException();
} catch (IOException e) {}
```

Adding a finally Block

English: The try statement also lets you run code at the end with a finally clause, regardless of whether an exception is thrown. Figure 11.4 shows the syntax of a try statement with this extra functionality.

Türkçe: Deneme ifadesi ayrıca, bir istisnanın atılıp atılmadığına bakılmaksızın, sonunda bir son madde ile kod çalıştırmanıza izin verir. Şekil 11.4, bu ekstra işlevsellik ile bir deneme ifadesi sözdizimini gösterir.

English: There are two paths through code with both a catch and a finally. If an exception is thrown, the finally block is run after the catch block. If no exception is thrown, the finally block is run after the try block completes. **Türkçe:** Kodlama yoluyla hem yakalama hem de sonunda iki yol vardır. Bir istisna atılırsa, catch block'den sonra finally block çalıştırılır. Herhangi bir istisna atılmazsa, try block tamamlandıktan sonra finally block çalıştırılır. **English:** Let's go back to our young girl example, this time with finally:

Türkçe: Genç kız örneğimize geri dönelim, bu sefer son olarak:

```
void explore() {
    try {
        seeAnimals();
        fall();
```

```
    } catch (Exception e) {
        getHugFromDaddy();
    } finally {
        seeMoreAnimals();
    }
    goHome();
}
```

English: FIGURE 11.4 The syntax of a try statement with finally The catch block is optionaltry { when finally is used. **Türkçe:** FIGURE 11.4 Son olarak catch block ile bir deneme ifadesinin sözdizimini, son olarak kullanıldığında Optional 'dır. **English:** // Protected code } catch (exception_type identifier) { // Exception handler } finally { The finally block always executes, whether or not an // finally block exception occurs. **Türkçe:** // Korunan kod yakalama (istisna_tipi tanımlayıcı) // Exception işleyici nihayet finally block her zaman çalışır, bir // finally block istisnası meydana gelip gelmediği. **English:** } The finally keyword The girl falls on line 15. If she gets up by herself, the code goes on to the finally block and runs line 19. Then the try statement is over, and the code proceeds on line 21. If the girl doesn't get up by herself, she throws an exception. The catch block runs, and she gets a hug on line 17. With that hug, she is ready to see more animals on line 19. Then the try statement is over, and the code proceeds on line 21. Either way, the ending is the same. The finally block is executed, and execution continues after the try statement. **Türkçe:** Son olarak anahtar kelime Kız 15. hatta düşüyor. Tek başına kalkarsa, kod finally block 'a gider ve 19. satırı çalıştırır. Sonra deneme ifadesi sona erdi ve kod 21. satırda devam etti. Kız tek başına kalkmazsa, bir istisna atar. catch block çalışır ve 17. hatta sarılır. Bu sarılmayla, 19 numaralı hatta daha fazla hayvan görmeye hazırdır. Sonra deneme ifadesi sona erdi ve kod 21. satırda devam etti. Her iki durumda da sonu aynıdır. finally block çalıştırılır ve deneme ifadesinden sonra yürütme devam eder. **English:** The exam will try to trick you with missing clauses or clauses in the wrong order. Do you see why the following do or do not compile? **Türkçe:** Sınav sizi yanlış sırayla eksik maddeler veya maddelerle kandırmaya çalışacaktır. Aşağıdakilerin neden derlendiğini veya derlenmediğini görüyor musunuz?

```
try { // DOES NOT COMPILE
    fall();
} finally {
    System.out.println("all better");
} catch (Exception e) {
    System.out.println("get up");
}
```

English: 32: Türkçe: 32:

```
try { // DOES NOT COMPILE
    fall();
}
```

English: 36: Türkçe: 36:

```
try {
    fall();
} finally {
    System.out.println("all better");
}
```

English: The first example (lines 25–31) does not compile because the catch and finally blocks are in the wrong order. The second example (lines 33–35) does not compile because there must be a catch or finally block. The third example (lines 37–41) is just fine. The catch block is not required if finally is present.

Türkçe: İlk örnek (hat 25-31) derlemez çünkü yakalama ve finally blocks yanlış sıradadır. İkinci örnek (33-35 satırları) derlemez, çünkü bir yakalama veya finally block olmalıdır. Üçüncü örnek (37-41) gayet iyi. Sonunda mevcutsa catch block gerekli değildir. **English:** Most of the examples you encounter on the exam with finally are going to look contrived. For example, you'll get asked questions such as what this code outputs: **Türkçe:** Sonunda sınavda karşılaştığınız örneklerin çoğu uydurma görünecek. Örneğin, bu kodun çıktıları gibi sorular sorulur:

```
public static void main(String[] unused) {
    StringBuilder sb = new StringBuilder();
    try {
        sb.append("t");
    } catch (Exception e) {
        sb.append("c");
    } finally {
        sb.append("f");
    }
    sb.append("a");
    System.out.print(sb.toString());
}
```

English: The answer is tfa. The try block is executed. Since no exception is thrown, Java goes straight to the finally block. Then the code after the try statement is run. We know that this is a silly example, but you can expect to see examples like this on the exam. **Türkçe:** Cevabı tfa. try block çalıştırılır. Herhangi bir istisna atılmadığından, Java doğrudan finally block adresine gider. Ardından, deneme ifadesinden sonraki kod çalıştırılır. Bunun aptalca bir örnek olduğunu biliyoruz, ancak sınavda bunun gibi örnekler görmeyi bekleyebilirsiniz. **English:** There is one additional rule you should know for finally blocks. If a try statement with a finally block is entered, then the finally block will always be executed, regardless of whether the code completes successfully. Take a look at the following goHome() method. Assuming an exception may or may not be thrown on line 14, what are the possible values that this method could print? Also, what would the return value be in each case? **Türkçe:** finally blocks için bilmeniz gereken bir ek kural vardır. finally block ile bir deneme ifadesi girilirse, kodun başarılı bir şekilde tamamlanıp tamamlanmadığına bakılmaksızın finally block her zaman çalıştırılır. Aşağıdaki goHome() yöntemine bir göz atın. Bir istisnanın 14. hatta atılabileceğini veya atılamayacağını varsayarsak, bu yöntemin yazdırabileceği olası değerler nelerdir? Ayrıca, her durumda geri dönüş değeri ne olur?

```
int goHome() {
    try {
        // Optionally throw an exception here
        System.out.print("1");
        return - 1;
    } catch (Exception e) {
        System.out.print("2");
        return - 2;
    }
}
```

```
    } finally {
        System.out.print("3");
        return - 3;
    }
}
```

English: If an exception is not thrown on line 14, then line 15 will be executed, printing 1. Before the method returns, though, the finally block is executed, printing 3. If an exception is thrown, then lines 15 and 16 will be skipped and lines 17–19 will be executed, printing 2, followed by 3 from the finally block. While the first value printed may differ, the method always prints 3 last since it's in the finally block.

Türkçe: 14'üncü hatta bir istisna atılmazsa, 15'inci satır çalıştırılır, 1'i yazdırır. Yöntem geri dönmeye önce, finally block çalıştırılır, baskı 3. Bir istisna atılırsa, 15 ve 16 satırları atlanır ve 17 satırları çalıştırılır, 2 yazdırılır ve ardından finally block'den 3 çıkarılır. Basılan ilk değer farklılık gösterse de, yöntem finally block olduğu için her zaman son 3 yazdırır. **English:** What is the return value of the goHome() method? In this case, it's always --- 3. Because the finally block is executed shortly before the method completes, it interrupts the return statement from inside both the try and catch blocks. **Türkçe:** goHome() yönteminin dönüş değeri nedir? Bu durumda, her zaman --- 3. finally block yöntemin tamamlanmasından kısa bir süre önce çalıştırıldığı için, hem denemenin içinden hem de catch blocks içinden geri bildirimin kesilmesini sağlar. **English:** For the exam, you need to remember that a finally block will always be executed. That said, it may not complete successfully. Take a look at the following code snippet. What would happen if info was null on line 32? **Türkçe:** Sınav için her zaman bir finally block çalıştırılacağını hatırlamanız gerekir. Bununla birlikte, başarılı bir şekilde tamamlanamayabilir. Aşağıdaki kod snippet'ine bir göz atın. Bilgi 32. hatta null olsaydı ne olurdu?

```
} finally {  
info.printDetails();  
System.out.print("Exiting");  
return "zoo";  
}
```

English: If info was null, then the finally block would be executed, but it would stop on line 32 and throw a NullPointerException. Lines 33 and 34 would not be executed. In this example, you see that while a finally block will always be executed, it may not finish. **Türkçe:** Eğer bilgi null olsaydı, finally block çalıştırılırdı, ancak 32. hatta durur ve NullPointerException atardı. 33 ve 34 numaralı hatlar idam edilmeyecekti. Bu örnekte, bir finally block her zaman çalıştırılacak olsa da, bitmeyebilir. **English:** System.exit() **Türkçe:** System.exit() **English:** There is one exception to “the finally block will always be executed” rule: Java defines a **Türkçe:** “finally block her zaman çalıştırılacak” kuralının bir istisnası vardır: Java bir

method that you call as System.exit(). It takes an integer parameter that represents the

English: status code that is returned. **Türkçe:** iade edilen durum kodu.

```
try {  
System.exit(0);  
} finally {  
System.out.print("Never going to get here"); // Not printed  
}  
  
System.exit() tells Java, “Stop. End the program right now. Do not pass Go. Do not col-  
lect $200.” When System.exit() is called in the try or catch block, the finally block
```

English: does not run. **Türkçe:** koşturmuyor.

Automating Resource Management

English: Often, your application works with files, databases, and various connection objects. Commonly, these external data sources are referred to as resources. In many cases, you open a connection to the resource, whether it's over the network or within a file system. You then read/write the data you want. Finally, you close the resource to indicate that you are done with it. **Türkçe:** Çoğu zaman, uygulamanız dosyalar, veritabanları ve çeşitli bağlantı nesneleri ile çalışır. Genel olarak, bu dış veri kaynakları kaynaklar olarak adlandırılır. Birçok durumda, ağ üzerinden veya bir dosya sistemi içinde olsun, kaynağa bir bağlantı açarsınız. Daha sonra istediğiniz verileri okur/yazın. Son olarak, onunla işiniz bittiğini belirtmek için kaynağı kapatırsınız. **English:** What happens if you don't close a resource when you are done with it? In short, a lot of bad things could happen. If you are connecting to a database, you could use up all available connections, meaning no one can talk to the database until you release your connections. Although you commonly hear about memory leaks causing programs to fail, a resource leak is just as bad and occurs when a program fails to release its connections to a resource, resulting in the resource becoming inaccessible. This could mean your program can no longer talk to the database— or, even worse, all programs are unable to reach the database! **Türkçe:** Bir kaynağı onunla işiniz bittiğinde kapatmazsanız ne olur? short'da birçok kötü şey olabilir. Bir veritabanına bağlanıyorsanız, mevcut tüm bağlantıları kullanabilirsiniz, yani bağlantılarınızı serbest bırakana kadar hiç kimse veritabanıyla konuşamaz. Programların başarısız olmasına neden olan bellek sızıntılarını sık sık duymanıza rağmen, bir kaynak sızıntısı da aynı derecede kötüdür ve bir program bir kaynağa bağlantılarını serbest bırakmadığında ortaya çıkar, bu da kaynağın erişilemez hale gelmesine neden olur. Bu, programınızın artık veritabanıyla konuşamayacağı anlamına gelebilir - veya daha da kötüsü, tüm programlar veritabanına ulaşamaz! **English:** For the exam, a resource is typically a file or database that requires some kind of stream or connection to read or write data. In Chapter 14 and Chapter 15, you create numerous resources that will need to be closed when you are finished with them. **Türkçe:** Sınav için, bir kaynak tipik olarak bir tür stream veya veri okumak veya yazmak için bağlantı gerektiren bir dosya veya veritabanıdır. Bölüm 14 ve Bölüm 15'te, onlarla işiniz bittiğinde kapatılması gereken çok sayıda kaynak oluşturursunuz.

Introducing Try-with-Resources

English: Let's take a look at a method that opens a file, reads the data, and closes it: **Türkçe:** Bir dosyayı açan, verileri okuyan ve kapatan bir yöntemle bir göz atalım:

```
public void readFile(String file) {
    FileInputStream is = null;
    try {
        is = new FileInputStream("myfile.txt");
        // Read file data
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        if(is != null) {
            try {
                is.close();
            }
        }
    }
}
```

English: 15: **Türkçe:** 15:

```
} catch (IOException e2) {  
e2.printStackTrace();  
}  
}  
}  
}
```

English: Wow, that's a long method! Why do we have two try and catch blocks? Well, lines 7 and 14 both include checked `IOException` calls, and those need to be caught in the method or rethrown by the method. Half the lines of code in this method are just closing a resource. And the more resources you have, the longer code like this becomes. For example, you may have multiple resources that need to be closed in a particular order. You also don't want an exception caused by closing one resource to prevent the closing of another resource. **Türkçe:** Vay canına, bu bir long yöntem! Neden iki deneme ve catch blocks var? 7 ve 14 satırlarının her ikisi de kontrol edilen `IOException` çağrılarını içerir ve bunların yöntemde yakalanması veya yöntemde göre yeniden atanması gerekir. Bu yöntemdeki kod satırlarının yarısı sadece bir kaynağı kapatıyor. Ve ne kadar çok kaynağa sahip olursanız, bunun gibi kodlar o kadar uzun olur. Örneğin, belirli bir sırayla kapatılması gereken birden fazla kaynağa sahip olabilirsiniz. Başka bir kaynağın kapanmasını önlemek için bir kaynağın kapatılmasının neden olduğu bir istisna da istemezsiniz. **English:** To solve this, Java includes the try-with-resources statement to automatically close all resources opened in a try clause. This feature is also known as automatic resource management, because Java automatically takes care of the closing. **Türkçe:** Bunu çözmek için Java, bir deneme maddesinde açılan tüm kaynakları otomatik olarak kapatmak için try-with-resources statement içerir. Bu özellik otomatik kaynak yönetimi olarak da bilinir, çünkü Java otomatik olarak kapanışla ilgilenir. **English:** Let's take a look at our same example using a try-with-resources statement: **Türkçe:** try-with-resources statement kullanarak aynı örneğimize bir göz atalım:

```
public void readFile(String file) {  
try (FileInputStream is = new FileInputStream("myfile.txt")) {  
// Read file data  
} catch (IOException e) {  
e.printStackTrace();  
}  
}
```

English: Functionally, they are similar, but our new version has half as many lines. More importantly, though, by using a try-with-resources statement, we guarantee that as soon as a connection passes out of scope, Java will attempt to close it within the same method. **Türkçe:** İşlevsel olarak, benzerler, ancak yeni versiyonumuzun yarısı kadar çizgisi var. Daha da önemlisi, bir try-with-resources statement kullanarak, bir bağlantı kapsam dışına çıkar çıkmaz, Java'nın aynı yöntem içinde kapatmaya çalışacağını garanti ediyoruz.

English: Behind the scenes, the compiler replaces a try-with-resources block with a try and finally block. We refer to this “hidden” finally block as an implicit finally block since it is created and used by the compiler automatically. You can still create a programmer-defined finally block when using a try-with-resources statement; just be aware that the implicit one will be called first. **Türkçe:** Sahnelerin arkasında, derleyici bir try-with-resources bloğunu bir deneme ve finally block ile değiştirir. Bu "gizli" finally block, derleyici tarafından otomatik olarak oluşturulduğu ve kullanıldığı için örtülü bir finally block olarak adlandırıyoruz. try-with-resources statement kullanırken yine de bir programcı tanımlı finally block oluşturabilirsiniz; sadece örtülü olanın ilk olarak adlandırılacağını unutmayın. **English:** Unlike garbage collection, resources are not automatically closed when they go out of scope. Therefore, it is recommended that you close resources in the same block of code that opens them. By using a try-with-resources statement to open all your resources, this happens automatically. **Türkçe:** Çöp collection 'den farklı olarak, kaynaklar kapsam dışına çıktıklarında otomatik olarak kapatılmaz. Bu nedenle, kaynakları onları açan aynı kod bloğunda kapatmanız önerilir. Tüm kaynaklarınızı açmak için bir try-with-resources statement kullanarak, bu otomatik olarak gerçekleşir.

Basics of Try-with-Resources

English: Figure 11.5 shows what a try-with-resources statement looks like. Notice that one or more resources can be opened in the try clause. When multiple resources are opened, they are closed in the reverse of the order in which they were created. Also, notice that parentheses are used to list those resources, and semicolons are used to separate the declarations. This works just like declaring multiple indexes in a for loop. **Türkçe:** Şekil 11.5, bir try-with-resources statement nin neye benzediğini gösterir. Deneme maddesinde bir veya daha fazla kaynağın açılabilmesine dikkat edin. Birden fazla kaynak açıldığında, yaratıldıkları sıranın tersine kapatılırlar. Ayrıca, parantezlerin list bu kaynaklar için kullanıldığına dikkat edin ve deklarasyonları ayırmak için noktalı virgüller kullanılır. Bu, aynı döngü için birden fazla indeksi beyan etmek gibi çalışır.

English: FIGURE 11.5 The syntax of a basic try-with-resources statement Required semicolon
Resources between resources **Türkçe:** FIGURE 11.5 Temel bir try-with-resources statement sözdizimi
Kaynaklar arasında gerekli virgül kaynakları

```
try (var in = new FileInputStream("data.txt");
    var out = new FileOutputStream("output.txt");) {
    // Protected code
```

English: Optional semicolon **Türkçe:** Optional virgül

```
} catch (IOException e) {
```

English: Resources are Optional catch and **Türkçe:** Kaynaklar Optional yakalama ve

```
// Exception handler
```

English: closed here in finally clauses reverse order. **Türkçe:** Burada son olarak ters düzen maddesi ile kapatılmıştır.

```
} finally {  
// finally block  
}
```

English: What happened to the catch block in Figure 11.5? Well, it turns out a catch block is optional with a try-with-resources statement. For example, we can rewrite the previous `readFile()` example so that the method declares the exception to make it even shorter: **Türkçe:** Şekil 11.5'teki catch block'ye ne oldu? Görünüşe göre bir yakalama bloğu Optional. Kaynakla deneme ifadesiyle. Örneğin, önceki `readFile()` örneğini yeniden yazabiliriz, böylece yöntem istisnayı daha da kısaltacak şekilde ilan eder:

```
public void readFile(String file) throws IOException {  
try (FileInputStream is = new FileInputStream("myfile.txt")) {  
// Read file data  
}  
}
```

English: Earlier in the chapter, you learned that a try statement must have one or more catch blocks or a finally block. A try-with-resources statement differs from a try statement in that neither of these is required, although a developer may add both. For the exam, you need to know that the implicit finally block runs before any programmer-coded ones. **Türkçe:** Bölümün başlarında, bir deneme ifadesinin bir veya daha fazla catch blocks veya finally block olması gerektiğini öğrendiniz. Bir try-with-resources statement, bir geliştirici her ikisini de ekleyebilse de, bunların hiçbirine gerek olmadığı için bir deneme ifadesinden farklıdır. Sınav için, örtülü finally block'nin herhangi bir programcı kodlu olanlardan önce çalıştığını bilmeniz gerekir.

Constructing Try-with-Resources Statements

English: Only classes that implement the `AutoCloseable` interface can be used in a try-with-resources statement. For example, the following does not compile as `String` does not implement the `AutoCloseable` interface: **Türkçe:** Sadece `AutoCloseable` arayüzünü uygulayan sınıflar try-with-resources statement içinde kullanılabilir. Örneğin, `String` `AutoCloseable` arayüzünü uygulamadığı için aşağıdakiler derlenmez:

```
try (String reptile = "lizard") {}
```

English: Inheriting `AutoCloseable` requires implementing a compatible `close()` method. **Türkçe:** `AutoCloseable`'in kalıtılması, uyumlu bir `close()` yönteminin uygulanmasını gerektirir.

```
interface AutoCloseable {  
public void close() throws Exception;  
}
```

English: From your studies of method overriding, this means that the implemented version of `close()` can choose to throw `Exception` or a subclass or not throw any exceptions at all. **Türkçe:** Metod overriding çalışmalarınızdan, bu, `close()` uygulamasının uygulanmış sürümünün `Exception` veya bir alt sınıfı atmayı seçebileceği veya herhangi bir istisna atmayacağı anlamına gelir.

English: Throughout the rest of this section, we use the following custom resource class that simply prints a message when the `close()` method is called: **Türkçe:** Bu bölümün geri kalanı boyunca, `close()` yöntemi çağırıldığında bir mesajı basitçe yazdıran aşağıdaki özel kaynak sınıfını kullanıyoruz:

```
public class MyFileClass implements AutoCloseable {
    private final int num;
    public MyFileClass(int num) { this.num = num; }
    @Override public void close() {
        System.out.println("Closing: " + num);
    }
}
```

English: In Chapter 14, you encounter resources that implement `Closeable` rather than `AutoCloseable`. Since `Closeable` extends `AutoCloseable`, they are both supported in try-with-resources state-ments. The only difference between the two is that `Closeable`'s `close()` method declares `IOException`, while `AutoCloseable`'s **Türkçe:** Bölüm 14'te, `AutoCloseable` yerine `Yakınlaştırılabilir`'i uygulayan kaynaklarla karşılaşsınız. `Closeable` `AutoCloseable` uzandığından, her ikisi de try-with-resources state-ments içinde desteklenir. İkisi arasındaki tek fark `Closeable`'in `close()` yönteminin `IOException`, `AutoCloseable`'in ise

```
close() method declares Exception.
```

Declaring Resources

English: While try-with-resources does support declaring multiple variables, each variable must be declared in a separate statement. For example, the following do not compile: **Türkçe:** try-with-resources birden fazla değişkenin ilan edilmesini desteklerken, her değişken ayrı bir ifadeyle ilan edilmelidir. Örneğin, aşağıdakiler derlemez:

```
try (MyFileClass is = new MyFileClass(1), // DOES NOT COMPILE
    os = new MyFileClass(2)) {
}
try (MyFileClass ab = new MyFileClass(1), // DOES NOT COMPILE
    MyFileClass cd = new MyFileClass(2)) {
}
```

English: The first example does not compile because it is missing the data type, and it uses a comma (,) instead of a semicolon (;). The second example does not compile because it also uses a comma (,) instead of a semicolon (;). Each resource must include the data type and be separated by a semicolon (;). **Türkçe:** İlk örnek veri türünü eksik olduğu için derlemez ve virgül (,) yerine virgül (;) kullanır. İkinci örnek derlenmez çünkü virgül (,) yerine virgül (;) kullanır. Her kaynak veri türünü içermeli ve bir virgül (;) ile ayrılmalıdır. **English:** You can declare a resource using `var` as the data type in a try-with-resources statement, since resources are local variables. **Türkçe:** `var` kullanarak bir kaynağı try-with-resources statement veri türü olarak beyan edebilirsiniz, çünkü kaynaklar yerel değişkenlerdir.

```
try (var f = new BufferedInputStream(new FileInputStream("it.txt"))) {  
    // Process file  
}
```

English: Declaring resources is a common situation where using var is quite helpful, as it shortens the already long line of code. **Türkçe:** Kaynakların açıklanması, var kodunun zaten long satırını kısalttığı için, var kullanımının oldukça yararlı olduğu yaygın bir durumdur.

Scope of Try-with-Resources

English: The resources created in the try clause are in scope only within the try block. This is another way to remember that the implicit finally runs before any catch/finally blocks that you code yourself. The implicit close has run already, and the resource is no longer available. Do you see why lines 6 and 8 don't compile in this example? **Türkçe:** Deneme maddesinde oluşturulan kaynaklar yalnızca try block kapsamındadır. Bu, örtülünün sonunda kendinizi kodladığınız herhangi bir yakalama / finally blocks öncesinde çalıştığını hatırlamanın başka bir yoludur. Örtülü yakın zaten çalıştı ve kaynak artık mevcut değil. 6 ve 8 satırlarının neden bu örnekte derlenmediğini görüyor musunuz?

```
try (Scanner s = new Scanner(System.in)) {  
    s.nextLine();  
} catch (Exception e) {  
    s.nextInt(); // DOES NOT COMPILE  
} finally {  
    s.nextInt(); // DOES NOT COMPILE  
}
```

English: The problem is that Scanner has gone out of scope at the end of the try clause. Lines 6 and 8 do not have access to it. This is a nice feature. You can't accidentally use an object that has been closed. In a traditional try statement, the variable has to be declared before the try statement so that both the try and finally blocks can access it, which has the unpleasant side effect of making the variable in scope for the rest of the method, just inviting you to call it by accident. **Türkçe:** Sorun, Scanner'ın deneme maddesinin sonunda kapsam dışına çıkmasıdır. 6 ve 8 numaralı hatlar buna erişemiyor. Bu güzel bir özellik. Yanlışlıkla kapatılmış bir nesneyi kullanamazsınız. Geleneksel bir deneme ifadesinde, değişkenin deneme ifadesinden önce ilan edilmesi gerekir, böylece hem deneme hem de finally blocks buna erişebilir, bu da değişkeni yöntemin geri kalanı için kapsam içinde yapmanın hoş olmayan yan etkisine sahiptir, sadece sizi yanlışlıkla çağırmaya davet eder.

Following Order of Operations

English: When working with try-with-resources statements, it is important to know that resources are closed in the reverse of the order in which they are created. Using our custom MyFileClass, can you figure out what this method prints? **Türkçe:** try-with-resources statements ile çalışırken, kaynakların oluşturuldukları sıranın tersine kapatıldığını bilmek önemlidir. Özel MyFileClass'ımızı kullanarak, bu yöntemin ne baskı yaptığını bulabilir misiniz?

```
public static void main(String... xyz) {
    try (MyFileClass bookReader = new MyFileClass(1);
        MyFileClass movieReader = new MyFileClass(2)) {
        System.out.println("Try Block");
        throw new RuntimeException();
    } catch (Exception e) {
        System.out.println("Catch Block");
    } finally {
        System.out.println("Finally Block");
    }
}
```

English: The output is as follows: **Türkçe:** Çıktı aşağıdaki gibidir: **English:** Try Block Closing: 2 Closing: 1 Catch Block Finally Block **Türkçe:** Try Block Kapanış: 2 Kapanış: 1 Catch Block Finally Block

English: For the exam, make sure you understand why the method prints the statements in this order. Remember, the resources are closed in the reverse of the order in which they are declared, and the implicit finally is executed before the programmer-defined finally. **Türkçe:** Sınav için, yöntemin ifadeleri neden bu sırayla yazdığını anladığınızdan emin olun. Unutmayın, kaynaklar ilan edildikleri sıranın tersine kapatılır ve örtük sonunda programcı tanımlı olandan önce yürütülür.

Applying Effectively Final

English: While resources are often created in the try-with-resources statement, it is possible to declare them ahead of time, provided they are marked final or effectively final. The syntax uses the resource name in place of the resource declaration, separated by a semicolon (;). Let's try another example: **Türkçe:** Kaynaklar genellikle try-with-resources statement içinde oluşturulurken, final veya effectively final işaretli olmaları koşuluyla, bunları önceden beyan etmek mümkündür. Sözdizimi, bir virgül (;) ile ayrılmış kaynak bildirimi yerine kaynak adını kullanır. Başka bir örnek deneyelim:

```
public static void main(String... xyz) {
    final var bookReader = new MyFileClass(4);
    MyFileClass movieReader = new MyFileClass(5);
    try (bookReader;
```

English: 15: **Türkçe:** 15:

```
var tvReader = new MyFileClass(6);
movieReader) {
    System.out.println("Try Block");
} finally {
    System.out.println("Finally Block");
}
}
```

English: Let's take this one line at a time. Line 12 declares a final variable bookReader, while line 13 declares an effectively final variable movieReader. Both of these resources can be used in a try-with-resources statement. We know movieReader is effectively final because it is a local variable that is assigned a value only once. Remember, the test for effectively final is that if we insert the final keyword when the variable is declared, the code still compiles. **Türkçe:** Bu tek çizgiyi teker teker ele alalım. 12. satır son değişkeni bookReader, 13. satır ise effectively final değişkeni movieReader olarak ilan eder. Bu kaynakların her ikisi de bir try-with-resources statement içinde kullanılabilir. movieReader effectively final olduğunu biliyoruz çünkü sadece bir kez bir değer atanmış yerel bir değişkendir. Unutmayın, effectively final testi, değişken ilan edildiğinde son anahtar kelimeyi eklersek, kod hala derlenir. **English:** Lines 14 and 16 use the new syntax to declare resources in a try-with-resources statement, using just the variable name and separating the resources with a semicolon (;). Line 15 uses the normal syntax for declaring a new resource within the try clause. **Türkçe:** Satır 14 ve 16, kaynakları bir try-with-resources statement içinde beyan etmek için yeni sözdizimi kullanır, sadece değişken adı kullanır ve kaynakları bir virgül (;) ile ayırır. 15. satır, try maddesi içinde yeni bir kaynak beyan etmek için normal sözdizimi kullanır. **English:** On execution, the code prints the following: **Türkçe:** Çalıştırma sırasında, kod aşağıdakileri yazdırır: **English:** Try Block Closing: 5 Closing: 6 Closing: 4 Finally Block If you come across a question on the exam that uses a try-with-resources statement with a variable not declared in the try clause, make sure it is effectively final. For example, the following does not compile: **Türkçe:** Try Block Kapanış: 5 Kapanış: 6 Kapanış: 4 Finally Block Sınavda try-with-resources statement değişkenini kullanan ve deneme maddesinde belirtilmeyen bir soruyla karşılaşırsanız effectively final olduğundan emin olun. Örneğin, aşağıdakiler derlemez:

```
var writer = Files.newBufferedWriter(path);
try (writer) { // DOES NOT COMPILE
```

```
writer.append("Welcome to the zoo!");
}
writer = null;
```

English: The writer variable is reassigned on line 35, resulting in the compiler not considering it effectively final. Since it is not an effectively final variable, it cannot be used in a try-with-resources statement on line 32. **Türkçe:** Yazar değişkeni 35. satırda yeniden atanır, bu da derleyicinin effectively final dikkate almamasına neden olur. effectively final değişkeni olmadığından, 32. satırdaki bir try-with-resources ifadesinde kullanılamaz. **English:** The other place the exam might try to trick you is accessing a resource after it has been closed. Consider the following: **Türkçe:** Sınavın sizi kandırmaya çalışabileceği diğer yer, kapatıldıktan sonra bir kaynağa erişmektir. Aşağıdakileri göz önünde bulundurun:

```
var writer = Files.newBufferedWriter(path);
writer.append("This write is permitted but a really bad idea!");
try (writer) {
    writer.append("Welcome to the zoo!");
}
writer.append("This write will fail!"); // IOException
```

English: This code compiles but throws an exception on line 46 with the message Stream closed. While it is possible to write to the resource before the try-with-resources statement, it is not afterward. **Türkçe:** Bu kod, Stream kapalı mesajıyla 46. hatta bir istisna oluşturur ancak atar. try-with-resources statement'dan önce kaynağa yazmak mümkün olsa da, daha sonra değil.

Understanding Suppressed Exceptions

English: We conclude our discussion of exceptions with probably the most confusing topic: suppressed exceptions. What happens if the `close()` method throws an exception? Let's try an illustrative example:

Türkçe: İstisnalar konusundaki tartışmamızı muhtemelen en kafa karıştırıcı konu ile sonlandırıyoruz: suppressed exceptions. `close()` yöntemi bir istisna atarsa ne olur? Bir örnek verelim:

```
public class TurkeyCage implements AutoCloseable {
    public void close() {
        System.out.println("Close gate");
    }
    public static void main(String[] args) {
        try (var t = new TurkeyCage()) {
            System.out.println("Put turkeys in");
        }
    }
}
```

English: If the `TurkeyCage` doesn't close, the turkeys could all escape. Clearly, we need to handle such a condition. We already know that the resources are closed before any programmer-coded catch blocks are run. This means we can catch the exception thrown by `close()` if we want to. Alternatively, we can allow the caller to deal with it. **Türkçe:** Eğer `TurkeyCage` kapanmazsa, hindilerin hepsi kaçabilir. Açıkçası, böyle bir durumu halletmemiz gerekiyor. `catch` blocks kodlu herhangi bir programcı çalıştırılmadan önce kaynakların kapalı olduğunu zaten biliyoruz. Bu, istersek `close()` tarafından atılan istisnayı yakalayabileceğimiz anlamına gelir. Alternatif olarak, arayanın bununla başa çıkmasına izin verebiliriz.

English: Let's expand our example with a new `JammedTurkeyCage` implementation, shown here:

Türkçe: Örneğimizi, burada gösterilen yeni bir `JammedTurkeyCage` uygulamasıyla genişletelim:

```
public class JammedTurkeyCage implements AutoCloseable {
    public void close() throws IllegalStateException {
        throw new IllegalStateException("Cage door does not close");
    }
    public static void main(String[] args) {
        try (JammedTurkeyCage t = new JammedTurkeyCage()) {
            System.out.println("Put turkeys in");
        } catch (IllegalStateException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

English: The close() method is automatically called by try-with-resources. It throws an exception, which is caught by our catch block and prints the following: Caught: Cage door does not close This seems reasonable enough. What happens if the try block also throws an exception? When multiple exceptions are thrown, all but the first are called suppressed exceptions. The idea is that Java treats the first exception as the primary one and tacks on any that come up while automatically closing. **Türkçe:** close() yöntemi try-with-resources tarafından otomatik olarak çağrılır. catch block tarafından yakalanan ve aşağıdakileri yazdıran bir istisna atar: Yakalanma: Kafes kapısı kapanmaz Bu yeterince makul görünüyor. try block ayrıca bir istisna atarsa ne olur? Birden fazla istisna atıldığında, ilk hariç hepsi suppressed exceptions olarak adlandırılır. Fikir, Java ilk istisnayı birincil istisna olarak ele alır ve otomatik olarak kapanırken ortaya çıkan herhangi bir şeyde tacks. **English:** What do you think the following implementation of our main() method outputs? **Türkçe:** main() yöntem çıktılarımızın aşağıdaki uygulaması hakkında ne düşünüyorsunuz?

```
public static void main(String[] args) {
    try (JammedTurkeyCage t = new JammedTurkeyCage()) {
```

English: 7: **Türkçe:** 7:

```
        throw new IllegalStateException("Turkeys ran off");
    } catch (IllegalStateException e) {
        System.out.println("Caught: " + e.getMessage());
        for (Throwable t: e.getSuppressed())
            System.out.println("Suppressed: "+t.getMessage());
    }
}
```

English: Line 7 throws the primary exception. At this point, the try clause ends, and Java automatically calls the close() method. Line 3 of JammedTurkeyCage throws an IllegalStateException, which is added as a suppressed exception. Then line 8 catches the primary exception. Line 9 prints the message for the primary exception. Lines 10 and 11 iterate through any suppressed exceptions and print them. The program prints the following: **Türkçe:** 7. satır primary exception atar. Bu noktada, deneme maddesi sona erer ve Java otomatik olarak close() yöntemini çağırır. JammedTurkeyCage'in Line 3'ü, suppressed exception olarak eklenen bir IllegalStateException atar. Daha sonra 8. satır primary exception'i yakalar. Satır 9, primary exception için mesajı yazdırır. 10 ve 11 satırları herhangi bir suppressed exceptions ile yinelenir ve yazdırır. Program aşağıdakileri yazdırır: **English:** Caught: Turkeys ran off Suppressed: Cage door does not close **Türkçe:** Yakalandı: Türkler Bastı: Kafes kapısı kapanmadı

English: Keep in mind that the catch block looks for matches on the primary exception. What do you think this code prints? **Türkçe:** catch block uygulamasının primary exception üzerindeki maçları aradığını unutmayın. Sence bu kod parmak izi ne?

```
public static void main(String[] args) {
    try (JammedTurkeyCage t = new JammedTurkeyCage()) {
```

English: 7: **Türkçe:** 7:

```
throw new RuntimeException("Turkeys ran off");
} catch (IllegalStateException e) {
System.out.println("caught: " + e.getMessage());
}
}
```

English: Line 7 again throws the primary exception. Java calls the close() method and adds a suppressed exception. Line 8 would catch the IllegalStateException. However, we don't have one of those. The primary exception is a RuntimeException. Since this does not match the catch clause, the exception is thrown to the caller. Eventually, the main() method would output something like the following: **Türkçe:** 7. satır yine primary exception atar. Java close() yöntemini çağırır ve suppressed exception ekler. 8. satır IllegalStateException'i yakalayacaktı. Ancak, bunlardan birine sahip değiliz. primary exception bir RuntimeException dir. Bu, yakalama maddesiyle uyuşmadığından, istisna arayan kişiye atılır. Sonunda, main() yöntemi aşağıdaki gibi bir şey çıkaracaktı: **English:** Exception in thread "main" java.lang.RuntimeException: Turkeys ran off **Türkçe:** Exception iş parçacığında "ana" java.lang.RuntimeException: Türkiyeler kaçtı

```
at JammedTurkeyCage.main(JammedTurkeyCage.java:7)
```

English: Suppressed: java.lang.IllegalStateException: **Türkçe:** Bastırılmış:
java.lang.IllegalStateException: **English:** Cage door does not close **Türkçe:** Kafes kapısı kapanmaz

```
at JammedTurkeyCage.close(JammedTurkeyCage.java:3)
at JammedTurkeyCage.main(JammedTurkeyCage.java:8)
```

English: Java remembers the suppressed exceptions that go with a primary exception even if we don't handle them in the code. **Türkçe:** Java kodda ele almasak bile primary exception ile giden suppressed exceptions'ı hatırlar. **English:** If more than two resources throw an exception, the first one to be thrown becomes the primary exception, and the rest are grouped as suppressed exceptions. And since resources are closed in the reverse of the order in which they are declared, the primary exception will be on the last declared resource that throws an exception. **Türkçe:** İki'den fazla kaynak bir istisna atarsa, atılan ilk primary exception olur ve geri kalanı suppressed exceptions olarak gruplandırılır. Ve kaynaklar, ilan edildikleri sıranın tersine kapatıldığından, primary exception bir istisna atan son ilan edilen kaynaktan olacaktır. **English:** Keep in mind that suppressed exceptions apply only to exceptions thrown in the try clause. The following example does not throw a suppressed exception: **Türkçe:** suppressed exceptions uygulamasının yalnızca deneme maddesinde belirtilen istisnalar için geçerli olduğunu unutmayın. Aşağıdaki örnek bir suppressed exception atmıyor:

```
public static void main(String[] args) {
try (JammedTurkeyCage t = new JammedTurkeyCage()) {
```

English: 7: **Türkçe:** 7:

```
throw new IllegalStateException("Turkeys ran off");
} finally {
```

English: 9: **Türkçe:** 9:

```
throw new RuntimeException("and we couldn't find them");
}
}
```

English: Line 7 throws an exception. Then Java tries to close the resource and adds a suppressed exception to it. Now we have a problem. The finally block runs after all this. Since line 9 **Türkçe:** 7. hat bir istisna yaratıyor. Ardından Java kaynağı kapatmaya çalışır ve buna suppressed exception ekler. Şimdi bir sorunuz var. finally block tüm bunlardan sonra çalışır. 9. satırdan beri

English: also throws an exception, the previous exception from line 7 is lost, with the code printing the following: **Türkçe:** Ayrıca bir istisna atar, satır 7'den önceki istisna kaybolur, kod aşağıdakileri yazdırır:

English: Exception in thread "main" java.lang.RuntimeException: **Türkçe:** Exception iş parçasığında "ana" java.lang.RuntimeException: **English:** and we couldn't find them **Türkçe:** ve onları bulamadık

```
at JammedTurkeyCage.main(JammedTurkeyCage.java:9)
```

English: This has always been and continues to be bad programming practice. We don't want to lose exceptions! Although out of scope for the exam, the reason for this has to do with backward compatibility. This behavior existed before automatic resource management was added. **Türkçe:** Bu her zaman kötü bir programlama uygulaması olmuştur ve olmaya devam etmektedir. İstisnaları kaybetmek istemiyoruz! Sınav için kapsam dışında olmasına rağmen, bunun nedeni geriye dönük uyumlulukla ilgilidir. Bu davranış otomatik kaynak yönetimi eklenmeden önce de vardı.

Formatting Values

English: We now shift gears a bit and talk about how to format data for users. In this section, we're going to be working with numbers, dates, and times. This is especially important in the next section when we expand customization to different languages and locales. You may want to review Chapter 4, "Core APIs," if you need a refresher on creating various date/time objects. **Türkçe:** Şimdi vitesleri biraz kaydırıyoruz ve kullanıcılar için verileri nasıl biçimlendireceğimizden bahsediyoruz. Bu bölümde sayılar, tarihler ve saatlerle çalışacağız. Bu, özelleştirmeyi farklı dillere ve locales genişlettiğimizde özellikle bir sonraki bölümde önemlidir. Çeşitli tarih/saat nesneleri oluşturmak için bir yenilemeye ihtiyacınız varsa, Bölüm 4, "Core API'leri"ni gözden geçirmek isteyebilirsiniz.

Formatting Numbers

English: In Chapter 4, you saw how to control the output of a number using the `String.format()` method. That's useful for simple stuff, but sometimes you need finer-grained control. With that, we introduce the `NumberFormat` interface, which has two commonly used methods: **Türkçe:** Bölüm 4'te, `String.format()` yöntemini kullanarak bir sayının çıktısını nasıl kontrol edeceğinizi gördünüz. Bu basit şeyler için yararlıdır, ancak bazen daha ince taneli kontrole ihtiyacınız vardır. Bununla, yaygın olarak kullanılan iki yöntemle sahip olan `NumberFormat` arayüzünü tanıtırız:

```
public final String format(double number)
public final String format(long number)
```

English: Since NumberFormat is an interface, we need the concrete DecimalFormat class to use it. It includes a constructor that takes a pattern String: **Türkçe:** NumberFormat bir arayüz olduğundan, kullanmak için somut DecimalFormat sınıfına ihtiyacımız var. String deseni alan bir constructor içerir:

```
public DecimalFormat(String pattern)
```

English: The patterns can get quite complex. But luckily, for the exam you only need to know about two formatting characters, shown in Table 11.5. **Türkçe:** Desenler oldukça karmaşık hale gelebilir. Ama neyse ki, sınav için sadece Tablo 11.5'te gösterilen iki biçimlendirme karakteri hakkında bilgi sahibi olmanız gerekir. **English:** TABLE 11.5 DecimalFormat symbols Symbol Meaning Examples # Omit position if no digit exists for it. **Türkçe:** TABLE 11.5 DecimalFormat sembolleri Sembol Anlamı Örnekleri # Eğer onun için bir rakam yoksa, konumu atlayın. **English:** \$2.2 0 Put 0 in position if no digit exists for it. **Türkçe:** \$2.2 0 Eğer bunun için bir rakam yoksa 0'ı pozisyona koyun. **English:** \$002.20 **Türkçe:** \$002.20

English: These examples should help illuminate how these symbols work: **Türkçe:** Bu örnekler, bu sembollerin nasıl çalıştığını aydınlatmaya yardımcı olmalıdır:

```
double d = 1234.567;
NumberFormat f1 = new DecimalFormat("###,###,###.0");
System.out.println(f1.format(d)); // 1,234.6
```

English: 15: **Türkçe:** 15:

```
NumberFormat f2 = new DecimalFormat("000,000,000.00000");
System.out.println(f2.format(d)); // 000,001,234.56700
```

English: 18: **Türkçe:** 18:

```
NumberFormat f3 = new DecimalFormat("Your Balance $#,###,###.##");
System.out.println(f3.format(d)); // Your Balance $1,234.57
```

English: Line 14 displays the digits in the number, rounding to the nearest 10th after the decimal. The extra positions to the left are omitted because we used #. Line 17 adds leading and trailing zeros to make the output the desired length. Line 20 shows prefixing a nonformatting character along with rounding because fewer digits are printed than available. Notice that the commas are automatically removed if they are used between # symbols. **Türkçe:** Satır 14, sayıdaki basamakları gösterir, ondalıktan sonra en yakın 10'a yuvarlanır. Soldaki ekstra pozisyonlar atlanır çünkü # kullandık. Line 17, çıktıyı istenen uzunlukta yapmak için önden ve takip eden sıfırlar ekler. Satır 20, yuvarlama ile birlikte formatlamayan bir karakterin önekini gösterir, çünkü daha az basamak basılır. # sembolleri arasında kullanılıyorsa virgüllerin otomatik olarak kaldırıldığını fark edin. **English:** As you see in the localization section, there's a second concrete class that inherits NumberFormat that you'll need to know for the exam. **Türkçe:** Yerelleştirme bölümünde gördüğünüz gibi, sınav için bilmeniz gereken NumberFormat miras kalan ikinci bir somut sınıf var.

Formatting Dates and Times

English: The date and time classes support many methods to get data out of them. **Türkçe:** Tarih ve zaman sınıfları, onlardan veri elde etmek için birçok yöntemi desteklemektedir.

```
LocalDate date = LocalDate.of(2022, Month.OCTOBER, 20);
System.out.println(date.getDayOfWeek()); // THURSDAY
System.out.println(date.getMonth()); // OCTOBER
System.out.println(date.getYear()); // 2022
System.out.println(date.getDayOfYear()); // 293
```

English: Java provides a class called `DateTimeFormatter` to display standard formats. **Türkçe:** Java standart formatları görüntülemek için `DateTimeFormatter` adlı bir sınıf sağlar.

```
LocalDate date = LocalDate.of(2022, Month.OCTOBER, 20);
LocalTime time = LocalTime.of(11, 12, 34);
LocalDateTime dt = LocalDateTime.of(date, time);
System.out.println(date.format(DateTimeFormatter.ISO_LOCAL_DATE));
System.out.println(time.format(DateTimeFormatter.ISO_LOCAL_TIME));
System.out.println(dt.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME));
```

English: The code snippet prints the following: **Türkçe:** Kod snippet'i aşağıdakileri yazdırır: **English:** 2022-10-20 11:12:34 2022-10-20T11:12:34 **Türkçe:** 2022-10-20 11:12:34 2022-10-20T11:12:34

English: The `DateTimeFormatter` will throw an exception if it encounters an incompatible type. For example, each of the following will produce an exception at runtime since it attempts to format a date with a time value, and vice versa: **Türkçe:** `DateTimeFormatter` uyumsuz bir türle karşılaşırsa bir istisna atar. Örneğin, aşağıdakilerin her biri, bir tarihi bir zaman değeri ile biçimlendirmeye çalıştığı için çalışma zamanında bir istisna üretecektir ve tam tersi:

```
date.format(DateTimeFormatter.ISO_LOCAL_TIME); // RuntimeException
time.format(DateTimeFormatter.ISO_LOCAL_DATE); // RuntimeException
```

Customizing the Date/Time Format

English: If you don't want to use one of the predefined formats, `DateTimeFormatter` supports a custom format using a date format String. **Türkçe:** Önceden tanımlanmış biçimlerden birini kullanmak istemiyorsanız, `DateTimeFormatter` tarih biçimi String kullanarak özel bir biçimi destekler.

```
var f = DateTimeFormatter.ofPattern("MMMM dd, yyyy 'at' hh:mm");
System.out.println(dt.format(f)); // October 20, 2022 at 11:12
```

English: Let's break this down a bit. Java assigns each letter or symbol a specific date/time part. For example, M is used for month, while y is used for year. And case matters! Using m instead of M means it will return the minute of the hour, not the month of the year. **Türkçe:** Bunu biraz kiralım. Java her harfi veya sembolünü belirli bir tarih/saat parçası atar. Örneğin, M ay için kullanılırken, y yıl için kullanılır. Ve dava önemli! M yerine m kullanmak, yılın ayı yerine saatin dakikasını geri getireceği anlamına gelir.

English: What about the number of symbols? The number often dictates the format of the date/time part. Using M by itself outputs the minimum number of characters for a month, such as 1 for January, while using MM always outputs two digits, such as 01. Furthermore, using MMM prints the three-letter abbreviation, such as Jul for July, while MMMM prints the full month name.

Türkçe: Peki ya sembollerin sayısı? Sayı genellikle tarih/zaman bölümünün formatını belirler. M'yi kendi başına kullanmak, Ocak ayı için 1 gibi bir ay boyunca minimum karakter sayısını çıkarırken, MM kullanmak her zaman 01 gibi iki rakam çıkarır. Ayrıca, MMM kullanarak Jul for Jul gibi üç harfli kısaltmayı basarken, MMMM tam ay adını yazdırır.

English: It's possible, albeit unlikely, to come across questions on the exam that use SimpleDateFormat rather than the more useful DateTimeFormatter. If you do see it on the exam used with an older java.util.Date object, just know that the custom formats that are likely to appear on the exam will be compatible with both. **Türkçe:** Sınavda daha kullanışlı DateTimeFormatter yerine SimpleDateFormat kullanan sorularla karşılaşmak mümkün olsa da mümkün. Daha eski bir java.util.Date nesnesi ile kullanılan sınavda görürseniz, sınavda görünmesi muhtemel olan özel biçimlerin her ikisiyle de uyumlu olacağını bilin.

Learning the Standard Date/Time Symbols

English: For the exam, you should be familiar enough with the various symbols that you can look at a date/time String and have a good idea of what the output will be. Table 11.6 includes the symbols you should be familiar with for the exam. **Türkçe:** Sınav için, bir tarih / saat String bakabileceğiniz çeşitli sembolere yeterince aşina olmalısınız ve çıktının ne olacağı hakkında iyi bir fikir sahibi olmalısınız. Tablo 11.6, sınav için aşina olmanız gereken sembollerini içerir. **English:** TABLE 11.6 Common date/time symbols Symbol Meaning Examples y Year 22, 2022 M Month 1, 01, Jan, January d Day 5, 05 **Türkçe:** TABLE 11.6 Ortak tarih/saat sembollerini Sembol Anlamı Örnekler y Yıl 22, 2022 M Ay 1, 01, Jan, Ocak d Gün 5, 05

English: Symbol Meaning Examples h Hour 9, 09 m Minute 45 S Second 52 a a.m./p.m. AM, PM z Time zone name Eastern Standard Time, EST Z Time zone offset---- 0400 Let's try some examples. What do you think the following prints? **Türkçe:** Sembol Anlamı Örnekler h Saat 9, 09 m Dakika 45 S Saniye 52 a a.m./p.m. AM, PM z Zaman dilimi adı Eastern Standard Time, EST Z Zaman dilimi ofset --- 0400 Bazı örnekler deneyelim. Aşağıdaki parmak izleri hakkında ne düşünüyorsunuz?

```
var dt = LocalDateTime.of(2022, Month.OCTOBER, 20, 6, 15, 30);
var formatter1 = DateTimeFormatter.ofPattern("MM/dd/yyyy hh:mm:ss");
System.out.println(dt.format(formatter1)); // 10/20/2022 06:15:30
var formatter2 = DateTimeFormatter.ofPattern("MM_yyyy_-_dd");
System.out.println(dt.format(formatter2)); // 10_2022_-_20
var formatter3 = DateTimeFormatter.ofPattern("h:mm z");
System.out.println(dt.format(formatter3)); // DateTimeException
```

English: The first example prints the date, with the month before the day, followed by the time. The second example prints the date in a weird format with extra characters that are just displayed as part of the output. **Türkçe:** İlk örnek, tarihi, günden önceki ayla birlikte, takip eden zamanı yazdırır. İkinci örnek, tarihi sadece çıktının bir parçası olarak görüntülenen ekstra karakterlerle garip bir biçimde yazdırır.

English: The third example throws an exception at runtime because the underlying `LocalDateTime` does not have a time zone specified. If `ZonedDateTime` were used instead, the code would complete successfully and print something like 06:15 EDT, depending on the time zone. **Türkçe:** Üçüncü örnek çalışma zamanında bir istisna atar, çünkü altta yatan `LocalDateTime`'da belirtilen bir zaman dilimi yoktur. Bunun yerine `ZonedDateTime` kullanılırsa, kod başarılı bir şekilde tamamlanacak ve saat dilimine bağlı olarak 06:15 EDT gibi bir şey yazdıracaktı. **English:** As you saw in the previous example, you need to make sure the format String is compatible with the underlying date/time type. Table 11.7 shows which symbols you can use with each of the date/time objects. **Türkçe:** Önceki örnekte de gördüğümüz gibi, String biçiminin altta yatan tarih/zaman türüyle uyumlu olduğundan emin olmanız gerekir. Tablo 11.7, tarih/saat nesnelerinin her biriyle hangi sembolleri kullanabileceğinizi gösterir. **English:** Make sure you know which symbols are compatible with which date/time types. For example, trying to format a month for a `LocalTime` or an hour for a `LocalDate` will result in a runtime exception. **Türkçe:** Hangi sembollerin hangi tarih/saat türleri ile uyumlu olduğunu bildiğinizden emin olun. Örneğin, bir `LocalTime` için bir ay veya bir `LocalDate` için bir saat biçimlendirmeye çalışmak runtime exception ile sonuçlanacaktır.

English: TABLE 11.7 Supported date/time symbols

Symbol	LocalDate	LocalTime	LocalDateTime	ZonedDateTime
y	✓	✓	✓	✓
M	✓	✓	✓	✓
d	✓	✓	✓	✓
h	✓	✓	✓	✓
m	✓	✓	✓	✓
s	✓	✓	✓	✓
a	✓	✓	✓	✓
z	✓	✓	✓	✓
Z	✓	✓	✓	✓

Türkçe: TABLE 11.7 Desteklenen tarih/saat sembolleri

Sembol	LocalDate	YerelSaat	YerelTarihSaati	BölgeLiSaat
y	✓	✓	✓	✓
M	✓	✓	✓	✓
d	✓	✓	✓	✓
h	✓	✓	✓	✓
m	✓	✓	✓	✓
s	✓	✓	✓	✓
a	✓	✓	✓	✓
z	✓	✓	✓	✓
Z	✓	✓	✓	✓

Selecting a format() Method

English: The date/time classes contain a `format()` method that will take a formatter, while the formatter classes contain a `format()` method that will take a date/time value. The result is that either of the following is acceptable: **Türkçe:** Tarih/saat sınıfları bir formatlayıcı alacak bir `format()` yöntemi içerirken, formatlayıcı sınıfları bir tarih/saat değeri alacak bir `format()` yöntemi içerir. Sonuç, aşağıdakilerden birinin kabul edilebilir olmasıdır:

```
var dateTime = LocalDateTime.of(2022, Month.OCTOBER, 20, 6, 15, 30);
var formatter = DateTimeFormatter.ofPattern("MM/dd/yyyy hh:mm:ss");
System.out.println(dateTime.format(formatter)); // 10/20/2022 06:15:30
System.out.println(formatter.format(dateTime)); // 10/20/2022 06:15:30
```

English: These statements print the same value at runtime. Which syntax you use is up to you. **Türkçe:** Bu ifadeler çalışma zamanında aynı değeri yazdırır. Hangi syntax'ı kullandığınız size kalmış.

Adding Custom Text Values

English: What if you want your format to include some custom text values? If you just type them as part of the format String, the formatter will interpret each character as a date/time symbol. In the best case, it will display weird data based on extra symbols you enter. In the worst case, it will throw an exception because the characters contain invalid symbols. Neither is desirable! **Türkçe:** Formatınızın bazı özel metin değerlerini içermesini istiyorsanız ne olur? Bunları String biçiminin bir parçası olarak yazarsanız, biçimlendirici her karakteri bir tarih/saat sembolü olarak yorumlayacaktır. En iyi durumda, girdiğiniz ekstra sembollere dayalı garip veriler görüntüler. En kötü durumda, karakterler geçersiz semboller içerdiğinden bir istisna atacaktır. İstenen de değil! **English:** One way to address this would be to break the formatter into multiple smaller formatters and then concatenate the results. **Türkçe:** Bunu ele almanın bir yolu, formatlayıcıyı birden fazla küçük formatlayıcıya bölmek ve daha sonra sonuçları birleştirmek olacaktır.

```
var dt = LocalDateTime.of(2022, Month.OCTOBER, 20, 6, 15, 30);
```

```
var f1 = DateTimeFormatter.ofPattern("MMMM dd, yyyy ");
var f2 = DateTimeFormatter.ofPattern(" hh:mm");
System.out.println(dt.format(f1) + "at" + dt.format(f2));
```

English: This prints October 20, 2022 at 06:15 at runtime. **Türkçe:** Bu baskılar 20 Ekim 2022'de saat 06:15'te. **English:** While this works, it could become difficult if a lot of text values and date symbols are intermixed. Luckily, Java includes a much simpler solution. You can escape the text by surrounding it with a pair of single quotes ('). Escaping text instructs the formatter to ignore the values inside the single quotes and just insert them as part of the final value. **Türkçe:** Bu işe yarasa da, birçok metin değeri ve tarih sembolü birbirine karışırsa zor olabilir. Neyse ki, Java çok daha basit bir çözüm içerir. Bir çift tek tırnak (') ile çevreleyerek metinden kaçabilirsiniz. Kaçan metin, formatlayıcıya tek tırnak içindeki değerleri görmezden gelmesini ve sadece son değer bir parçası olarak eklemesini emreder.

```
var f = DateTimeFormatter.ofPattern("MMMM dd, yyyy 'at' hh:mm");
System.out.println(dt.format(f)); // October 20, 2022 at 06:15
```

English: But what if you need to display a single quote in the output, too? Welcome to the fun of escaping characters! Java supports this by putting two single quotes next to each other. **Türkçe:** Ama ya çıktıda da tek bir alıntı görüntülemeniz gerekiyorsa? Karakterlerden kaçmanın eğlencesine hoş geldiniz! Java iki tek tırnak yan yana koyarak bunu destekler. **English:** We conclude our discussion of date formatting with some examples of formats and their output that rely on text values, shown here: **Türkçe:** Tarih biçimlendirme konusundaki tartışmamızı bazı format örnekleri ve burada gösterilen metin değerlerine dayanan çıktıları ile sonuçlandırıyoruz:

```
var g1 = DateTimeFormatter.ofPattern("MMMM dd', Party's at' hh:mm");
System.out.println(dt.format(g1)); // October 20, Party's at 06:15
var g2 = DateTimeFormatter.ofPattern("'System format, hh:mm: 'hh:mm'");
System.out.println(dt.format(g2)); // System format, hh:mm: 06:15
var g3 = DateTimeFormatter.ofPattern("'NEW! 'yyyy', yay!'");
System.out.println(dt.format(g3)); // NEW! 2022, yay!
```

English: If you don't escape the text values with single quotes, an exception will be thrown at runtime if the text cannot be interpreted as a date/time symbol. **Türkçe:** Metin değerlerinden tek tırnakla kaçmazsanız, metin tarih/saat sembolü olarak yorumlanamazsa, çalışma zamanında bir istisna atılır.

```
DateTimeFormatter.ofPattern("The time is hh:mm"); // Exception thrown
```

English: This line throws an exception since T is an unknown symbol. The exam might also present you with an incomplete escape sequence. **Türkçe:** Bu çizgi, T bilinmeyen bir sembol olduğu için bir istisna atar. Sınav ayrıca size tamamlanmamış bir kaçış sekansı da sunabilir.

```
DateTimeFormatter.ofPattern("'Time is: hh:mm: "); // Exception thrown
```

English: Failure to terminate an escape sequence will trigger an exception at runtime. **Türkçe:** Bir kaçış dizisini sonlandırmak, çalışma zamanında bir istisnayı tetikleyecektir.

Supporting Internationalization and Localization

English: Many applications need to work in different countries and with different languages. For example, consider the sentence “The zoo is holding a special event on 4/1/22 to look at animal behaviors.” When is the event? In the United States, it is on April 1. However, a British reader would interpret this as January 4. A British reader might also wonder why we **Türkçe:** Birçok uygulamanın farklı ülkelerde ve farklı dillerde çalışması gerekir. Örneğin, “Hayvan davranışlarına bakmak için hayvanat bahçesi 4/1/22’de özel bir etkinlik düzenliyor.” cümlesini düşünün. Etkinlik ne zaman? Amerika Birleşik Devletleri’nde ise 1 Nisan’da. Bununla birlikte, bir İngiliz okuyucu bunu 4 Ocak olarak yorumlayacaktır. Bir İngiliz okuyucu da merak edebilir neden biz

English: didn’t write “behaviours.” If we are making a website or program that will be used in multiple countries, we want to use the correct language and formatting. **Türkçe:** “Davranışlar” yazmadık. Birden fazla ülkede kullanılacak bir web sitesi veya program yapıyorsak, doğru dili ve biçimlendirmeyi kullanmak istiyoruz. **English:** Internationalization is the process of designing your program so it can be adapted. This involves placing strings in a properties file and ensuring that the proper data formatters are used. Localization means supporting multiple locales or geographic regions. You can think of a locale as being like a language and country pairing. Localization includes translating strings to different languages. It also includes outputting dates and numbers in the correct format for that locale. **Türkçe:** Uluslararasılaşma, programınızı uyarlanabilmesi için tasarlama sürecidir. Bu, strings’yi bir properties file içine yerleştirmeyi ve uygun veri formatlayıcılarının kullanılmasını sağlamayı içerir. Yerelleştirme, çoklu locales veya coğrafi bölgeleri desteklemek anlamına gelir. locale bir dil ve ülke eşleştirmesi gibi düşünebilirsiniz. Yerelleştirme, strings ’yi farklı dillere çevirmeyi içerir. Ayrıca, locale için doğru formatta çıkış tarihleri ve sayıları içerir. **English:** Initially, your program does not need to support multiple locales. The key is to future-proof your application by using these techniques. This way, when your product becomes successful, you can add support for new languages or regions without rewriting everything. **Türkçe:** Başlangıçta, programınızın birden fazla locales desteklemesine gerek yoktur. Anahtar, bu teknikleri kullanarak uygulamanızı geleceğe dönük olarak kanıtlamaktır. Bu şekilde, ürününüz başarılı olduğunda, her şeyi yeniden yazmadan yeni diller veya bölgeler için destek ekleyebilirsiniz. **English:** In this section, we look at how to define a locale and use it to format dates, numbers, and strings. **Türkçe:** Bu bölümde, bir locale tanımlamasına ve tarihleri, sayıları ve strings biçimlendirmek için nasıl kullanılacağına bakıyoruz.

Picking a Locale

English: While Oracle defines a locale as “a specific geographical, political, or cultural region,” you’ll only see languages and countries on the exam. Oracle certainly isn’t going to delve into political regions that are not countries. That’s too controversial for an exam! **Türkçe:** Oracle, bir locale’u “belirli bir coğrafi, politik veya kültürel bölge” olarak tanımlarken, yalnızca sınavda dilleri ve ülkeleri göreceksiniz. Oracle kesinlikle ülke olmayan siyasi bölgelere girmeyecek. Bu bir sınav için çok tartışmalı! **English:** The Locale class is in the java.util package. The first useful Locale to find is the user’s current locale. Try running the following code on your computer: **Türkçe:** Locale sınıfı java.util paketindedir. Bulmak için ilk yararlı Locale kullanıcının mevcut locale dir. Bilgisayarınızda aşağıdaki kodu çalıştırmayı deneyin:

```
Locale locale = Locale.getDefault();
System.out.println(locale);
```

English: When we run it, it prints en_US. It might be different for you. This default output tells us that our computers are using English and are sitting in the United States. **Türkçe:** Çalıştırdığımızda, en_US’u yazdırıyor. Senin için farklı olabilir. Bu varsayılan çıktı bize bilgisayarlarımızın İngilizce kullandığını ve Amerika Birleşik Devletleri’nde oturduğunu söylüyor. **English:** Notice the format. First comes the lowercase language code. The language is always required. Then comes an underscore followed by the uppercase country code. The country is optional. Figure 11.6 shows the two formats for Locale objects that you are expected to remember. **Türkçe:** Formata dikkat edin. Önce küçük harfli dil kodu gelir. Dil her zaman gereklidir. Daha sonra büyük ülke kodunun ardından bir alt çizgi gelir. Ülke optional’dır. Şekil 11.6, hatırlamanız beklenen Locale nesneleri için iki biçimi gösterir. **English:** FIGURE 11.6 Locale formats
Locale Locale (language) (language, country) **Türkçe:** FIGURE 11.6 Locale formatları
Locale Locale (dil) (dil, ülke) **English:** fr en_US Lowercase Lowercase Uppercase language language country code code
Türkçe: fr en_US Küçük harf Küçük harf Büyük harf dil ülke kodu kodu

English: As practice, make sure that you understand why each of these Locale identifiers is invalid:
Türkçe: Uygulama olarak, bu Locale tanımlayıcılarının her birinin neden geçersiz olduğunu anladığınızdan emin olun:

```
US // Cannot have country without language
enUS // Missing underscore
US_en // The country and language are reversed
EN // Language must be lowercase
```

English: The corrected versions are en and en_US. **Türkçe:** Düzeltilmiş versiyonlar en ve en_US'tur.

English: You do not need to memorize language or country codes. The exam will let you know about any that are being used. You do need to recognize valid and invalid formats. Pay attention to uppercase/lowercase and the underscore. For example, if you see a locale expressed as es_CO, then you should know that the language is es and the country is CO, even if you didn't know that they represent Spanish and Colombia, respectively. **Türkçe:** Dil veya ülke kodlarını ezberlemenize gerek yoktur. Sınav, kullanılan herhangi bir şey hakkında size bilgi verecektir. Geçerli ve geçersiz formatları tanımanız gerekir. Büyük/küçük harf ve alt çizgiye dikkat edin. Örneğin, es_CO olarak ifade edilen bir locale görürseniz, o zaman dilin es olduğunu ve ülkenin CO olduğunu bilmelisiniz, sırasıyla İspanyolca ve Kolombiya'yı temsil ettiklerini bilmeseniz bile. **English:** As a developer, you often need to write code that selects a locale other than the default one. There are three common ways of doing this. The first is to use the built-in constants in the Locale class, available for some common locales. **Türkçe:** Bir geliştirici olarak, varsayılan olandan başka bir locale seçen bir kod yazmanız gerekir. Bunu yapmanın üç yaygın yolu vardır. Birincisi, bazı ortak locales için mevcut olan Locale sınıfındaki yerleşik sabitleri kullanmaktır.

```
System.out.println(Locale.GERMAN); // de
System.out.println(Locale.GERMANY); // de_DE
```

English: The first example selects the German language, which is spoken in many countries, including Austria (de_AT) and Liechtenstein (de_LI). The second example selects both German the language and Germany the country. While these examples may look similar, they are not the same. Only one includes a country code. **Türkçe:** İlk örnek, Avusturya (de_AT) ve Lihtenştayn (de_LI) da dahil olmak üzere birçok ülkede konuşulan Almanca dilini seçer. İkinci örnek, hem Almanca dili hem de Almanya'yı ülkeyi seçer. Bu örnekler benzer görünse de aynı değildir. Sadece bir tanesi ülke kodu içerir. **English:** The second way of selecting a Locale is to use the constructors to create a new object. You can pass just a language, or both a language and country: **Türkçe:** Locale seçmenin ikinci yolu, yeni bir nesne oluşturmak için yapıları kullanmaktır. Sadece bir dili veya hem bir dili hem de ülkeyi geçebilirsiniz:

```
System.out.println(new Locale("fr")); // fr
System.out.println(new Locale("hi", "IN")); // hi_IN
```

English: The first is the language French, and the second is Hindi in India. Again, you don't need to memorize the codes. There is another constructor that lets you be even more specific about the locale. Luckily, providing a variant value is not on the exam. **Türkçe:** Birincisi Fransızca, ikincisi Hindistan'da Hintçe'dir. Yine kodları ezberlemenize gerek yok. locale hakkında daha da spesifik olmanızı sağlayan başka bir constructor var. Neyse ki, bir varyant değeri sağlamak sınavda değil. **English:** Java will let you create a Locale with an invalid language or country, such as xx_XX. However, it will not match the Locale that you want to use, and your program will not behave as expected. **Türkçe:** Java, xx_XX gibi geçersiz bir dil veya ülke ile Locale oluşturmanıza izin verecektir. Ancak, kullanmak istediğiniz Locale ile eşleşmez ve programınız beklendiği gibi davranmaz. **English:** There's a third way to create a Locale that is more flexible. The builder design pattern lets you set all of the properties that you care about and then build the Locale at the end. This means that you can specify the properties in any order. The following two Locale values both represent en_US: **Türkçe:** Daha esnek bir Locale oluşturmanın üçüncü bir yolu var. Oluşturucu tasarım deseni, önemsedığınız set tüm properties'yi sağlar ve daha sonra Locale'yi sonunda inşa etmenizi sağlar. Bu, herhangi bir sırada properties belirtebileceğiniz anlamına gelir. Aşağıdaki iki Locale değeri her ikisi de en_US'u temsil eder:

```
Locale l1 = new Locale.Builder()  
.setLanguage("en")
```

```
.setRegion("US")  
.build();  
Locale l2 = new Locale.Builder()  
.setRegion("US")  
.setLanguage("en")  
.build();
```

English: When testing a program, you might need to use a Locale other than your computer's default.

Türkçe: Bir programı test ederken, bilgisayarınızın varsayılanından başka bir Locale kullanmanız gerekebilir.

```
System.out.println(Locale.getDefault()); // en_US  
Locale locale = new Locale("fr");  
Locale.setDefault(locale);  
System.out.println(Locale.getDefault()); // fr
```

English: Try it, and don't worry— the Locale changes for only that one Java program. It does not change any settings on your computer. It does not even change future executions of the same program. **Türkçe:** Deneyin ve endişelenmeyin Locale yalnızca bir Java programı için değişir. Bilgisayarınızdaki herhangi bir ayarı değiştirmez. Aynı programın gelecekteki infazlarını bile değiştirmiyor.

The exam may use `setDefault()` because it can't make assumptions

English: about where you are located. In practice, we rarely write code to change a user's default locale.

Türkçe: Bulunduğunuz yer hakkında. Uygulamada, bir kullanıcının default locale değerini değiştirmek için nadiren kod yazıyoruz.

Localizing Numbers

English: It might surprise you that formatting or parsing currency and number values can change depending on your locale. For example, in the United States, the dollar sign is prepended before the value along with a decimal point for values less than one dollar, such as \$2.15. In Germany, though, the euro symbol is appended to the value along with a comma for values less than one euro, such as 2,15 €.

Türkçe: locale 'nize bağlı olarak para birimi ve sayı değerlerini biçimlendirmek veya ayrıştırmak sizi şaşırtabilir. Örneğin, Amerika Birleşik Devletleri'nde, dolar işareti değerden önce, bir dolardan daha az değerler için ondalık bir nokta ile birlikte, örneğin 2.15 \$ gibi hazırlanır. Bununla birlikte, Almanya'da, euro sembolü, 2,15 gibi bir avrodan daha az değerler için bir virgül ile birlikte değere eklenir. **English:** Luckily, the java.text package includes classes to save the day. The following sections cover how to format numbers, currency, and dates based on the locale. **Türkçe:** Neyse ki, java.text paketi günü kurtarmak için sınıflar içerir. Aşağıdaki bölümler, sayıların, para biriminin ve tarihlerin locale'e göre nasıl

biçimlendirileceğini kapsar. **English:** The first step to formatting or parsing data is the same: obtain an instance of a NumberFormat. Table 11.8 shows the available factory methods. **Türkçe:** Verileri biçimlendirmek veya ayrıştırmak için ilk adım aynıdır: bir NumberFormat örneği elde edin. Tablo 11.8 kullanılabilir factory methods gösterir. **English:** Once you have the NumberFormat instance, you can call format() to turn a number into a String, or you can use parse() to turn a String into a number. **Türkçe:** NumberFormat örneğine sahip olduktan sonra, bir sayıyı String'ye dönüştürmek için format()'u arayabilir veya bir String'yi bir sayıya dönüştürmek için parse()'i kullanabilirsiniz. **English:** The format classes are not thread-safe. Do not store them in instance variables or static variables. You learn more about thread safety in Chapter 13, "Concurrency." **Türkçe:** Format sınıfları thread-safe değildir. Bunları örnek değişkenlerde veya statik değişkenlerde saklamayın. Bölüm 13'te iş parçacığı güvenliği hakkında daha fazla bilgi edirsiniz, "Concurrency".

English: TABLE 11.8 Factory methods to get a NumberFormat Description Using default Locale and a specified Locale **Türkçe:** TABLE 11.8 Factory methods için NumberFormat Açıklama default Locale ve belirtilen Locale kullanarak

```
General-purpose formatter NumberFormat.getInstance()
```

English: NumberFormat.getInstance(Locale locale) **Türkçe:** NumberFormat.getInstance(Locale locale)

English: Same as getInstance NumberFormat.getNumberInstance() **Türkçe:** getInstance NumberFormat.getNumberInstance() ile aynı **English:** NumberFormat.getNumberInstance(Locale locale)

Türkçe: NumberFormat.getNumberInstance(Locale locale) **English:** For formatting monetary NumberFormat.getCurrencyInstance() amounts NumberFormat.getCurrencyInstance(Locale locale)

Türkçe: Parasal NumberFormat.getCurrencyInstance() miktarlarını biçimlendirmek için

NumberFormat.getCurrencyInstance(Locale locale) **English:** For formatting

NumberFormat.getPercentInstance() percentages NumberFormat.getPercentInstance(Locale locale)

Türkçe: NumberFormat.getPercentInstance() yüzdelerini biçimlendirmek için

NumberFormat.getPercentInstance(Locale locale) **English:** Rounds decimal values

NumberFormat.getIntegerInstance() before displaying NumberFormat.getIntegerInstance(Locale locale)

Türkçe: NumberFormat.getIntegerInstance(Locale locale) görüntülenmeden önce

NumberFormat.getIntegerInstance() ondalık değerlerini döndürür **English:** Returns compact number

NumberFormat.getCompactNumberInstance() formatter NumberFormat.getCompactNumberInstance(

Locale locale, NumberFormat.Style formatStyle) **Türkçe:** Kompakt sayı

NumberFormat.getCompactNumberInstance() formatlayıcı NumberFormat.getCompactNumberInstance(Locale locale, NumberFormat.Style formatStyle) döndürür

Formatting Numbers

English: When we format data, we convert it from a structured object or primitive value into a String. The `NumberFormat.format()` method formats the given number based on the locale associated with the `NumberFormat` object. **Türkçe:** Verileri biçimlendirdiğimizde, yapılandırılmış bir nesneden veya ilkel değerden bir String değerine dönüştürürüz. `NumberFormat.format()` yöntemi, verilen sayıyı `NumberFormat` nesnesi ile ilişkili locale temel alınarak biçimlendirir. **English:** Let's go back to our zoo for a minute. For marketing literature, we want to share the average monthly number of visitors to the San Diego Zoo. The following shows printing out the same number in three different locales: **Türkçe:** Bir dakikalığına hayvanat bahçemize geri dönelim. Pazarlama literatürü için, San Diego Hayvanat Bahçesi'ne ortalama aylık ziyaretçi sayısını paylaşmak istiyoruz. Aşağıdakiler aynı sayıyı üç farklı locales olarak yazdırır:

```
int attendeesPerYear = 3_200_000;
int attendeesPerMonth = attendeesPerYear / 12;
var us = NumberFormat.getInstance(Locale.US);
System.out.println(us.format(attendeesPerMonth)); // 266,666
var gr = NumberFormat.getInstance(Locale.GERMANY);
System.out.println(gr.format(attendeesPerMonth)); // 266.666
var ca = NumberFormat.getInstance(Locale.CANADA_FRENCH);
System.out.println(ca.format(attendeesPerMonth)); // 266 666
```

English: This shows how our U.S., German, and French Canadian guests can all see the same information in the number format they are accustomed to using. In practice, we would just call `NumberFormat.getInstance()` and rely on the user's default locale to format the output. **Türkçe:** Bu, U.S., Almanca ve Fransız Kanadalı konukların hepsinin alışık oldukları sayı biçiminde aynı bilgileri nasıl görebildiğini gösterir. Pratikte, çıktıyı biçimlendirmek için `NumberFormat.getInstance()`'i arar ve kullanıcının default locale'sine güvenirdik. **English:** Formatting currency works the same way. **Türkçe:** Para birimini biçimlendirmek de aynı şekilde çalışır.

```
double price = 48;
var myLocale = NumberFormat.getCurrencyInstance();
System.out.println(myLocale.format(price));
```

English: When run with the default locale of `en_US` for the United States, this code outputs \$48.00. On the other hand, when run with the default locale of `en_GB` for Great Britain, it outputs £48.00. **Türkçe:** Amerika Birleşik Devletleri için `en_US`'un default locale ile çalıştırıldığında, bu kod 48.00 \$ 'lık bir çıktı verir. Öte yandan, Büyük Britanya için `en_GB`'nin default locale ile çalıştırıldığında, 48.00 çıktı alır. **English:** In the real world, use `int` or `BigDecimal` for money and not `double`. **Türkçe:** Gerçek dünyada, para için `int` veya `BigDecimal` kullanın ve `double` değil. **English:** Doing math on amounts with `double` is dangerous because the values are stored as floating-point numbers. Your boss won't appreciate it if you lose pennies or fractions of pennies during transactions! **Türkçe:** `double` ile miktarlar üzerinde matematik yapmak tehlikelidir, çünkü değerler kayan nokta numaraları olarak saklanır. İşlemler sırasında paraları veya para kesirlerini kaybederseniz patronunuz bunu takdir etmeyecektir! **English:** Finally, the exam may have examples that show formatting percentages. **Türkçe:** Son olarak, sınavda biçimlendirme yüzdeleri gösteren örnekler olabilir:

```
double successRate = 0.802;
var us = NumberFormat.getPercentInstance(Locale.US);
System.out.println(us.format(successRate)); // 80%
var gr = NumberFormat.getPercentInstance(Locale.GERMANY);
System.out.println(gr.format(successRate)); // 80 %
```

English: Not much difference, we know, but you should at least be aware that the ability to print a percentage is locale-specific for the exam! **Türkçe:** Çok fazla fark yok, biliyoruz, ama en azından bir yüzde yazdırma yeteneğinin sınav için locale-özel olduğunu bilmelisiniz!

Parsing Numbers

English: When we parse data, we convert it from a String to a structured object or primitive value. The `NumberFormat.parse()` method accomplishes this and takes the locale into consideration. **Türkçe:** Verileri ayrıştırdığımızda, bir String 'den yapılandırılmış bir nesneye veya ilkel değere dönüştürürüz.

`NumberFormat.parse()` yöntemi bunu gerçekleştirir ve locale değerini dikkate alır. **English:** For example, if the locale is the English/United States (`en_US`) and the number contains commas, the commas are treated as formatting symbols. If the locale relates to a country or language that uses commas as a decimal separator, the comma is treated as a decimal point. **Türkçe:** Örneğin, locale İngilizce/Amerika Birleşik Devletleri (`en_US`) ise ve sayı virgül içeriyorsa, virgüller biçimlendirme sembolleri olarak ele alınır. locale, virgülleri ondalık ayırıcı olarak kullanan bir ülke veya dille ilgiliyse, virgül ondalık nokta olarak ele alınır.

The `parse()` method, found in various types, declares a checked

English: exception `ParseException` that must be handled or declared in the method in which it is called. **Türkçe:** exception `ParseException` olarak adlandırıldığı yöntemde ele alınması veya ilan edilmesi gerekir.

English: Let's look at an example. The following code parses a discounted ticket price with different locales. The `parse()` method throws a checked `ParseException`, so make sure to handle or declare it in your own code. **Türkçe:** Bir örneğe bakalım. Aşağıdaki kod, farklı locales ile indirimli bilet fiyatını ayrıştırır. `parse()` yöntemi, işaretli bir `ParseException` atar, bu nedenle kendi kodunuzda ele aldığınızdan veya beyan ettiğinden emin olun.

```
String s = "40.45";
var en = NumberFormat.getInstance(Locale.US);
System.out.println(en.parse(s)); // 40.45
var fr = NumberFormat.getInstance(Locale.FRANCE);
System.out.println(fr.parse(s)); // 40
```

English: In the United States, a dot (.) is part of a number, and the number is parsed as you might expect. France does not use a decimal point to separate numbers. Java parses it as a formatting character, and it stops looking at the rest of the number. The lesson is to make sure that you parse using the right locale!

Türkçe: Amerika Birleşik Devletleri'nde, bir nokta (.) bir sayının parçasıdır ve sayı tahmin edebileceğiniz gibi ayrıştırılır. Fransa, sayıları ayırmak için ondalık bir nokta kullanmaz. Java biçimlendirme karakteri olarak ayrıştırır ve sayının geri kalanına bakmayı bırakır. Ders, sağ locale kullanarak ayrıştırdığınızdan emin olmaktır! **English:** The parse() method is also used for parsing currency. For example, we can read in the zoo's monthly income from ticket sales: **Türkçe:** parse() yöntemi de para birimini ayrıştırmak için kullanılır. Örneğin, hayvanat bahçesinin aylık gelirini bilet satışlarından okuyabiliriz:

```
String income = "$92,807.99";
var cf = NumberFormat.getCurrencyInstance();
double value = (Double) cf.parse(income);
System.out.println(value); // 92807.99
```

English: The currency string "\$92,807.99" contains a dollar sign and a comma. The parse method strips out the characters and converts the value to a number. The return value of parse is a Number object. Number is the parent class of all the java.lang wrapper classes, so the return value can be cast to its appropriate data type. The Number is cast to a Double and then automatically unboxed into a double.

Türkçe: string "\$92,807.99" para birimi bir dolar işareti ve bir virgül içerir. Ayrıştırma yöntemi karakterleri ortadan kaldırır ve değeri bir sayıya dönüştürür. Parsersın dönüş değeri bir Sayı nesnesidir. Sayı, tüm java.lang ambalaj sınıflarının ana sınıfıdır, bu nedenle dönüş değeri uygun veri türüne atılabilir. Numara bir Double 'ye gönderilir ve daha sonra otomatik olarak bir double 'e kutudan çıkar.

Formatting with CompactNumberFormat

English: The second class that inherits NumberFormat that you need to know for the exam is CompactNumberFormat. It is new to the Java 17 exam, so you're likely to see a question on it! **Türkçe:** Sınav için bilmeniz gereken NumberFormat ikinci sınıf CompactNumberFormat'tır. Java 17 sınavında yenidir, bu yüzden üzerinde bir soru görmeniz muhtemeldir! **English:** CompactNumberFormat is similar to DecimalFormat, but it is designed to be used in places where print space may be limited. It is opinionated in the sense that it picks a format for you, and locale-specific in that output can change depending on your location. **Türkçe:** CompactNumberFormat, DecimalFormat ile benzerdir, ancak baskı alanının sınırlı olabileceği yerlerde kullanılmak üzere tasarlanmıştır. Sizin için bir format seçmesi anlamında fikirlendirilmiş ve bu çıktıdaki locale-specific konumunuza bağlı olarak değişebilir. **English:** Consider the following sample code that applies a CompactNumberFormat five times to two locales, using a static import for Style (an enum with value SHORT or LONG): **Türkçe:** Stil için statik bir içe aktarma kullanarak CompactNumberFormat'ı beş kez iki locales uygulayan aşağıdaki örnek kodu düşünün (SHORT veya LONG değeri olan bir enum):

```
var formatters = Stream.of(
    NumberFormat.getCompactNumberInstance(),
    NumberFormat.getCompactNumberInstance(Locale.getDefault(), Style.SHORT),
    NumberFormat.getCompactNumberInstance(Locale.getDefault(), Style.LONG),
```

```
    NumberFormat.getCompactNumberInstance(Locale.GERMAN, Style.SHORT),
    NumberFormat.getCompactNumberInstance(Locale.GERMAN, Style.LONG),
    NumberFormat.getNumberInstance());
formatters.map(s -> s.format(7_123_456)).forEach(System.out::println);
```

English: The following is printed by this code when run in the en_US locale (line breaks added for readability): **Türkçe:** Aşağıdakiler en_US locale çalıştırıldığında bu kod tarafından yazdırılır (okunabilirlik için satır araları eklendi): **English:** 7M 7M 7 million 7 Mio. 7 Millionen 7,123,456 Notice that the first two lines are the same. If you don't specify a style, SHORT is used by default. Next, notice that the values except the last one (which doesn't use a compact number formatter) are truncated. There's a reason it's called a compact number formatter! Also, notice that the short form uses common labels for large values, such as K for thousand. Last but not least, the output may differ for you when you run this, as it was run in an en_US locale. **Türkçe:** 7M 7M 7 milyon 7 Mio. 7 Millionen 7,123,456 İlk iki satırın aynı olduğuna dikkat edin. Bir stil belirtmezseniz, varsayılan olarak SHORT kullanılır. Daha sonra, sonuncu dışındaki değerlerin (kompakt sayı biçimleyicisi kullanmayan) kesildiğini fark edin. Kompakt sayı biçimlendirici olarak adlandırılmasının bir nedeni var! Ayrıca, short formunun bin için K gibi büyük değerler için ortak etiketler kullandığına dikkat edin. Son olarak, en_US locale çalıştırıldığında çıktı sizin için farklı olabilir. **English:** Using the same formatters, let's try another example: **Türkçe:** Aynı formatlayıcıları kullanarak, başka bir örnek deneyelim:

```
formatters.map(s -> s.format(314_900_000)).forEach(System.out::println);
```

English: This prints the following when run in the en_US locale: 315M 315M 315 million 315 Mio. 315 Millionen 314,900,000 Notice that the third digit is automatically rounded up for the entries that use a CompactNumberFormat. The following summarizes the rules for CompactNumberFormat: ■ ■ First it determines the highest range for the number, such as thousand (K), million (M), billion (B), or trillion (T). ■ ■ It then returns up to the first three digits of that range, rounding the last digit as needed. ■ ■ Finally, it prints an identifier. If SHORT is used, a symbol is returned. If LONG is used, a space followed by a word is returned. **Türkçe:** Bu, en_US locale 'de çalıştırıldığında aşağıdakileri yazdırır: 315M 315M 315 milyon 315 Mio. 315 Millionen 314.900.000 CompactNumberFormat kullanan girişler için üçüncü hanenin otomatik olarak toplandığına dikkat edin. Aşağıdaki CompactNumberFormat için kuralları özetler: Önce bin (K), milyon (M), milyar (B), veya trilyon (T) gibi sayı için en yüksek aralığı belirler. Daha sonra bu aralığın ilk üç basamağına kadar döner, son basamağı gerektiği gibi yuvarlar. Son olarak, bir tanımlayıcı yazdırır. SHORT kullanılırsa, bir sembol döndürülür. LONG kullanılırsa, bir sözcüğün ardından gelen bir boşluk döndürülür.

English: For the exam, make sure you understand the difference between the SHORT and LONG formats and common symbols like M for million. **Türkçe:** Sınav için, SHORT ve LONG formatları ve milyon M gibi ortak semboller arasındaki farkı anladığınızdan emin olun.

Localizing Dates

English: Like numbers, date formats can vary by locale. Table 11.9 shows methods used to retrieve an instance of a `DateTimeFormatter` using the default locale. **Türkçe:** Sayılar gibi, tarih biçimleri de locale ile değişebilir. Tablo 11.9, default locale kullanarak bir `DateTimeFormatter` örneğini almak için kullanılan yöntemleri gösterir. **English:** TABLE 11.9 Factory methods to get a `DateTimeFormatter` Description Using default Locale For formatting dates `DateTimeFormatter.ofLocalizedDate(FormatStyle dateStyle)` **Türkçe:** TABLE 11.9 Factory methods için `DateTimeFormatter` Açıklama default Locale kullanarak biçimlendirme tarihleri `DateTimeFormatter.ofLocalizedDate(FormatStyle dateStyle)` **English:** For formatting times `DateTimeFormatter.ofLocalizedTime(FormatStyle timeStyle)` **Türkçe:** Biçimlendirme süreleri için `DateTimeFormatter.ofLocalizedTime(FormatStyle timeStyle)` **English:** For formatting dates and `DateTimeFormatter.ofLocalizedDateTime` times (FormatStyle dateStyle, FormatStyle timeStyle) **Türkçe:** Biçimlendirme tarihleri ve `DateTimeFormatter.ofLocalizedDateTime` süreleri için (FormatStyle dateStyle, FormatStyle timeStyle) **English:** `DateTimeFormatter.ofLocalizedDateTime` (FormatStyle dateTimeStyle) **Türkçe:** `DateTimeFormatter.ofLocalizedDateTime` (FormatStyle dateTimeStyle) **English:** Each method in the table takes a `FormatStyle` parameter (or two) with possible values SHORT, MEDIUM, LONG, and FULL. For the exam, you are not required to know the format of each of these styles. **Türkçe:** Tablodaki her yöntem, SHORT, MEDIUM, LONG ve FULL olası değerleriyle bir `FormatStyle` parametresi (veya iki) alır. Sınav için bu tarzların her birinin formatını bilmeniz gerekmez. **English:** What if you need a formatter for a specific locale? Easy enough— just append `withLocale(locale)` to the method call. **Türkçe:** Belirli bir locale için formatlayıcıya ihtiyacınız varsa ne olur? Yeterince kolay sadece yöntem çağırısına `withLocale(locale)` ekleyin. **English:** Let's put it all together. Take a look at the following code snippet, which relies on a static import for the `java.time.format.FormatStyle.SHORT` value: `public static void print(DateTimeFormatter dtf, Türkçe: Hepsini bir araya getirelim. java.time.format.FormatStyle.SHORT değeri için statik içe aktarmaya dayanan aşağıdaki kod snippet'ine bir göz atın: public static void print(DateTimeFormatter dtf,`

```
LocalDateTime dateTime, Locale locale) {
    System.out.println(dtf.format(dateTime) + " -----
    "
    + dtf.withLocale(locale).format(dateTime));
}
public static void main(String[] args) {
    Locale.setDefault(new Locale("en", "US"));
    var italy = new Locale("it", "IT");
    var dt = LocalDateTime.of(2022, Month.OCTOBER, 20, 15, 12, 34);
```

```
// 10/20/22 -----
    20/10/22
    print(DateTimeFormatter.ofLocalizedDate(SHORT),dt,italy);
// 3:12 PM -----
    15:12
    print(DateTimeFormatter.ofLocalizedTime(SHORT),dt,italy);
// 10/20/22, 3:12 PM -----
    20/10/22, 15:12
    print(DateTimeFormatter.ofLocalizedDateTime(SHORT,SHORT),dt,italy);
}
```

English: First we establish `en_US` as the default locale, with `it_IT` as the requested locale. We then output each value using the two locales. As you can see, applying a locale has a big impact on the built-in date and time formatters. **Türkçe:** Önce `en_US`'u default locale olarak, onunla `it_IT`'yi istenen locale olarak kuruyoruz. Daha sonra her değeri iki locales kullanarak çıkarırız. Gördüğünüz gibi, bir locale uygulamak yerleşik tarih ve saat biçimlendiricileri üzerinde büyük bir etkiye sahiptir.

Specifying a Locale Category

English: When you call `Locale.setDefault()` with a locale, several display and formatting options are internally selected. If you require finer-grained control of the default locale, Java subdivides the underlying formatting options into distinct categories with the `Locale.Category` enum. **Türkçe:** locale ile `Locale.setDefault()` çağırıldığında, birkaç görüntüleme ve biçimlendirme seçeneği dahili olarak seçilir. default locale'nin daha ince bir şekilde kontrol edilmesi gerekiyorsa, Java altta yatan biçimlendirme seçeneklerini `Locale.Category` enum ile farklı kategorilere ayırır. **English:** The `Locale.Category` enum is a nested element in `Locale` that supports distinct locales for displaying and formatting data. For the exam, you should be familiar with the two enum values in Table 11.10. **Türkçe:** `Locale.Category` enum, verileri görüntülemek ve biçimlendirmek için farklı locales destekleyen `Locale` içindeki yuvalanmış bir elemandır. Sınav için Tablo 11.10'daki iki enum değerine aşina olmalısınız. **English:** TABLE 11.10 `Locale.Category` values

Value	Description
<code>DISPLAY</code>	Category used for displaying data about locale
<code>FORMAT</code>	Category used for formatting dates, numbers, or currencies

When you call `Locale.setDefault()` with a locale, the `DISPLAY` and `FORMAT` are set together. Let's take a look at an example: **Türkçe:** TABLE 11.10 `Locale.Category` değerleri Değer Açıklaması `DISPLAY` locale `FORMAT` ile ilgili verileri görüntülemek için kullanılan kategori `Locale.setDefault()` ile locale'yi aradığınızda, `DISPLAY` ve `FORMAT` birlikte set'dur. Bir örneğe bakalım:

```
public static void printCurrency(Locale locale, double money) {
    System.out.println(
        NumberFormat.getCurrencyInstance().format(money)
        + ", " + locale.getDisplayLanguage());
}
public static void main(String[] args) {
    var spain = new Locale("es", "ES");
    var money = 1.23;
```

English: 18: Türkçe: 18:

```
// Print with default locale
Locale.setDefault(new Locale("en", "US"));
printCurrency(spain, money); // $1.23, Spanish
```

English: 22: Türkçe: 22:

```
// Print with selected locale display
Locale.setDefault(Category.DISPLAY, spain);
printCurrency(spain, money); // $1.23, español
```

English: 26: Türkçe: 26:

```
// Print with selected locale format
Locale.setDefault(Category.FORMAT, spain);
printCurrency(spain, money); // 1,23 €, español
}
```

English: The code prints the same data three times. First it prints the language of the spain and money variables using the locale en_US. Then it prints it using the DISPLAY category of es_ES, while the FORMAT category remains en_US. Finally, it prints the data using both categories set to es_ES. **Türkçe:** Kod aynı veriyi üç kez basıyor. İlk önce locale en_US kullanarak spain ve para değişkenlerinin dilini yazdırır. Daha sonra DISPLAY es_ES kategorisini kullanarak yazdırırken, FORMAT kategorisi en_US olarak kalır. Son olarak, her iki kategoriye de kullanarak verileri set es_ES olarak yazdırır. **English:** For the exam, you do not need to memorize the various display and formatting options for each category. You just need to know that you can set parts of the locale independently. You should also know that calling `Locale.setDefault(us)` after the previous code snippet will change both locale categories to en_US. **Türkçe:** Sınav için, her kategori için çeşitli ekran ve biçimlendirme seçeneklerini ezberlemeniz gerekmez. Sadece locale bölümlerini bağımsız olarak set yapabileceğinizi bilmeniz gerekir. Ayrıca, önceki kod snippet'inden sonra `Locale.setDefault(us)` aramasının her iki locale kategorisini de en_US olarak değiştireceğini bilmelisiniz.

Loading Properties with Resource Bundles

English: Up until now, we've kept all of the text strings displayed to our users as part of the program inside the classes that use them. Localization requires externalizing them to elsewhere. **Türkçe:** Şimdiye kadar, strings metninin tümünü, onları kullanan sınıfların içindeki programın bir parçası olarak kullanıcılarımıza gösterdik. Yerelleştirme, onları başka bir yere dışsallaştırmayı gerektirir. **English:** A resource bundle contains the locale-specific objects to be used by a program. It is like a map with keys and values. The resource bundle is commonly stored in a properties file. A properties file is a text file in a specific format with key/value pairs. **Türkçe:** Bir resource bundle, bir program tarafından kullanılacak locale özellikli nesneleri içerir. Anahtarları ve değerleri olan bir map gibidir. resource bundle genellikle bir properties file içinde saklanır. properties file, anahtar/değer çiftleri ile belirli bir formatta bir metin dosyasıdır. **English:** Our zoo program has been successful. We are now getting requests to use it at three more zoos! We already have support for U.S.- based zoos. We now need to add Zoo de La Palmyre in France, the Greater Vancouver Zoo in English-speaking Canada, and Zoo de Granby in French-speaking Canada. **Türkçe:** Hayvanat bahçemiz başarılı oldu. Şimdi üç hayvanat bahçesinde daha kullanmak için talep alıyoruz! U.S.- tabanlı hayvanat bahçeleri için zaten desteğimiz var. Şimdi Fransa'daki Zoo de La Palmyre, İngilizce konuşan Kanada'daki Greater Vancouver Hayvanat Bahçesi ve Fransızca konuşan Kanada'daki Zoo de Granby'yi eklememiz gerekiyor. **English:** We immediately realize that we are going to need to internationalize our program. Resource bundles will be quite helpful. They will let us easily translate our application to multiple locales or even support multiple locales at once. It will also be easy to add more locales later if zoos in even more countries are interested. We thought about which locales we need to support, and we came up with four: **Türkçe:** Programımızı uluslararasılaştırmamız gerektiğini hemen fark ediyoruz. Resource bundles oldukça yardımcı olacaktır. Uygulamamızı kolayca birden fazla locales'ye çevirmemize ve hatta aynı anda birden fazla locales'yi desteklememize izin verecekler. Daha da fazla ülkede hayvanat bahçelerinin ilgilenmesi durumunda daha fazla locales eklemek de kolay olacaktır. Hangi locales'ü desteklememiz gerektiğini düşündük ve dört tane bulduk:

English: `Locale us` **Türkçe:** `Localebiz`

```
= new Locale("en", "US");
Locale france = new Locale("fr", "FR");
Locale englishCanada = new Locale("en", "CA");
Locale frenchCanada = new Locale("fr", "CA");
```

English: In the next sections, we create a resource bundle using properties files. It is conceptually similar to a `Map<String,String>`, with each line representing a different key/value. The key and value are separated by an equal sign (=) or colon (:). To keep things simple, we use an equal sign throughout this chapter. We also look at how Java determines which resource bundle to use. **Türkçe:** Sonraki bölümlerde properties files kullanarak bir resource bundle oluşturuyoruz. Kavramsal olarak bir `Map<String,String>` ile benzerlik gösterir ve her satır farklı bir anahtar/değeri temsil eder. Anahtar ve değer eşit bir işaret (=) veya kolon (:) ile ayrılır. İşleri basit tutmak için, bu bölüm boyunca eşit bir işaret kullanıyoruz. Ayrıca Java'nın hangi resource bundle'nin kullanılacağını nasıl belirlediğine de bakıyoruz.

Creating a Resource Bundle

English: We're going to update our application to support the four locales listed previously. Luckily, Java doesn't require us to create four different resource bundles. If we don't have a countryspecific resource bundle, Java will use a language-specific one. It's a bit more involved than this, but let's start with a simple example. **Türkçe:** Daha önce listelenen dört locales desteklemek için uygulamamızı güncelleyeceğiz. Neyse ki, Java dört farklı resource bundles oluşturmamızı gerektirmez. Ülkeye özgü bir resource bundle yoksa, Java dile özgü bir tane kullanır. Bundan biraz daha ilgili, ama basit bir örnek ile başlayalım. **English:** For now, we need English and French properties files for our Zoo resource bundle. First, create two properties files. **Türkçe:** Şimdilik resource bundle Hayvanat Bahçemiz için İngilizce ve Fransızca properties files gerekiyor. İlk olarak, iki properties files oluşturun. **English:** Zoo_en.properties hello=Hello open=The zoo is open Zoo_fr.properties hello=Bonjour open=Le zoo est ouvert The filenames match the name of our resource bundle, Zoo. They are then followed by an underscore (_), target locale, and.properties file extension. We can write our very first program that uses a resource bundle to print this information. **Türkçe:** Zoo_en.properties alo=Hello open=The zoo is open Zoo_fr.properties alo=Bonjour open=Le zoo est ouvert Dosya adları resource bundle, Zoo'nun adı ile eşleşir. Daha sonra bir alt çizgi (_), hedef locale, and.properties dosya uzantısı takip edilir. Bu bilgileri yazdırmak için resource bundle kullanan ilk programımızı yazabiliriz.

```
public static void printWelcomeMessage(Locale locale) {
    var rb = ResourceBundle.getBundle("Zoo", locale);
    System.out.println(rb.getString("hello")
        + ", " + rb.getString("open"));
}

public static void main(String[] args) {
    var us = new Locale("en", "US");
    var france = new Locale("fr", "FR");
    printWelcomeMessage(us); // Hello, The zoo is open
    printWelcomeMessage(france); // Bonjour, Le zoo est ouvert
}
```

English: Lines 16 and 17 create the two locales that we want to test, but the method on lines 10–14 does the actual work. Line 11 calls a factory method on `ResourceBundle` to get the right resource bundle. Lines 12 and 13 retrieve the right string from the resource bundle and print the results. **Türkçe:** 16 ve 17 satırları test etmek istediğimiz iki locales oluşturur, ancak 10-14 satırlarındaki yöntem gerçek işi yapar. Doğru resource bundle elde etmek için `ResourceBundle` üzerindeki factory method satırı çağrılır. 12 ve 13 numaralı satırlar resource bundle'den sağ string'u alır ve sonuçları yazdırır. **English:** Since a resource bundle contains key/value pairs, you can even loop through them to list all of the pairs. The `ResourceBundle` class provides a `keySet()` method to get a set of all keys. **Türkçe:** Bir resource bundle anahtar/değer çiftleri içerdiğinden, bunların içinden list çiftlerinin tümüne bile döngü yapabilirsiniz. `ResourceBundle` sınıfı, tüm anahtarlardan bir set elde etmek için bir `keySet()` yöntemi sağlar.

```
var us = new Locale("en", "US");
ResourceBundle rb = ResourceBundle.getBundle("Zoo", us);
rb.keySet().stream()
    .map(k -> k + ": " + rb.getString(k))
    .forEach(System.out::println);
```

English: This example goes through all of the keys. It maps each key to a `String` with both the key and the value before printing everything. **Türkçe:** Bu örnek tüm anahtarlardan geçer. maps her anahtarı, her şeyi yazdırmadan önce hem anahtarı hem de değeri olan bir `String` için. **English:** hello: Hello open: The zoo is open Loading Resource Bundle Files at Runtime For the exam, you don't need to know where the properties files for the resource bundles are stored. If the exam provides a properties file, it is safe to assume that it exists and is loaded at runtime. **Türkçe:** Merhaba: Merhaba açık: Hayvanat bahçesi açık Yükleme Resource Bundle Runtime'da Dosyalar Sınav için, resource bundles için properties files'nin nerede saklandığını bilmenize gerek yoktur. Sınav bir properties file sağlıyorsa, var olduğunu ve çalışma zamanında yüklendiğini varsaymak güvenlidir. **English:** In your own applications, though, the resource bundles can be stored in a variety of places. While they can be stored inside the JAR that uses them, doing so is not recommended. This approach forces you to rebuild the application JAR any time some text changes. One of the benefits of using resource bundles is to decouple the application code from the localespecific text data. **Türkçe:** Bununla birlikte, kendi uygulamalarınızda, resource bundles çeşitli yerlerde saklanabilir. Onları kullanan JAR içinde saklanabilirlerken, bunu yapmak önerilmez. Bu yaklaşım, bazı metin değişiklikleri olduğunda uygulamayı JAR yeniden oluşturmaya zorlar. resource bundles kullanmanın avantajlarından biri, uygulama kodunu yerel özgül metin verilerinden ayırmaktır. **English:** Another approach is to have all of the properties files in a separate properties JAR or folder and load them in the classpath at runtime. In this manner, a new language can be added without changing the application JAR. **Türkçe:** Başka bir yaklaşım, properties files'nin tümünü ayrı bir properties JAR veya klasörde bulundurmak ve çalışma zamanında sınıf yoluna yüklemektir. Bu şekilde JAR uygulamasını değiştirmeden yeni bir dil eklenebilir.

Picking a Resource Bundle

English: There are two methods for obtaining a resource bundle that you should be familiar with for the exam. **Türkçe:** Sınav için aşina olmanız gereken bir resource bundle elde etmek için iki yöntem vardır.

```
ResourceBundle.getBundle("name");
ResourceBundle.getBundle("name", locale);
```

English: The first uses the default locale. You are likely to use this one in programs that you write. Either the exam tells you what to assume as the default locale, or it uses the second approach. **Türkçe:** İlkinde default locale kullanılır. Bunu yazdığınız programlarda kullanmanız muhtemeldir. Sınav size default locale olarak neyi varsaymanız gerektiğini söyler veya ikinci yaklaşımı kullanır. **English:** Java handles the logic of picking the best available resource bundle for a given key. It tries to find the most specific value. Table 11.11 shows what Java goes through when asked for resource bundle Zoo with the locale new Locale("fr", "FR") when the default locale is U.S. English. **Türkçe:** Java, belirli bir anahtar için mevcut en iyi resource bundle seçme mantığını ele alır. En özel değeri bulmaya çalışır. Tablo 11.11, default locale U.S olduğunda locale yeni Locale("fr", "FR") ile resource bundle Hayvanat Bahçesi istendiğinde Java'nın neler yaşadığını gösterir. İngilizce. **English:** TABLE 11.11 Picking a resource bundle for French/France with default locale English/US Step Looks for file Reason 1 Zoo_fr_FR.properties Requested locale 2 Zoo_fr.properties Language we requested with no country 3 Zoo_en_US.properties Default locale 4 Zoo_en.properties Default locale's language with no country 5 Zoo.properties No locale at all— default bundle 6 If still not found, throw No locale or default bundle available MissingResource Exception As another way of remembering the order of Table 11.11, learn these steps: **Türkçe:** TABLE 11.11 Varsayılan locale ile Fransızca / Fransa için bir kaynak paketi seçmek İngilizce / ABD Adım Dosyaya bakar Nedeni 1 Zoo_fr_FR.properties Requested locale 2 Zoo_fr.properties Hiçbir ülke ile talep ettiğimiz dil 3 Zoo_en_US.properties Öntanımlı locale 4 Zoo_en.properties Öntanımlı olarak bu locale'nin dilini öğrenin 5 Zoo.properties No locale no locale atreaded **English:** 1.Look for the resource bundle for the requested locale, followed by the one for the default locale. **Türkçe:** 1.İstenen locale için resource bundle, ardından default locale için olanı arayın. **English:** 2.For each locale, check the language/country, followed by just the language. 3.Use the default resource bundle if no matching locale can be found. **Türkçe:** 2.Her locale için dili/ülkeyi kontrol edin, sadece dili takip edin. 3. Eğer locale ile eşleşen yoksa varsayılan resource bundle kullanın. **English:** As we mentioned earlier, Java supports resource bundles from Java classes and properties alike. When Java is searching for a matching resource bundle, it will first check for a resource bundle file with the matching class name. For the exam, you just need to know how to work with properties files. **Türkçe:** Daha önce de belirttiğimiz gibi, Java Java sınıflarından resource bundles ve properties benzer şekilde destekler. Java eşleşen bir resource bundle aradığında, önce eşleşen sınıf adına sahip bir resource bundle dosyası olup olmadığını kontrol eder. Sınav için, sadece properties files ile nasıl çalışılacağını bilmeniz gerekir.

English: Let's see if you understand Table 11.11. What is the maximum number of files that Java would need to consider in order to find the appropriate resource bundle with the following code? **Türkçe:** Tablo 11.11'i anlayıp anlamadığınızı görelim. Aşağıdaki kodla uygun kaynak paketini bulmak için Java'nın göz önünde bulundurması gereken maksimum dosya sayısı nedir?

```
Locale.setDefault(new Locale("hi"));
ResourceBundle rb = ResourceBundle.getBundle("Zoo", new Locale("en"));
```

English: The answer is three. They are listed here: 1. Zoo_en.properties 2. Zoo_hi.properties 3. Zoo.properties The requested locale is en, so we start with that. Since the en locale does not contain a country, we move on to the default locale, hi. Again, there's no country, so we end with the default bundle. **Türkçe:** Cevap üç. Burada listelenmiştir: 1. Zoo_en.properties 2. Zoo_hi.properties 3. Zoo.properties İstenen locale en'dir, bu yüzden bununla başlarız. En locale bir ülke içermediğinden, default locale, merhaba'ya geçiyoruz. Yine, ülke yok, bu yüzden varsayılan bohça ile bitiyoruz.

Selecting Resource Bundle Values

English: Got all that? Good— because there is a twist. The steps that we've discussed so far are for finding the matching resource bundle to use as a base. Java isn't required to get all of the keys from the same resource bundle. It can get them from any parent of the matching resource bundle. A parent resource bundle in the hierarchy just removes components of the name until it gets to the top. Table 11.12 shows how to do this. **Türkçe:** Hepsi bu kadar mı? İyi - çünkü bir bükülme var. Şimdiye kadar tartıştığımız adımlar, temel olarak kullanmak için eşleşen resource bundle bulmak içindir. Java tüm anahtarları aynı resource bundle dan almak için gerekli değildir. Onları eşleşen resource bundle herhangi bir ebeveynden alabilir. Hiyerarşideki bir ebeveyn resource bundle sadece üst seviyeye gelene kadar ismin bileşenlerini kaldırır. Tablo 11.12 bunun nasıl yapılacağını gösterir. **English:** TABLE 11.12 Selecting resource bundle properties Matching resource bundle Properties files keys can come from Zoo_fr_FR Zoo_fr_FR.properties Zoo_fr.properties Zoo.properties Once a resource bundle has been selected, only properties along a single hierarchy will be used. Contrast this behavior with Table 11.11, in which the default en_US resource bundle is used if no other resource bundles are available. **Türkçe:** TABLE 11.12 resource bundle properties Eşleştirme resource bundle Properties files tuşları Zoo_fr_FR Zoo_fr_FR.properties Zoo_fr.properties Zoo.properties Bir kez resource bundle seçildikten sonra, tek bir hiyerarşi boyunca sadece properties kullanılacaktır. Bu davranışı, başka bir resource bundles yoksa varsayılan en_US resource bundle kullanıldığı Tablo 11.11 ile karşılaştırın. **English:** What does this mean, exactly? Assume the requested locale is fr_FR and the default is en_US. The JVM will provide data from en_US only if there is no matching fr_FR or fr resource bundle. If it finds a fr_FR or fr resource bundle, then only those bundles, along with the default bundle, will be used. **Türkçe:** Bu tam olarak ne anlama geliyor? İstenen locale fr_FR olduğunu ve varsayılanın en_US olduğunu varsayalım. JVM, yalnızca eşleşen fr_FR veya fr resource bundle yoksa en_US'tan veri sağlayacaktır. Bir fr_FR veya fr resource bundle bulursa, varsayılan demetle birlikte yalnızca bu demetler kullanılacaktır. **English:** Let's put all of this together and print some information about our zoos. We have a number of properties files this time. **Türkçe:** Bütün bunları bir araya getirelim ve hayvanat bahçelerimiz hakkında bazı bilgiler yayınlayalım. Bu sefer properties files sayımız var.

English: Zoo.properties name=Vancouver Zoo Zoo_en.properties hello=Hello open=is open Zoo_en_US.properties name=The Zoo Zoo_en_CA.properties visitors=Canada visitors Suppose that we have a visitor from Québec (which has a default locale of French Canada) who has asked the program to provide information in English. What do you think this outputs? **Türkçe:** Zoo.properties name=Vancouver Zoo Zoo_en.properties alo=Hello open=is open Zoo_en_US.properties name=The Zoo Zoo_en_CA.properties visitors=Canada visitors Diyelim ki Qubec'ten (Fransız Kanada'nın default locale bir ziyaretçimiz var) programdan İngilizce bilgi vermesini istedi. Sence bu ne çıktı?

```
Locale.setDefault(new Locale("en", "US"));
Locale locale = new Locale("en", "CA");
ResourceBundle rb = ResourceBundle.getBundle("Zoo", locale);
System.out.print(rb.getString("hello"));
System.out.print(". ");
System.out.print(rb.getString("name"));
System.out.print(" ");
System.out.print(rb.getString("open"));
System.out.print(" ");
System.out.print(rb.getString("visitors"));
```

English: The program prints the following: **Türkçe:** Program aşağıdakileri yazdırır: **English:** Hello. Vancouver Zoo is open Canada visitors The default locale is en_US, and the requested locale is en_CA. First, Java goes through the available resource bundles to find a match. It finds one right away with Zoo_en_CA.properties. This means the default locale of en_US is irrelevant. **Türkçe:** Merhabalar. Vancouver Hayvanat Bahçesi açık Kanada ziyaretçileri default locale en_US ve talep edilen locale en_CA'dır. İlk olarak, Java bir eşleşme bulmak için mevcut resource bundles üzerinden geçer. Zoo_en_CA.properties ile hemen bir tane bulur. Bu, en_US'un default locale ile alakasız olduğu anlamına gelir. **English:** Line 14 doesn't find a match for the key hello in Zoo_en_CA.properties, so it goes up the hierarchy to Zoo_en.properties. Line 16 doesn't find a match for name in either of the first two properties files, so it has to go all the way to the top of the hierarchy to Zoo.properties. Line 18 has the same experience as line 14, using Zoo_en.properties. Finally, line 20 has an easier job of it and finds a matching key in Zoo_en_CA.properties. **Türkçe:** Satır 14, Zoo_en_CA.properties'deki anahtar merhaba için bir eşleşme bulamaz, bu nedenle hiyerarşiyi Zoo_en.properties'e çıkarır. Hat 16, ilk iki properties files da isim için bir eşleşme bulmaz, bu nedenle hiyerarşinin en üst noktasına kadar Zoo.properties gitmek zorundadır. Line 18, Zoo_en.properties kullanarak line 14 ile aynı deneyime sahiptir. Son olarak, 20. satır daha kolay bir işe sahiptir ve Zoo_en_CA.properties içinde eşleşen bir anahtar bulur. **English:** In this example, only three properties files were used: Zoo_en_CA.properties, Zoo_en.properties, and Zoo.properties. Even when the property wasn't found in en_CA or en resource bundles, the program preferred using Zoo.properties (the default resource bundle) rather than Zoo_en_US.properties (the default locale). **Türkçe:** Bu örnekte sadece üç properties files kullanıldı: Zoo_en_CA.properties, Zoo_en.properties ve Zoo.properties. Mülk en_CA'da veya en resource bundles'te bulunmadığında bile, program Zoo_en_US.properties (default locale) yerine Zoo.properties (varsayılan resource bundle) kullanmayı tercih etti.

English: What if a property is not found in any resource bundle? Then an exception is thrown. For example, attempting to call `rb.getString("close")` in the previous program results in a `MissingResourceException` at runtime. **Türkçe:** Bir mülk herhangi bir resource bundle içinde bulunmazsa ne olur? Sonra bir istisna atılır. Örneğin, önceki programdaki `rb.getString("close")` çağırma çalışmak, çalışma zamanında `MissingResourceException` ile sonuçlanır.

Formatting Messages

English: Often we just want to output the text data from a resource bundle, but sometimes you want to format that data with parameters. In real programs, it is common to substitute variables in the middle of a resource bundle string. The convention is to use a number inside braces such as `{0}`, `{1}`, etc. The number indicates the order in which the parameters will be passed. Although resource bundles don't support this directly, the `MessageFormat` class does. **Türkçe:** Genellikle sadece metin verilerini bir resource bundle'dan çıkarmak isteriz, ancak bazen bu verileri parametrelerle biçimlendirmek istersiniz. Gerçek programlarda, değişkenleri resource bundle string ortasında değiştirmek yaygındır. Sözleşme, 0, 1, vb. gibi dış tellerinin içindeki bir sayıyı kullanmaktır. Sayı, parametrelerin geçileceği sırayı belirtir. resource bundles bunu doğrudan desteklemese de, `MessageFormat` sınıfı destekler. **English:** For example, suppose that we had this property defined: **Türkçe:** Örneğin, bu özelliği tanımladığımızı varsayalım:

```
helloByName=Hello, {0} and {1}
```

English: In Java, we can read in the value normally. After that, we can run it through the `MessageFormat` class to substitute the parameters. The second parameter to `format()` is a `vararg`, allowing you to specify any number of input values. **Türkçe:** Java 'de değeri normal olarak okuyabiliriz. Bundan sonra, parametreleri değiştirmek için `MessageFormat` sınıfı üzerinden çalıştırabiliriz. `format()` için ikinci parametre bir `vararg` olup, herhangi bir giriş değerini belirtmenizi sağlar. **English:** Suppose we have a resource bundle `rb`: **Türkçe:** resource bundle `rb`'miz olduğunu varsayalım:

```
String format = rb.getString("helloByName");
System.out.print(MessageFormat.format(format, "Tammy", "Henry"));
```

English: This will print the following: **Türkçe:** Bu, aşağıdakileri yazdıracaktır: **English:** Hello, Tammy and Henry **Türkçe:** Merhaba Tammy ve Henry.

Using the Properties Class

English: When working with the `ResourceBundle` class, you may also come across the `Properties` class. It functions like the `HashMap` class that you learned about in Chapter 9, "Collections and Generics," except that it uses `String` values for the keys and values. Let's create one and set some values. **Türkçe:** `ResourceBundle` sınıfıyla çalışırken `Properties` sınıfıyla da karşılaşabilirsiniz. Bölüm 9'da öğrendiğiniz `HashMap` sınıfı gibi functions, anahtarlar ve değerler için `String` değerlerini kullanması dışında, "Collections ve Genel". Bir ve set değerleri oluşturalım.

```
import java.util.Properties;
public class ZooOptions {
    public static void main(String[] args) {
        var props = new Properties();
        props.setProperty("name", "Our zoo");
        props.setProperty("open", "10am");
    }
}
```

English: The `Properties` class is commonly used in handling values that may not exist. **Türkçe:** `Properties` sınıfı, mevcut olmayan değerlerin ele alınmasında yaygın olarak kullanılır.

```
System.out.println(props.getProperty("camel")); // null
System.out.println(props.getProperty("camel", "Bob")); // Bob
```

English: If a key were passed that actually existed, both statements would print it. This is commonly referred to as providing a default, or a backup value, for a missing key. **Türkçe:** Eğer gerçekten var olan bir anahtar geçilseydi, her iki ifade de onu basardı. Bu genellikle eksik bir anahtar için bir varsayılan veya bir yedekleme değeri sağlama olarak adlandırılır. **English:** The `Properties` class also includes a `get()` method, but only `getProperty()` allows for a default value. For example, the following call is invalid since `get()` takes only a single parameter: **Türkçe:** `Properties` sınıfı ayrıca bir `get()` yöntemi içerir, ancak yalnızca `getProperty()` varsayılan bir değere izin verir. Örneğin, `get()` yalnızca tek bir parametre aldığından aşağıdaki çağrı geçersizdir:

```
props.get("open"); // 10am
props.get("open", "The zoo will be open soon"); // DOES NOT COMPILE
```

Summary

English: This chapter covered a wide variety of topics centered around building applications that respond well to change. We started our discussion with exception handling. Exceptions can be divided into two categories: checked and unchecked. In Java, checked exceptions inherit `Exception` but not `RuntimeException` and must be handled or declared. Unchecked exceptions inherit `RuntimeException` or `Error` and do not need to be handled or declared. It is considered a poor practice to catch an `Error`.

Türkçe: Bu bölüm, değişime iyi yanıt veren yapı uygulamaları etrafında yoğunlaşan çok çeşitli konuları kapsıyordu. İstisnai işlemlerle görüşmeye başladık. İstisnalar iki kategoriye ayrılabilir: kontrol edilmiş ve kontrol edilmemiş. Java'de checked exceptions varis `Exception` ancak `RuntimeException` değil ve ele alınmalı veya ilan edilmelidir. Unchecked exceptions miras `RuntimeException` veya `Error` ve ele alınması veya ilan edilmesi gerekmez. Bir `Error` yakalamak için kötü bir uygulama olarak kabul edilir.

English: You can create your own checked or unchecked exceptions by extending `Exception` or `RuntimeException`, respectively. You can also define custom constructors and messages for your exceptions, which will show up in stack traces. **Türkçe:** Sırasıyla `Exception` veya `RuntimeException` uzatarak kendi kontrol edilen veya unchecked exceptions oluşturabilirsiniz. İstisnalarınız için özel yapıcılar ve mesajlar da tanımlayabilirsiniz, bu da yığın izlerinde görünecektir.

English: Automatic resource management can be enabled by using a try-with-resources statement to ensure that the resources are properly closed. Resources are closed at the conclusion of the try block, in the reverse of the order in which they are declared. A suppressed exception occurs when more than one exception is thrown, often as part of a finally block or try-with-resources `close()` operation. **Türkçe:** Otomatik kaynak yönetimi, kaynakların düzgün bir şekilde kapatılmasını sağlamak için bir try-with-resources statement kullanılarak etkinleştirilebilir. Kaynaklar, ilan edildikleri sıranın tersine, try block sonunda kapatılır. Bir suppressed exception, genellikle bir finally block veya `close()` işleminin bir parçası olarak birden fazla istisna atıldığında ortaya çıkar.

English: Java includes a number of built-in classes to format numbers and dates. We reviewed how to create custom formatters for each. You should be able to read these custom formats when you encounter them on the exam. **Türkçe:** Java, sayıları ve tarihleri biçimlendirmek için bir dizi yerleşik sınıf içerir. Her biri için özel biçimlendiricilerin nasıl oluşturulacağını gözden geçirdik. Sınavda karşılaştığınızda bu özel formatları okuyabilmelisiniz.

English: Localization involves creating programs that adapt to change. You can create a `Locale` class with a required lowercase language code and optional uppercase country code. For example, `en` and `en_US` are locales for English and U.S. English, respectively. You need to know how to format number and date/time values based on locale, including the new `CompactNumberFormat` class. **Türkçe:** Yerelleştirme, değişime uyum sağlayan programlar oluşturmayı içerir. Gerekli bir küçük harf dil kodu ve optional büyük harf ülke koduna sahip bir `Locale` sınıfı oluşturabilirsiniz. Örneğin, `en` ve `en_US` İngilizce için locales ve U.S. dir. Sırasıyla İngilizce. Yeni `CompactNumberFormat` sınıfı da dahil olmak üzere locale tabanlı sayı ve tarih/saat değerlerini nasıl biçimlendireceğinizi bilmeniz gerekir.

English: A ResourceBundle allows specifying key/value pairs in a properties file. Java goes through candidate resource bundles from the most specific to the most general to find a match. If no matches are found for the requested locale, Java switches to the default locale and then finally the default resource bundle. Once a matching resource bundle is found, Java looks only in the hierarchy of that resource bundle to select values. **Türkçe:** Bir ResourceBundle, bir properties file içindeki anahtar/değer çiftlerinin belirlenmesine izin verir. Java bir eşleşme bulmak için en özelden en genele resource bundles aday geçer. İstenen locale için herhangi bir eşleşme bulunmazsa, Java default locale ve son olarak varsayılan resource bundle'e geçer. Eşleşen bir resource bundle bulunduktan sonra, Java değerleri seçmek için yalnızca o resource bundle hiyerarşisinde görünür.

English: By applying the principles you learned about in this chapter to your own projects, you can build applications that last longer, with built-in support for whatever unexpected events may arise. **Türkçe:** Bu bölümde öğrendiğiniz ilkeleri kendi projelerinize uygulayarak, beklenmedik olaylar ortaya çıkarsa yerleşik destekle daha uzun ömürlü uygulamalar oluşturabilirsiniz.

Exam Essentials

English: Understand the various types of exceptions. All exceptions are subclasses of `java.lang.Throwable`. Subclasses of `java.lang.Error` should never be caught. Only subclasses of `java.lang.Exception` should be handled in application code. **Türkçe:** exceptions. All istisnalarının çeşitli türlerini anlamak `java.lang.Throwable` alt sınıflarıdır. `java.lang.Error` alt sınıfları asla yakalanmamalıdır. Sadece `java.lang.Exception` alt sınıfları uygulama kodunda ele alınmalıdır. **English:** Differentiate between checked and unchecked exceptions. Unchecked exceptions do not need to be caught or handled and are subclasses of `java.lang.RuntimeException` or `java.lang.Error`. All other subclasses of `java.lang.Exception` are checked exceptions and must be handled or declared. **Türkçe:** Kontrol edilen ve kontrol edilmeyen exceptions. Unchecked istisnaları arasında ayırım yapmak, yakalanmak veya ele almak zorunda değildir ve `java.lang.RuntimeException` veya `java.lang.Error` alt sınıflarıdır. `java.lang.Exception`'un diğer tüm alt sınıfları checked exceptions'tür ve ele alınmalı veya ilan edilmelidir. **English:** Understand the flow of a try statement. A try statement must have a catch or a finally block. Multiple catch blocks can be chained together, provided no superclass exception type appears in an earlier catch block than its subclass. A multi-catch expression may be used to handle multiple exceptions in the same catch block, provided one exception is not a subclass of another. The finally block runs last regardless of whether an exception is thrown. **Türkçe:** Bir try statement. A try ifadesinin akışını anlayın, bir yakalama veya finally block olmalıdır. Birden fazla catch blocks, alt sınıfından daha önceki bir catch block içinde süper sınıf istisna türü görünmediği sürece birlikte zincirlenebilir. Aynı catch block içinde birden fazla istisnayı işlemek için çok yakalamalı bir ifade kullanılabilir, ancak bir istisna diğerinin alt sınıfı değildir. finally block bir istisna atılıp atılmadığına bakılmaksızın son çalışır. **English:** Be able to follow the order of a try-with-resources statement. A try-with-resources statement is a special type of try block in which one or more resources are declared and automatically closed in the reverse of the order in which they are declared. It can be used with or without a catch or finally block, with the implicit finally block always executed first. **Türkçe:** try-with-resources statement. A try-with-resources statement sırasını takip edebilmek, bir veya daha fazla kaynağın beyan edildiği ve ilan edildikleri sıranın tersine otomatik olarak kapatıldığı özel bir try block türüdür. Bir yakalama veya finally block ile birlikte veya olmadan kullanılabilir, örtülü finally block her zaman önce çalıştırılır. **English:** Be able to write methods that declare exceptions. Understand the difference between the throw and throws keywords and how to declare methods with exceptions. Know how to correctly override a method that declares exceptions. **Türkçe:** exceptions. Understand atma ve atma arasındaki farkı ve istisnalarla yöntemlerin nasıl beyan edileceğini bildiren yöntemler yazabilin. İstisnalar bildiren bir yöntemi doğru bir şekilde geçersiz kılmayı bilin. **English:** Identify valid locale strings. Know that the language code is lowercase and mandatory, while the country code is uppercase and optional. Be able to select a locale using a built-in constant, constructor, or builder class. **Türkçe:** Geçerli locale strings. Know dil kodunun küçük ve zorunlu olduğunu, ülke kodunun büyük ve optional olduğunu belirtin. Yerleşik sabit, constructor veya builder sınıfı kullanarak bir locale seçebilme. **English:** Format dates, numbers, and messages. Be able to format dates, numbers, and messages into various String formats, and know how locale influences these formats. Know how the various number formatters (currency, percent, compact) differ. Be able to write a custom date or number formatter using symbols, including how to escape literal values. **Türkçe:** Tarihleri, sayıları ve messages. Be iletileri çeşitli String formatlarına biçimlendirebilir ve locale'nin bu formatları nasıl etkilediğini bilir. Çeşitli sayı formatlayıcılarının (para birimi, yüzde, kompakt) nasıl farklılık gösterdiğini bilin. Gerçek değerlerden nasıl kaçılacağı da dahil olmak üzere semboller kullanarak özel bir tarih veya sayı biçimlendirici yazabilme. **English:** Determine which resource bundle Java will use to look up a key. Be able to create resource bundles for a set of locales using properties files. Know the search order that Java uses to select a resource bundle and how the default locale and default resource bundle are considered. Once a resource bundle is found, recognize the hierarchy used to select values. **Türkçe:** resource bundle properties files kullanarak set locales için key. Be oluşturabilen bir resource bundles aramak için hangi Java kullanacağını belirleyin. Java'nın bir resource bundle seçmek için kullandığı arama sırasını ve default locale ve varsayılan resource bundle'un nasıl kabul edildiğini bilin. Bir resource bundle bulunduktan sonra, değerleri seçmek için kullanılan hiyerarşiyi tanıyın.

Review Questions

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Bölüm inceleme sorularının cevapları Ek'te bulunabilir. **English:** 1. Which of the following can be inserted on line 8 to make this code compile? (Choose all that apply.) **Türkçe:** 1. Bu kodu derlemek için aşağıdakilerden hangisi 8. satıra eklenebilir? (Tüm geçerli olanları seçin.)

```
public void whatHappensNext() throws IOException {
// INSERT CODE HERE
}
```

English: A. System.out.println("it's ok"); **Türkçe:** A. System.out.println("it's ok"); **English:** B. throw new Exception(); **Türkçe:** B. yeni Exception() atın; **English:** C. throw new IllegalArgumentException(); **Türkçe:** C. yeni IllegalArgumentException() at; **English:** D. throw new java.io.IOException(); **Türkçe:** D. yeni java.io.IOException() atın; **English:** E. throw new RuntimeException(); **Türkçe:** E. yeni RuntimeException() at; **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbirisi **English:** 2. Which statement about the following class is correct? **Türkçe:** 2. Aşağıdaki sınıfla ilgili hangi ifade doğrudur?

```
class Problem extends Exception {
public Problem() {}
}
class YesProblem extends Problem {}
public class MyDatabase {
public static void connectToDatabase() throw Problem {
throws new YesProblem();
}
public static void main(String[] c) throw Exception {
connectToDatabase();
}
}
```

English: A. The code compiles and prints a stack trace for YesProblem at runtime. **Türkçe:** A. The kodu, çalışma zamanında YesProblem için bir yığın izi derler ve yazdırır. **English:** B. The code compiles and prints a stack trace for Problem at runtime. **Türkçe:** B. The kodu, çalışma zamanında Problem için bir yığın izi derler ve yazdırır. **English:** C. The code does not compile because Problem defines a constructor. **Türkçe:** C. The kodu derlemez çünkü Problem bir constructor tanımlar. **English:** D. The code does not compile because YesProblem does not define a constructor. **Türkçe:** D. The kodu derlemez çünkü YesProblem bir constructor tanımlamaz. **English:** E. The code does not compile but would if Problem and YesProblem were switched on lines 6 and 7. **Türkçe:** E. The kodu derlemez, ancak Sorun ve YesProblem satırlarda açılırsa **English:** 6 and 7. **Türkçe:** 6 ve 7. **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbirisi

English: 3. Which of the following are common types to localize? (Choose all that apply.) **Türkçe:** 3. Aşağıdakilerden hangisini lokalize etmek için yaygın türleri vardır? (Tüm geçerli olanları seçin.) **English:** A. Dates **Türkçe:** A. Tarihler **English:** B. Lambda expressions **Türkçe:** B. Lambda ifadeleri **English:** C. Class names **Türkçe:** C. Sınıf isimleri **English:** D. Currency **Türkçe:** D. Para Birimi **English:** E. Numbers **Türkçe:** E. Sayılar **English:** F. Variable names **Türkçe:** F. Değişken isimler **English:** 4. What is the output of the following snippet, assuming a and b are both 0? **Türkçe:** 4. a ve b'nin her ikisinin de 0 olduğunu varsayarak aşağıdaki snippet'in çıktısı nedir?

```
try {
    System.out.print(a / b);
} catch (RuntimeException e) {
    System.out.print(- 1);
} catch (ArithmeticException e) {
    System.out.print(0);
} finally {
    System.out.print("done");
}
```

English: A. **Türkçe:** A. **English:** ---- 1 B. 0 C. done-1 D. done0 E. The code does not compile. **Türkçe:** ---- 1 B. 0 C. doed-1 D. doed0 E. The kodu derlemez. **English:** F. An uncaught exception is thrown. **Türkçe:** F. An Yakalanamayan istisna atılır. **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri
English: 5. Assuming the current locale uses dollars (\$) and the following method is called with a double value of 100_102.2, which of the following values are printed? (Choose all that apply.) **Türkçe:** 5. Mevcut locale dolar kullandığını varsayarsak (\$) ve aşağıdaki yöntem 100_102.2 değerinde bir double değeri ile adlandırılır, aşağıdaki değerlerden hangisi yazdırılır? (Tüm geçerli olanları seçin.)

```
public void print(double t) {
    System.out.print(NumberFormat.getCompactNumberInstance().format(t));
    System.out.print(
```

English: NumberFormat.getCompactNumberInstance(**Türkçe:** NumberFormat.getCompactNumberInstance(

```
Locale.getDefault(), Style.SHORT).format(t));
    System.out.print(NumberFormat.getCurrencyInstance().format(t));
}
```

English: A. 100 **Türkçe:** A. 100 **English:** B. \$100,000.00 **Türkçe:** B. \$100,000.00 **English:** C. 100K **Türkçe:** C. 100K **English:** D. 100 thousand **Türkçe:** D. 100 bin **English:** E. 100M **Türkçe:** E. 100M **English:** F. \$100,102.20 **Türkçe:** F. \$100,102.20 **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri **English:** 6. What is the output of the following code? **Türkçe:** 6. Aşağıdaki kodun çıktısı nedir?

```
LocalDate date = LocalDate.parse("2022-04-30",
    DateTimeFormatter.ISO_LOCAL_DATE_TIME);
System.out.println(date.getYear() + " "
    + date.getMonth() + " " + date.getDayOfMonth());
```

English: A. 2022 APRIL 2 **Türkçe:** A. 2022 APRIL 2 **English:** B. 2022 APRIL 30 **Türkçe:** B. 2022 APRIL 30 **English:** C. 2022 MAY 2 **Türkçe:** C. 2022 MAY 2 **English:** D. The code does not compile. **Türkçe:** D. The kodu derlemez. **English:** E. A runtime exception is thrown. **Türkçe:** E. A runtime exception atılır. **English:** 7. What does the following method print? **Türkçe:** 7. Aşağıdaki yöntem ne yazdırır?

```
public void tryAgain(String s) {
    try (FileReader r = null, p = new FileReader("")) {
        System.out.print("X");
        throw new IllegalArgumentException();
    } catch (Exception s) {
        System.out.print("A");
        throw new FileNotFoundException();
    } finally {
        System.out.print("O");
    }
}
```

English: A. XAO **Türkçe:** A. XAO **English:** B. XOA **Türkçe:** B. XOA **English:** C. One line of this method contains a compiler error. **Türkçe:** Bu yöntemin C. One satırı bir derleyici hatası içerir. **English:** D. Two lines of this method contain compiler errors. **Türkçe:** Bu yöntemin D. Two satırları derleyici hataları içerir. **English:** E. Three or more lines of this method contain compiler errors. **Türkçe:** E. Three veya bu yöntemin daha fazla satırı derleyici hataları içerir. **English:** F. The code compiles, but a NullPointerException is thrown at runtime. **Türkçe:** F. The kodu derlenir, ancak çalışma zamanında bir NullPointerException atılır. **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbirisi

English: 8. Assume that all of the files mentioned in the answer choices exist and define the same keys. **Türkçe:** 8. Cevap seçeneklerinde belirtilen tüm dosyaların var olduğunu ve aynı anahtarları tanımladığını varsayalım. **English:** Which one will be used to find the key in line 8? **Türkçe:** 8. satırdaki anahtarı bulmak için hangisini kullanacaksınız?

```
Locale.setDefault(new Locale("en", "US"));
var b = ResourceBundle.getBundle("Dolphins");
System.out.println(b.getString("name"));
```

English: A. Dolphins.properties **Türkçe:** A. Dolphins.properties **English:** B. Dolphins_US.properties **Türkçe:** B. Dolphins_US.properties **English:** C. Dolphins_en.properties **Türkçe:** C. Dolphins_en.properties **English:** D. Whales.properties **Türkçe:** D. Whales.properties **English:** E. Whales_en_US.properties **Türkçe:** E. Whales_en_US.properties **English:** F. The code does not compile. **Türkçe:** F. The kodu derlemez. **English:** 9. For what value of pattern will the following print <005.21> <008.49> <1,234.0>? **Türkçe:** 9. Aşağıdaki desenin hangi değeri için 005.21> 008.49> 1,234.0> yazdırılacak?

```
String pattern = " ";
var message = DoubleStream.of(5.21, 8.49, 1234)
    .mapToObj(v -> new DecimalFormat(pattern).format(v))
    .collect(Collectors.joining("> <"));
System.out.println("<" + message + ">");
```

English: A. ### **Türkçe:** A. ### **English:** B. 0,000.0# **Türkçe:** B. 0,000.0# **English:** C. #,###.0 **Türkçe:** C. #,###.0 **English:** D. #,###,000.0# **Türkçe:** D. #,###,000.0# **English:** E. The code does not compile regardless of what is placed in the blank. **Türkçe:** E. The kodu, boşluğa ne yerleştirildiğinden bağımsız olarak derlemez. **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri **English:** 10. Which scenario is the best use of an exception? **Türkçe:** 10. Bir istisnanın en iyi kullanımı hangi senaryodur? **English:** A. An element is not found when searching a list. **Türkçe:** A. An öğesi bir list aramasında bulunamadı. **English:** B. An unexpected parameter is passed into a method. **Türkçe:** B. An beklenmeyen parametre bir yönteme aktarılır. **English:** C. The computer caught fire. **Türkçe:** C. Bilgisayar alev aldı. **English:** D. You want to loop through a list. **Türkçe:** D. You bir list üzerinden döngü yapmak istiyor. **English:** E. You don't know how to code a method. **Türkçe:** E. You bir yöntemin nasıl kodlanacağını bilmiyor. **English:** 11. Which of the following exceptions must be handled or declared in the method in which they are thrown? (Choose all that apply.) **Türkçe:** 11. Atıldıkları yöntemde aşağıdaki istisnalardan hangisi ele alınmalı veya ilan edilmelidir? (Tüm geçerli olanları seçin.)

```
class Apple extends RuntimeException {}
class Orange extends Exception {}
class Banana extends Error {}
class Pear extends Apple {}
class Tomato extends Orange {}
class Peach extends Throwable {}
```

English: A. Apple **Türkçe:** A. Elma **English:** B. Orange **Türkçe:** B. Portakal **English:** C. Banana **Türkçe:** C. Muz **English:** D. Pear **Türkçe:** D. Armut **English:** E. Tomato **Türkçe:** E. Domates **English:** F. Peach **Türkçe:** F. Şeftali **English:** 12. Which of the following changes, when made independently, would make this code compile? **Türkçe:** 12. Aşağıdaki değişikliklerden hangisi, bağımsız olarak yapıldığında, bu kodu derleyecektir? **English:** (Choose all that apply.) **Türkçe:** (Tüm geçerli olanları seçin.)

```
import java.io.*;
public class StuckTurkeyCage implements AutoCloseable {
    public void close() throws IOException {
        throw new FileNotFoundException("Cage not closed");
    }
    public static void main(String[] args) {
        try (StuckTurkeyCage t = new StuckTurkeyCage()) {
            System.out.println("put turkeys in");
        }
    }
}
```

English: A.Remove throws IOException from the declaration on line 3. **Türkçe:** A.Remove, 3. satırdaki bildirimden IOException atar. **English:** B.Add throws Exception to the declaration on line 6. **Türkçe:** B.Add 6. satırdaki bildirime Exception atar. **English:** C. Change line 9 to } catch (Exception e) {}. **Türkçe:** C. 9. satırı yakalama (Exception e) olarak değiştir. **English:** D. Change line 9 to } finally {}. **Türkçe:** D. 9. satırı sonunda olarak değiştirin. **English:** E.The code compiles as is. **Türkçe:** E.The kodu olduğu gibi derlenir. **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri **English:** 13.Which of the following are true statements about exception handling in Java? (Choose all that apply.) **Türkçe:** 13. Aşağıdakilerden hangisi Java 'da istisna işleme ile ilgili doğru ifadelerdir? (Tüm geçerli olanları seçin.) **English:** A.A traditional try statement without a catch block requires a finally block. **Türkçe:** A.A catch block olmadan geleneksel deneme ifadesi finally block gerektirir. **English:** B.A traditional try statement without a finally block requires a catch block. **Türkçe:** B.Son olarak bir blok olmadan geleneksel bir deneme ifadesi bir yakalama bloğu gerektirir. **English:** C.A traditional try statement with only one statement can omit the {}. **Türkçe:** C.A geleneksel deneme ifadesi sadece bir ifade ile 'yi atlayabilir. **English:** D.A try-with-resources statement without a catch block requires a finally block. **Türkçe:** catch block olmadan D.A try-with-resources statement bir finally block gerektirir. **English:** E.A try-with-resources statement without a finally block requires a catch block. **Türkçe:** finally block olmadan E.A try-with-resources statement bir catch block gerektirir. **English:** F.A try-with-resources statement with only one statement can omit the {}. **Türkçe:** F.A try-with-resources statement tek bir ifadeyle 'yi atlayabilir.

English: 14.Assuming -g:vars is used when the code is compiled to include debug information, what is the output of the following code snippet? **Türkçe:** 14. Varsayım -g:vars kod hata ayıklama bilgilerini içerecek şekilde derlendiğinde kullanılır, aşağıdaki kod snippet'inin çıktısı nedir?

```
var huey = (String)null;
Integer dewey = null;
Object louie = null;
if(louie == huey.substring(dewey.intValue())) {
    System.out.println("Quack!");
}
```

English: A.A NullPointerException that does not include any variable names in the stack **Türkçe:** A.A NullPointerException bu yığına herhangi bir değişken ad içermez **English:** trace B.A NullPointerException naming huey in the stack trace C.A NullPointerException naming dewey in the stack trace D.A NullPointerException naming louie in the stack trace E.A NullPointerException naming huey and louie in the stack trace F.A NullPointerException naming huey and dewey in the stack trace G. None of the above **Türkçe:** B.A NullPointerException yığın izinde huey adlandırma C.A NullPointerException yığın izinde dewey adlandırma D.A NullPointerException yığın izinde louie adlandırma E.A NullPointerException yığın izinde huey ve louie adlandırma F.A NullPointerException yığın izinde huey ve dewey adlandırma G. Yukarıdakilerin hiçbiri **English:** 15.Which of the following, when inserted independently in the blank, use locale parameters that are properly formatted? (Choose all that apply.) **Türkçe:** 15. Aşağıdakilerden hangisi, boşluğa bağımsız olarak yerleştirildiğinde, uygun şekilde biçimlendirilmiş locale parametrelerini kullanın? (Tüm geçerli olanları seçin.)

```
import java.util.Locale;
public class ReadMap implements AutoCloseable {
    private Locale locale;
    private boolean closed = false;
    @Override public void close() {
        System.out.println("Folding map");
        locale = null;
        closed = true;
    }
    public void open() {
        this.locale =;
    }
    public void use() {
        // Implementation omitted
    }
}
```

English: A. new Locale("xM") **Türkçe:** A. yeni Locale("xM") **English:** B. new Locale("MQ", "ks") **Türkçe:** B. yeni Locale("MQ", "ks") **English:** C. new Locale("qw") **Türkçe:** C. yeni Locale("qw") **English:** D. new Locale("wp", "VW") **Türkçe:** D. yeni Locale("wp", "VW")

English: E. Locale.create("zp") **Türkçe:** E. Locale.create("zp") **English:** F. new Locale.Builder().setLanguage("yw").setRegion("PM") **Türkçe:** F. yeni Locale.Builder().setLanguage("yw").setRegion("PM") **English:** G. The code does not compile regardless of what is placed in the blank. **Türkçe:** G. The kodu, boşluğa ne yerleştirildiğinden bağımsız olarak derlemez. **English:** 16. Which of the following can be inserted into the blank to allow the code to compile and run without throwing an exception? (Choose all that apply.) **Türkçe:** 16. Kodun bir istisna atmadan derlenip çalıştırılmasına izin vermek için aşağıdakilerden hangisi boşluğa eklenebilir? (Tüm geçerli olanları seçin.)

```
var f = DateTimeFormatter.ofPattern("hh o'clock");
System.out.println(f.format(.now()));
```

English: A. ZonedDateTime **Türkçe:** A. ZonedDateTime **English:** B. LocalDate **Türkçe:** B. LocalDate **English:** C. LocalDateTime **Türkçe:** C. YerelDateTime **English:** D. LocalTime **Türkçe:** D. Yerel Zaman **English:** E. The code does not compile regardless of what is placed in the blank. **Türkçe:** E. The kodu, boşluğa ne yerleştirildiğinden bağımsız olarak derlemez. **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri **English:** 17. Which of the following statements about resource bundles are correct? (Choose all that apply.) **Türkçe:** 17. resource bundles ile ilgili aşağıdaki ifadelerden hangisi doğrudur? (Tüm geçerli olanları seçin.) **English:** A. All keys must be in the same resource bundle to be used. **Türkçe:** A. All tuşları kullanılacak aynı resource bundle içinde olmalıdır. **English:** B. A resource bundle is loaded by calling the new ResourceBundle() constructor. **Türkçe:** B. A resource bundle yeni ResourceBundle() constructor'u çağırarak yüklenir. **English:** C. Resource bundle values are always read using the Properties class. **Türkçe:** C. Resource demet değerleri her zaman Properties sınıfı kullanılarak okunur. **English:** D. Changing the default locale lasts for only a single run of the program. **Türkçe:** D. Changing default locale programı sadece tek bir çalışma için sürer. **English:** E. If a resource bundle for a specific locale is requested, then the resource bundle for the **Türkçe:** Belirli bir locale için E. If a resource bundle istenir, daha sonra resource bundle için **English:** default locale will not be used. **Türkçe:** default locale kullanılmayacaktır. **English:** F. It is possible to use a resource bundle for a locale without specifying a default locale. **Türkçe:** F. It bir default locale belirtmeden resource bundle için locale kullanmak mümkündür. **English:** 18. What is the output of the following code? **Türkçe:** 18. Aşağıdaki kodun çıktısı nedir?

```
import java.io.*;
public class FamilyCar {
    static class Door implements AutoCloseable {
        public void close() {
            System.out.print("D");
        }
    }
    static class Window implements Closeable {
        public void close() {
            System.out.print("W");
            throw new RuntimeException();
        }
    }
    public static void main(String[] args) {
        var d = new Door();
        try (d; var w = new Window()) {
```

```
            System.out.print("T");
        } catch (Exception e) {
            System.out.print("E");
        } finally {
            System.out.print("F");
        } } }
```

English: A. TWF **Türkçe:** A. TWF **English:** B. TWDF **Türkçe:** B. TWDF **English:** C. TWDEF **Türkçe:** C. TWDEF **English:** D. TWF followed by an exception **Türkçe:** D. TWF ardından bir istisna **English:** E. TWDF followed by an exception **Türkçe:** E. TWDF ardından bir istisna **English:** F. TWEF followed by an exception **Türkçe:** F. TWEF ardından bir istisna **English:** G. The code does not compile. **Türkçe:** G. The kodu derlemez. **English:** 19. Suppose that we have the following three properties files and code. Which bundles are used on lines 8 and 9, respectively? **Türkçe:** 19. Aşağıdaki üç properties files ve koda sahip olduğumuzu varsayalım. 8 ve 9 satırlarında sırasıyla hangi demetler kullanılır? **English:** Dolphins.properties name=The Dolphin age=0 Dolphins_en.properties name=Dolly age=4 Dolphins_fr.properties name=Dolly **Türkçe:** Dolphins.properties name=The Dolphin age=0 Dolphins_en.properties name=Dolly age=4 Dolphins_fr.properties name=Dolly

```
var fr = new Locale("fr");
Locale.setDefault(new Locale("en", "US"));
var b = ResourceBundle.getBundle("Dolphins", fr);
b.getString("name");
b.getString("age");
```

English: A. Dolphins.properties and Dolphins.properties **Türkçe:** A. Dolphins.properties ve Dolphins.properties **English:** B. Dolphins.properties and Dolphins_en.properties **Türkçe:** B. Dolphins.properties ve Dolphins_en.properties **English:** C. Dolphins_en.properties and Dolphins_en.properties **Türkçe:** C. Dolphins_en.properties ve Dolphins_en.properties **English:** D. Dolphins_fr.properties and Dolphins.properties **Türkçe:** D. Dolphins_fr.properties ve Dolphins.properties **English:** E. Dolphins_fr.properties and Dolphins_en.properties **Türkçe:** E. Dolphins_fr.properties ve Dolphins_en.properties **English:** F. The code does not compile. **Türkçe:** F. The kodu derlemez. **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbirisi

English: 20. What is printed by the following program? **Türkçe:** 20. Aşağıdaki program tarafından ne basılır?

```

public class DriveBus {
    public void go() {
        System.out.print("A");
        try {
            stop();
        } catch (ArithmeticException e) {
            System.out.print("B");
        } finally {
            System.out.print("C");
        }
        System.out.print("D");
    }
    public void stop() {
        System.out.print("E");
        Object x = null;
        x.toString();
        System.out.print("F");
    }
    public static void main(String n[]) {
        new DriveBus().go();
    } }

```

English: A. AE **Türkçe:** A. AE **English:** B. AEBCD **Türkçe:** B. AEBCD **English:** C. AEC **Türkçe:** C. AEC **English:** D. AECD **Türkçe:** D. AECD **English:** E. AE followed by a stack trace **Türkçe:** E. AE ardından bir yığın izi **English:** F. AEBCD followed by a stack trace **Türkçe:** F. AEBCD ardından bir yığın izi **English:** G. AEC followed by a stack trace **Türkçe:** G. AEC ardından bir yığın izi **English:** H. A stack trace with no other output **Türkçe:** H. A başka bir çıkış olmadan yığın izi **English:** 21. Which changes, when made independently, allow the following program to compile? (Choose all that apply.) **Türkçe:** 21. Hangi değişiklikler, bağımsız olarak yapıldığında, aşağıdaki programın derlenmesine izin verir? (Tüm geçerli olanları seçin.)

```

public class AhChoo {
    static class SneezeException extends Exception {}
    static class SniffleException extends SneezeException {}
    public static void main(String[] args) {
        try {
            throw new SneezeException();

```

```

        } catch (SneezeException | SniffleException e) {
        } finally {}
    } }

```

English: A. Add throws SneezeException to the declaration on line 4. **Türkçe:** A. Add, 4. satırdaki bildirime SneezeException atar. **English:** B. Add throws Throwable to the declaration on line 4. **Türkçe:** B. Add, 4. satırdaki bildirime Throwable atar. **English:** C. Change line 7 to } catch (SneezeException e) {. **Türkçe:** C. 7. satırı yakalama (SneezeException e) olarak değiştir. **English:** D. Change line 7 to } catch (SniffleException e) {. **Türkçe:** D. 7. satırı yakalama (SniffleException e) olarak değiştir. **English:** E. Remove line 7. **Türkçe:** E. 7. satırı kaldır. **English:** F. The code compiles correctly as is. **Türkçe:** F. The kodu olduğu gibi doğru bir şekilde derlenir. **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbirisi **English:** 22. What is the output of the following code? **Türkçe:** 22. Aşağıdaki kodun çıktısı nedir?

```
try {
    LocalDateTime book = LocalDateTime.of(2022, 4, 5, 12, 30, 20);
    System.out.print(book.format(DateTimeFormatter.ofPattern("m")));
    System.out.print(book.format(DateTimeFormatter.ofPattern("z")));
    System.out.print(DateTimeFormatter.ofPattern("y").format(book));
} catch (Throwable e) {}
```

English: A. 4 **Türkçe:** A. 4 **English:** B. 30 **Türkçe:** B. 30 **English:** C. 402 **Türkçe:** C. 402 **English:** D. 3002 **Türkçe:** D. 3002 **English:** E. 3002022 **Türkçe:** E. 3002022 **English:** F. 402022 **Türkçe:** F. 402022 **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri **English:** 23. Fill in the blank: A class that implements _____ may be in a try-with-resources statement. (Choose all that apply.) **Türkçe:** 23. Boşlukta Doldur: try-with-resources statement içinde _____ uygulayan bir sınıf olabilir. (Tüm geçerli olanları seçin.) **English:** A. AutoCloseable **Türkçe:** A. AutoCloseable **English:** B. Resource **Türkçe:** B. Kaynak **English:** C. Exception **Türkçe:** C. Exception **English:** D. AutomaticResource **Türkçe:** D. OtomatikKaynak **English:** E. Closeable **Türkçe:** E. Kapatılabilir **English:** F. RuntimeException **Türkçe:** F. RuntimeException **English:** G. Serializable **Türkçe:** G. Serileştirilebilir

English: 24. What is the output of the following program? **Türkçe:** 24. Aşağıdaki programın çıktısı nedir?

```
public class SnowStorm {
    static class WalkToSchool implements AutoCloseable {
        public void close() {
            throw new RuntimeException("flurry");
        }
    }
    public static void main(String[] args) {
        WalkToSchool walk1 = new WalkToSchool();
        try (walk1; WalkToSchool walk2 = new WalkToSchool()) {
            throw new RuntimeException("blizzard");
        } catch (Exception e) {
            System.out.println(e.getMessage()
                + " " + e.getSuppressed().length);
        }
        walk1 = null;
    }
}
```

English: A. blizzard 0 **Türkçe:** A. kar fırtınası 0 **English:** B. blizzard 1 **Türkçe:** B. kar fırtınası 1 **English:** C. blizzard 2 **Türkçe:** C. kar fırtınası 2 **English:** D. flurry 0 **Türkçe:** D. flury 0 **English:** E. flurry 1 **Türkçe:** E. flury 1 **English:** F. flurry 2 **Türkçe:** F. Flury 2 **English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri **English:** 25. Assuming U.S. currency is in dollars (\$) and German currency is in euros (€), what is the output of the following program? **Türkçe:** 25. U.S. para biriminin dolar (\$) ve Alman para biriminin avro (€) olduğunu varsayarsak, aşağıdaki programın çıktısı nedir?

```
import java.text.NumberFormat;
import java.util.Locale;
import java.util.Locale.Category;
public record Wallet(double money) {
    private String openWallet() {
```

English: Locale.setDefault(Category.DISPLAY, **Türkçe:** Locale.setDefault(Category.DISPLAY,

```
new Locale.Builder().setRegion("us").build());
```

English: Locale.setDefault(Category.FORMAT, **Türkçe:** Locale.setDefault(Category.FORMAT,

```
new Locale.Builder().setLanguage("en").build());
return NumberFormat.getCurrencyInstance(Locale.GERMANY)
    .format(money);
}
public void printBalance() {
    System.out.println(openWallet());
}
```

```
public static void main(String... unused) {
    new Wallet(2.4).printBalance();
} }
```

English: A. 2,40 € **Türkçe:** A. 2,40 **English:** B. \$2.40 **Türkçe:** B. \$2.40 **English:** C. 2.4 **Türkçe:** C. 2.4
English: D.The code does not compile. **Türkçe:** D.The kodu derlemez. **English:** E. None of the above
Türkçe: E. Yukarıdakilerin hiçbirisi **English:** 26.Which lines can fill in the blank to make the following code
 compile? (Choose all that apply.) **Türkçe:** 26. Aşağıdaki kodu derlemek için boşluğu hangi satırlar
 doldurabilir? (Tüm geçerli olanları seçin.)

```
void rollOut() throws ClassCastException {}
public void transform(String c) {
    try {
        rollOut();
    } catch (IllegalArgumentException |
    ) {
    }
}
```

English: A. IOException a **Türkçe:** A. IOException a **English:** B. Error b **Türkçe:** B. Error b **English:** C.
 NullPointerException c **Türkçe:** C. NullPointerException c **English:** D. RuntimeException d **Türkçe:** D.
 RuntimeException d **English:** E. NumberFormatException e **Türkçe:** E. NumberFormatException e
English: F. ClassCastException f **Türkçe:** F. ClassCastException f **English:** G.None of the above. The
 code contains a compiler error regardless of what is inserted **Türkçe:** Yukarıdaki G.None. Kod,
 eklenenden bağımsız olarak bir derleyici hatası içerir **English:** into the blank. **Türkçe:** Boşluğa doğru.

Appendix · Official Review Question Answers / Resmî Cevaplar

Aşağıdaki cevaplar kaynak Appendix bölümündeki sıra ve gerekçeleri korur. Türkçe bloklar doğal teknik çeviridir.

Official Answer 1

English: 1.A, C, D, E. A method that declares an exception isn't required to throw one, making option A correct. Unchecked exceptions can be thrown in any method, making options C and E correct. Option D matches the exception type declared, so it's also correct. Option B is incorrect because a broader exception is not allowed. **Türkçe:** 1.A, C, D, E. Bir istisnayı beyan eden bir yöntem, bir tanesini atmak için gerekli değildir, A seçeneğini doğru hale getirir. Unchecked exceptions herhangi bir yöntemde atılabilir, C ve E seçeneklerini doğru hale getirir. D seçeneği, belirtilen istisna türüyle eşleşir, bu nedenle de doğrudur. B seçeneği incorrect'tir, çünkü daha geniş bir istisnaya izin verilmez.

Official Answer 2

English: 2.F. The code does not compile because the throw and throws keywords are incorrectly used on lines 6, 7, and 9. If the keywords were fixed, the rest of the code would compile and print a stack trace with YesProblem at runtime. For this reason, option F is correct. **Türkçe:** 2.F. Kod derlemez, çünkü atma ve atma anahtar kelimeleri 6, 7 ve 9 satırlarında yanlış kullanılır. Anahtar kelimeler sabit olsaydı, kodun geri kalanı, çalışma zamanında YesProblem ile bir yığın izi derleyip yazdırırdı. Bu nedenle F seçeneği doğrudur.

Official Answer 3

English: 3.A, D, E. Localization refers to user-facing elements. Dates, currency, and numbers are commonly used in different formats for different countries, making options A, D, and E correct. **Türkçe:** 3.A, D, E. Yerelleştirme, kullanıcıya bakan unsurları ifade eder. Tarihler, para birimi ve sayılar, farklı ülkeler için farklı formatlarda commonly olarak kullanılır ve A, D ve E seçeneklerini doğru hale getirir.

English: Class and variable names, along with lambda expressions, are internal to the application, so there is no need to translate them for users. **Türkçe:** Sınıf ve değişken adları, lambda ifadeleri ile birlikte, uygulamanın içinde yer alır, bu nedenle kullanıcılar için çevirmeye gerek yoktur.

Official Answer 4

English: 4.E. The order of catch blocks is important because they're checked in the order they appear after the try block. Because ArithmeticException is a child class of RuntimeException, the catch block on line 7 is unreachable (if an ArithmeticException is thrown in the try block, it will be caught on line 5). Line 7 generates a compiler error because it is unreachable code, making option E correct. **Türkçe:** 4.E. catch blocks sırası önemlidir, çünkü try block'ten sonra göründükleri sırada kontrol edilirler. ArithmeticException RuntimeException'in bir çocuk sınıfı olduğu için, catch block 7 satırına erişilemez (bir ArithmeticException try block satırına atılırsa, 5. hatta yakalanır). Line 7, erişilemez bir kod olduğu için bir derleyici hatası oluşturur ve E seçeneğini doğru hale getirir.

Official Answer 5

English: 5.C, F. The code compiles and runs without issue. When a `CompactNumberFormat` instance is requested without a style, it uses the `SHORT` style by default. This results in both of the first two statements printing 100K, making option C correct. If the `LONG` style were used, then 100 thousand would be printed. Option F is also correct, as the full value is printed with a currency formatter. **Türkçe:** 5.C, F. Kod sorunsuz bir şekilde derlenir ve çalışır. Bir `CompactNumberFormat` örneği bir stil olmadan istendiğinde, varsayılan olarak `SHORT` stilini kullanır. Bu, 100K yazdıran ilk iki ifadenin her ikisi ile sonuçlanır ve C seçeneğini doğru hale getirir. `LONG` stili kullanılırsa, 100 bin adet basılırdı. F seçeneği de doğrudur, çünkü tam değer bir para birimi biçimlendirici ile basılır.

Official Answer 6

English: 6.E. A `LocalDate` does not have a time element. Therefore, a date/time formatter is not appropriate. The code compiles but throws an exception at runtime, making option E correct. **Türkçe:** 6.E. Bir `LocalDate` zaman öğesine sahip değildir. Bu nedenle, bir date/time formatter uygun değildir. Kod derlenir, ancak çalışma zamanında bir istisna atar ve E seçeneğini doğru hale getirir. **English:** If `ISO_LOCAL_DATE` were used, the code would print 2022 APRIL 30. **Türkçe:** `ISO_LOCAL_DATE` kullanılırsa, kod 2022 APRIL 30 yazdırır.

Official Answer 7

English: 7.E. The first compiler error is on line 12 because each resource in a try-with-resources state-ment must have its own data type and be separated by a semicolon (;). Line 15 does not compile because the variable `s` is already declared in the method. Line 17 also does not compile. The `FileNotFoundException`, which inherits from `IOException` and `Exception`, is a checked exception, so it must be handled in a try/catch block or declared by the method. Because these three lines of code do not compile, option E is the correct answer. **Türkçe:** 7.E. İlk derleyici hatası satır 12'dedir, çünkü bir try-with-resources devlet-mentindeki her kaynak kendi veri türüne sahip olmalı ve bir virgül (;) ile ayrılmalıdır. 15. satır derlenmez çünkü `s` değişkeni yöntemde zaten beyan edilmiştir. 17 numaralı hat da uyumlu değil. `IOException` ve `Exception`'den miras kalan `FileNotFoundException` bir checked exception'dir, bu nedenle bir denemede/catch block ele alınmalıdır veya yöntem tarafından ilan edilmelidir. Bu üç kod satırı derlenmediğinden, E seçeneği doğru cevaptır.

Official Answer 8

English: 8.C. Java will first look for the most specific matches it can find, starting with `Dolphins_en_US.properties`. Since that is not an answer choice, it drops the country and looks for `Dolphins_en.properties`, making option C correct. Option B is incorrect because a country without a language is not a valid locale. **Türkçe:** 8.C. Java ilk önce `Dolphins_en_US.properties` ile başlayarak bulabileceği en özel eşleşmeleri arayacaktır. Bu bir cevap seçimi olmadığından, ülkeyi düşürür ve C seçeneğini doğru hale getirerek `Dolphins_en.properties` arar. B seçeneği yanlıştır, çünkü dili olmayan bir ülke geçerli bir locale değildir.

Official Answer 9

English: 9.D. When working with a custom number formatter, the 0 symbol displays the digit as 0, even if it's not present, while the # symbol omits the digit from the start or end of the String if it is not present. Based on the requested output, a String that displays at least three digits before the decimal (including a comma) and at least one after the decimal is required. It should display a second digit after the decimal if one is available. For this reason, option D is the correct answer. **Türkçe:** 9.D. Özel bir sayı biçimlendirici ile çalışırken, 0 sembolü, mevcut olmasa bile, basamak 0 olarak görüntülenirken, # sembolü, eğer mevcut değilse, String'un başlangıcından veya sonundan itibaren basamakları atlar. İstenen çıktıya dayanarak, ondalıktan önce (virgül dahil) en az üç basamak gösteren ve ondalık gereklilikten sonra en az bir basamak gösteren bir String. Eğer varsa ondalıktan sonra ikinci bir rakam göstermelidir. Bu nedenle, D seçeneği doğru cevaptır.

Official Answer 10

English: 10.B. An IllegalArgumentException is used when an unexpected parameter is passed into a method, making option B correct. Option A is incorrect because returning null or ---- 1 is a common return value for searching for data. Option D is incorrect because a for loop is typ-ically used for this scenario. Option E is incorrect because you should find out how to code the method and not leave it for the unsuspecting programmer who calls your method. Option C is incorrect because you should run! **Türkçe:** 10.B. Beklenmedik bir parametre bir yömeme aktarıldığında, B seçeneğini doğru hale getiren bir IllegalArgumentException kullanılır. A seçeneği yanlıştır, çünkü geri dönen null veya ---- 1, veri aramak için ortak bir getiri değeridir. D seçeneği yanlıştır, çünkü bir for loop bu senaryo için tip olarak kullanılır. E seçeneği yanlıştır, çünkü yöntemi nasıl kodlayacağınızı öğrenmeli ve yönteminizi çağıran şüpheli programcıya bırakmamalısınız. C seçeneği yanlıştır, çünkü koşmanız gerekir!

Official Answer 11

English: 11.B, E, F. An exception that must be handled or declared is a checked exception. A checked exception inherits Exception but not RuntimeException. The entire hierarchy counts, so options B and E are both correct. Option F is also correct, as a class that inherits Throwable but not RuntimeException or Error is also checked. **Türkçe:** 11.B, E, F. Halledilmesi veya ilan edilmesi gereken bir istisna checked exception'dur. Bir checked exception varis Exception ama RuntimeException değil. Tüm hiyerarşi sayılır, bu yüzden B ve E seçeneklerinin ikisi de doğrudur. F seçeneği de doğrudur, Throwable varisli olan ancak RuntimeException veya Error olmayan bir sınıf olarak da kontrol edilir.

Official Answer 12

English: 12.B, C. The code does not compile as is because the exception declared by the close() **Türkçe:** 12.B, C. Kod, close() tarafından ilan edilen istisna nedeniyle olduğu gibi derlemez. **English:** method must be handled or declared. Option A is incorrect because removing the exception from the declaration causes a compilation error on line 4, as FileNotFoundException is a checked exception that must be handled or declared. Option B is correct because the unhandled exception within the main() method becomes declared. Option C is also correct because the exception becomes handled. Option D is incorrect because the exception remains unhandled. **Türkçe:** yöntem ele alınmalı veya ilan edilmelidir. A seçeneği yanlıştır, çünkü istisnayı deklarasyondan kaldırmak, FileNotFoundException ele alınması veya ilan edilmesi gereken bir checked exception olduğu için, 4. satırda bir derleme hatasına neden olur. B seçeneği doğrudur, çünkü main() yöntemindeki işlenmemiş istisna ilan edilir. C seçeneği de doğrudur, çünkü istisna ele alınır. D seçeneği yanlıştır, çünkü istisna kaldırılmamıştır.

Official Answer 13

English: 13.A, B. A try-with-resources statement does not require a catch or finally block. A traditional try statement requires at least one of the two. Neither statement can be written without a body encased in braces, {}. For these reasons, options A and B are correct. **Türkçe:** 13.A, B. Bir try-with-resources statement yakalama veya finally block gerektirmez. Traditional bir deneme ifadesi en az ikisinden birini gerektirir. Her iki ifade de tellerle kaplanmış bir gövde olmadan yazılamaz,. Bu nedenlerden dolayı A ve B seçenekleri doğrudur.

Official Answer 14

English: 14.C. Starting with Java 15, NullPointerException stack traces include the name of the variable that is null by default, making option A incorrect. The first NullPointerException encountered at runtime is when dewey.intValue() **Türkçe:** 14.C. Java 15 ile başlayarak, NullPointerException yığın izleri, varsayılan olarak null olan değişkenin adını içerir ve A seçeneğini yanlış yapar. Çalışma zamanında karşılaşılan ilk NullPointerException, dewey.intValue() **English:** is called, making option C correct. Options E and F are incorrect as only one NullPointerException exception can be thrown at a time. **Türkçe:** C seçeneğinin doğru olması için çağrılır. E ve F seçenekleri bir seferde sadece bir NullPointerException istisnası atılabileceği için yanlıştır.

Official Answer 15

English: 15.C, D. The code compiles with the appropriate input, so option G is incorrect. A locale consists of a required lowercase language code and optional uppercase country code. In the Locale() constructor, the language code is provided first. For these reasons, options C and D are correct. Option E is incorrect because a Locale is created using a constructor or Locale.Builder class. Option F is really close but is missing build() at the end. **Türkçe:** 15.C, D. Kod uygun girdi ile derlenir, bu nedenle G seçeneği yanlıştır. Gerekli bir küçük harf dil kodunun ve optional büyük harf ülke kodunun bir locale con-sist'i. Locale() constructor'da dil kodu ilk olarak sağlanır. Bu nedenlerden dolayı, C ve D seçenekleri doğrudur. E seçeneği yanlıştır, çünkü bir Locale constructor veya Locale.Builder sınıfı kullanılarak oluşturulur. Option F gerçekten yakın ama sonunda build() eksik. **English:** Without that, option F does not compile. **Türkçe:** Bu olmadan, F seçeneği derlemez.

Official Answer 16

English: 16.F. The code compiles, but the first line produces a runtime exception regardless of what is inserted into the blank, making option F correct. When creating a custom formatter, any nonsymbol code must be properly escaped using pairs of single quotes (''). In this case, it fails because o is not a symbol. Even if you didn't know o wasn't a symbol, the code contains an unmatched single quote. If the properly escaped value of "hh' o'clock" were used, then the correct answers would be ZonedDateTime, LocalDateTime, and LocalTime. **Türkçe:** 16.F. Kod derlenir, ancak ilk satır boşluğa ne eklendiğinden bağımsız olarak bir runtime exception üretir ve F seçeneğini doğru hale getirir. Özel bir formatlayıcı oluştururken, herhangi bir nonsembol kodu, tek tırnak çiftleri (') kullanılarak düzgün bir şekilde kaçınılmalıdır. Bu durumda başarısız olur çünkü o bir sembol değildir. O'nun bir sembol olmadığını bilmeseniz bile, kod benzersiz bir tek alıntı içerir. Eğer "h' o'clock"un düzgün bir şekilde kaçmış değeri kullanılmışsa, o zaman doğru cevaplar ZonedDateTime, LocalDateTime ve LocalTime olacaktır. **English:** Option B would not be correct because LocalDate values do not have an hour part. **Türkçe:** B seçeneği doğru olmaz çünkü LocalDate değerlerinin bir saatlik kısmı yoktur.

Official Answer 17

English: 17.D, F. Option A is incorrect because Java will look at parent bundles if a key is not found in a specified resource bundle. Option B is incorrect because resource bundles are loaded from static factory methods. Option C is incorrect, as resource bundle values are read from the ResourceBundle object directly. Option D is correct because the locale is changed only in memory. Option E is incorrect, as the resource bundle for the default locale may be used if there is no resource bundle for the specified locale (or its locale without a country code). **Türkçe:** 17.D, F. A seçeneği yanlıştır, çünkü Java bir anahtar belirtilen resource bundle içinde bulunmazsa ana paketlere bakacaktır. B seçeneği yanlış çünkü resource bundles statik factory methods yüklü. C seçeneği yanlıştır, çünkü resource bundle değerleri doğrudan ResourceBundle nesnesinden okunur. D seçeneği doğrudur, çünkü locale yalnızca bellekte değiştirilir. E seçeneği yanlıştır, çünkü default locale için resource bundle, belirtilen locale için resource bundle yoksa (veya ülke kodu olmayan locale) kullanılabilir. **English:** Finally, option F is correct. The JVM will set a default locale automatically. **Türkçe:** Son olarak, F seçeneği doğrudur. JVM otomatik olarak set a default locale olacaktır.

Official Answer 18

English: 18.C. After both resources are declared and created in the try-with-resources statement, T is printed as part of the body. Then the try-with-resources completes and closes the resources in the reverse of the order in which they were declared. After W is printed, an exception is thrown. However, the remaining resource still needs to be closed, so D is printed. Once all the resources are closed, the exception is thrown and swallowed in the catch block, causing E to be printed. Last, the finally block is run, printing F. Therefore, the answer is TWDEF and option C is correct. **Türkçe:** 18.C. Her iki kaynak da try-with-resources statement olarak ilan edildikten ve oluşturulduktan sonra, T vücudun bir parçası olarak basılır. Daha sonra try-with-resources, ilan edildikleri sıranın tersine kaynakları tamamlar ve kapatır. W basıldıktan sonra bir istisna atılır. Ancak, kalan kaynağın hala kapatılması gerekir, bu nedenle D yazdırılır. Tüm kaynaklar kapatıldıktan sonra, istisna catch block içinde atılır ve yutulur, bu da E'nin basılmasına neden olur. Son olarak, finally block çalıştırılır, F yazdırılır. Bu nedenle, cevap TWDEF ve C seçeneği doğrudur.

Official Answer 19

English: 19.D. Java will use Dolphins_fr.properties as the matching resource bundle on line 7 because it is an exact match on the language of the requested locale. Line 8 finds a match-ing key in this file. Line 9 does not find a match in that file; therefore, it has to look higher up in the hierarchy. Once a bundle is chosen, only resources in that hierarchy are allowed. It cannot use the default locale anymore, but it can use the default resource bundle specified by Dolphins.properties. For these reasons, option D is correct.

Türkçe: 19.D. Java Dolphins_fr.properties'u 7. satırdaki resource bundle ile eşleştirecektir, çünkü istenen locale diliyle birebir eşleşir. Satır 8, bu dosyada bir eşleme anahtarı bulur. Satır 9, o dosyada bir eşleşme bulamaz; bu nedenle, hiyerarşide daha yukarı bakması gerekir. Bir paket seçildikten sonra, yalnızca bu hiyerarşideki kaynaklara izin verilir. default locale artık kullanamaz, ancak Dolphins.properties tarafından belirtilen varsayılan resource bundle kullanabilir. Bu nedenlerden dolayı, D seçeneği doğrudur.

Official Answer 20

English: 20.G. The main() method invokes go(), and A is printed on line 3. The stop() method is invoked, and E is printed on line 14. Line 16 throws a NullPointerException, so stop() immediately ends, and line 17 doesn't execute. The exception isn't caught in go(), **Türkçe:** 20.G. main() yöntemi go()'yi çağırır ve A, 3. satırda basılır. stop() yöntemi çağırılır ve E, 14. satırda basılır. Satır 16 bir NullPointerException atar, bu nedenle stop() hemen sona erer ve satır 17 çalışmaz. İstisna go() içinde yakalanmaz,

English: so the go() method ends as well, but not before its finally block executes and C is printed on line 9. Because main() doesn't catch the exception, the stack trace displays, and no further output occurs. For these reasons, AEC is printed followed by a stack trace for a NullPointerException, making option G correct. **Türkçe:** Böylece go() yöntemi de sona erer, ancak finally block çalıştırılmadan önce ve C 9. satırda basılmaz. main() istisnayı yakalamadığı için, yığın izi görüntülenir ve başka bir çıktı oluşmaz. Bu nedenlerden dolayı, AEC bir NullPointerException için bir yığın izi ile basılır ve G seçeneğini doğru hale getirir.

Official Answer 21

English: 21.C. The code does not compile because the multi-catch block on line 7 cannot catch both a superclass and a related subclass. Options A and B do not address this problem, so they are incorrect. Since the try body throws SneezeException, it can be caught in a catch block, making option C correct. Option D allows the catch block to compile but causes a compiler error on line 6. Both of the custom exceptions are checked and must be handled or declared in the main() method. A SneezeException is not a SniffleException, so the exception is not handled. Likewise, option E leads to an unhandled exception compiler error on line 6. **Türkçe:** 21.C. Kod derlemez, çünkü 7. satırdaki multi-catch block hem bir süper sınıfı hem de ilgili bir alt sınıfı yakalayamaz. A ve B seçenekleri bu sorunu çözmez, bu yüzden yanlıştır. Deneme gövdesi SneezeException attığından, bir catch block içine yakalanabilir ve C seçeneğini doğru hale getirir. Option D, catch block'nin derlenmesine izin verir, ancak 6. satırda bir derleyici hatasına neden olur. Özel istisnaların her ikisi de kontrol edilir ve main() yönteminde ele alınmalı veya ilan edilmelidir. SneezeException bir SniffleException değildir, bu nedenle istisna ele alınmaz. Aynı şekilde, E seçeneği 6. satırda işlenmemiş bir istisna derleyici hatasına yol açar.

Official Answer 22

English: 22.B. For this question, the date used is April 5, 2022 at 12:30:20pm. The code compiles, and either form of the formatter is correct: `dateTime.format(formatter)` or `formatter.format(dateTime)`. The custom format `m` returns the minute, so 30 is output first. The next line throws an exception as `z` relates to time zone, and date/time does not have a zone component. This exception is then swallowed by the try/catch block. Since this is the only value printed, option B is correct. If the code had not thrown an exception, the last line would have printed 2022. **Türkçe:** 22.B. Bu soru için kullanılan tarih 5 Nisan 2022, saat 12:30:20. Kod derlenir ve biçimlendiricinin her iki biçimi de doğrudur: `dateTime.format(formatter)` veya `formatter.format(dateTime)`. Özel format `m` dakikayı döndürür, yani 30 önce çıktır. Bir sonraki satır, `z`'nin zaman dilimiyle ilgili olarak bir istisna atar ve tarih / saatin bir bölge bileşeni yoktur. Bu istisna daha sonra try/catch block tarafından yutulur. Basılan tek değer bu olduğundan, B seçeneği doğrudur. Kod bir istisna atmasaydı, son satır 2022'yi basacaktı.

Official Answer 23

English: 23.A, E. Resources must inherit `AutoCloseable` to be used in a try-with-resources block. **Türkçe:** 23.A, E. Kaynaklar, bir try-with-resources bloğunda kullanılmak üzere `AutoCloseable` miras almalıdır. **English:** Since `Closeable`, which is used for I/O classes, extends `AutoCloseable`, both may be used, making options A and E correct. **Türkçe:** I / O sınıfları için kullanılan `Closeable`, `AutoCloseable` uzandığından, her ikisi de kullanılabilir ve A ve E seçeneklerini doğru hale getirir.

Official Answer 24

English: 24.G. The code does not compile because the resource `walk1` is not final or effectively final and cannot be used in the declaration of a try-with-resources statement. For this reason, option G is correct. If the line that set `walk1` to null were removed, then the code would compile and print blizzard 2 at runtime, with the exception inside the try block being the primary exception since it is thrown first. Then two suppressed exceptions would be added to it when trying to close the `AutoCloseable` resources. **Türkçe:** 24.G. Kod derlemez çünkü kaynak `walk1` son veya effectively final değildir ve try-with-resources statement bildiriminde kullanılamaz. Bu nedenle, G seçeneği doğrudur. `set walk1` ile `null` arasındaki çizgi kaldırılmışsa, kod try block içindeki istisna ilk atıldığından beri primary exception olmak üzere çalışma zamanında kar fırtınası 2'yi derler ve yazdırır. Ardından, `AutoCloseable` kaynaklarını kapatmaya çalışırken ona iki suppressed exceptions eklenir.

Official Answer 25

English: 25.A. The code compiles and prints the value for Germany, 2,40 €, making option A the correct answer. Note that the default locale category is ignored since an explicit currency locale is selected. **Türkçe:** 25.A. Kod, Almanya için değeri 2,40 derler ve yazdırır, bu da A seçeneğini doğru cevap yapar. Açık bir para birimi locale seçildiğinden, varsayılan locale category görmezden gelindiğini unutmayın.

Official Answer 26

English: 26.B, F. The try block is not capable of throwing an IOException, making the catch block unreachable code and option A incorrect. Options B and F are correct, as both are unchecked exceptions that do not extend or inherit from IllegalArgumentException. Remember, it is not a good idea to catch Error in practice, although because it is possible, it may come up on the exam. Option C is incorrect because the variable c is declared already in the method declaration. Option D is incorrect because the IllegalArgumentException inherits from RuntimeException, making the first declaration unnecessary.

Türkçe: 26.B, F. try block bir IOException atma yeteneğine sahip değildir, bu da catch block erişilemez kod ve Option A'yı yanlış yapar. B ve F seçenekleri doğrudur, çünkü her ikisi de IllegalArgumentException 'den genişlemeyen veya miras almayan unchecked exceptions'tir. Unutmayın, pratikte Error yakalamak iyi bir fikir değildir, ancak mümkün olduğu için sınavda ortaya çıkabilir. C seçeneği yanlıştır, çünkü c değişkeni yöntem deklarasyonunda zaten beyan edilmiştir. D seçeneği yanlıştır, çünkü IllegalArgumentException RuntimeException'den miras alır ve ilk bildirimi gereksiz kılar.

English: Similarly, option E is incorrect because NumberFormatException inherits from IllegalArgumentException, making the second declaration unnecessary. Since options B and F are correct, option G is incorrect. **Türkçe:** Benzer şekilde, E seçeneği yanlıştır, çünkü NumberFormatException IllegalArgumentException'den miras alır ve ikinci bildirimi gereksiz kılar. B ve F seçenekleri doğru olduğundan, G seçeneği yanlıştır.

Coverage ledger

- Chapter body marker'ları: 591–660
- Appendix answer marker'ları: 945–948
- Resmî cevap hedefi: 1–26
- Kod blokları özgün dilinde tutulmuştur.
- Çeviri ayrıntıları ünite vocabulary ve grammar kaynaklarıyla desteklenir.